CSET340

Advanced Computer Vision and Video Analytics

2nd Feb. to 06th Feb. 2026

Overall Course Coordinator-
Dr. Gaurav Kumar Dashondhi
Gaurav.dashondhi@bennett.edu.in

Note : Any query related to course then first connect with overall course coordinator.

Fourier Transform

Sampling theory and aliasing

Fourier Series

Fourier Transform

Convolution theorem

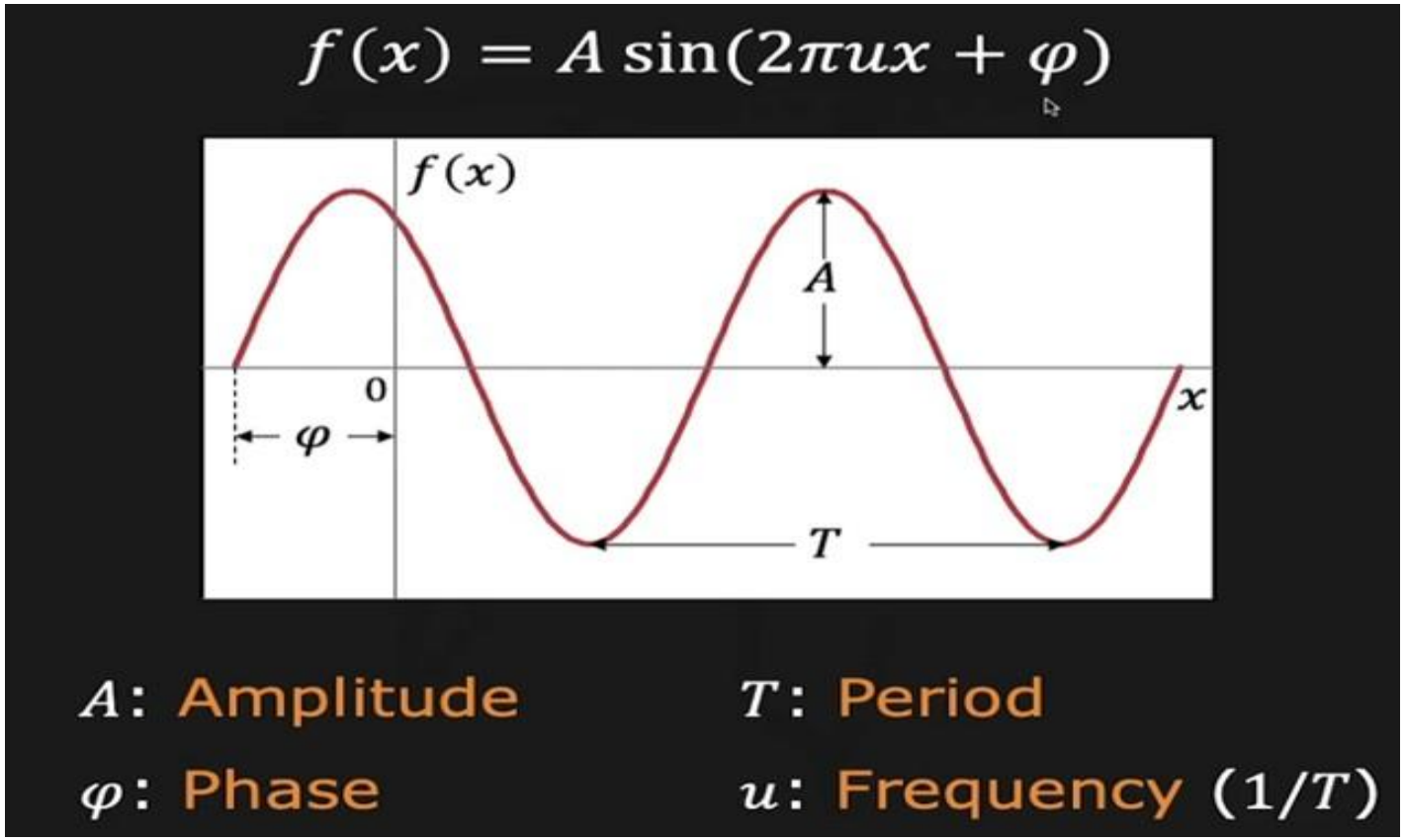
Frequency domain Filtering

Importance of the phase information

Application of FT

Deconvolution in Frequency domain

Math Primer.



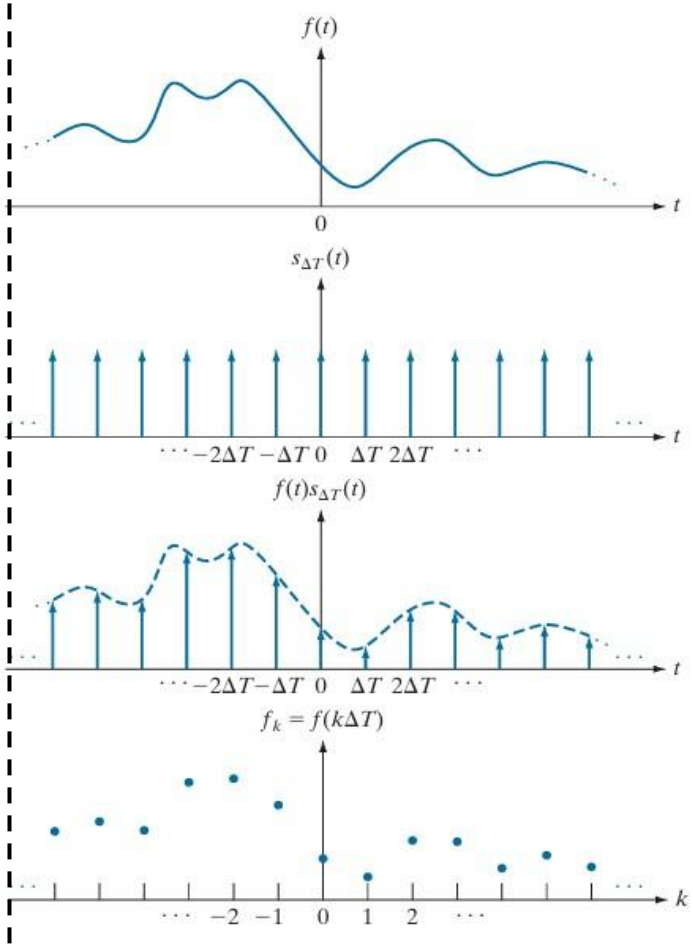
Sampling

Motivation:
As we know, lens of the camera creates the continuous optical image on the image plane.

Image sensor convert this continuous image into discrete digital image.

How densely we should sample the continuous image so that we do not lose important information in it. At the same time, sampling not introduced any artifacts or any undesirable effects in the final captured image.

Sampling: A continuous functions has to be converted into the sequence of discrete values before they processed in the computers.



(a) Continuous Function

(b) Train of Impulses used to model sampling.

(c) Sampled function formed as product of **a** and **b**.

Aliasing in 2D image

If sampling not done properly then some undesirable artifacts added in the image

Frequency Domain Fundamentals: Sampling theorem, Nyquist rate

Sampling Theorem: A continuous bandlimited signal can be recovered completely from the set of its samples if the samples are acquired at a rate exceeding at least twice the highest frequency content of the function.

Fs > 2fm (Oversampled)

Fs = 2fm (Perfect sampling or Nyquist rate)

A sampling rate exactly equal to twice the highest frequency is called the **Nyquist rate.**

Fs < 2fm (Under sampling or **aliasing** effect)

Where,

Fs = sampling frequency

fm = highest frequency component present in the signal

Nyquist Frequency = Nyquist rate / 2

Example -

$m(t) = \sin 2\Pi t + \sin 3\Pi t + \sin 4\Pi t$

$W = 2\Pi$

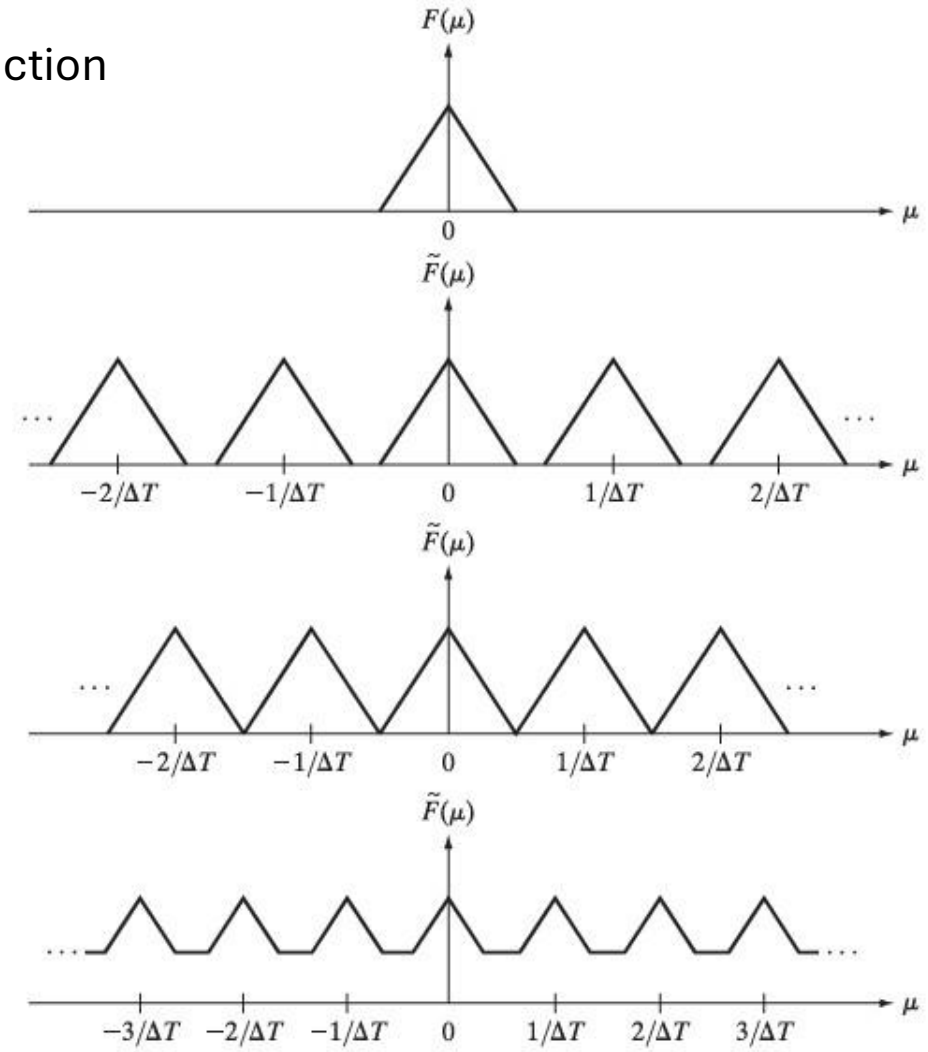
$2\Pi f = 2\Pi$

$f = 1$

Similarly f = 1.5 and 2 for other two cases respectively.

Frequency Domain Fundamentals: Sampling theorem, Nyquist rate

FT of band limited function



Oversampling

Fs > 2fm (Oversampled)

Critical sampling

Fs = 2fm (Perfect sampling or Nyquist rate)

A sampling rate exactly equal to twice the highest frequency is called the **Nyquist rate.**

Under sampling

Fs < 2fm (Under sampling or **aliasing** effect)

Where,
Fs = sampling frequency, fm = highest frequency component present in the signal

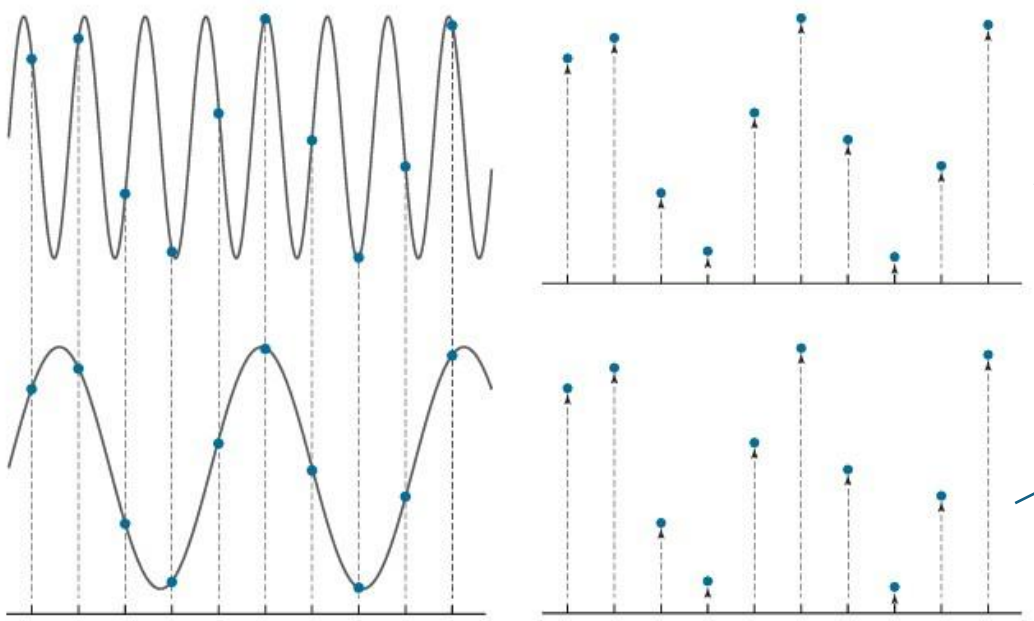
Frequency Domain Fundamentals: Aliaising

What happened if during sampling of continuous function, someone is not following the sampling theorem ?

Answer: Aliasing or false identity

Aliasing: It's a phenomena where different signals are indistinguishable from one another after sampling.

Fs < 2fm (Under sampling or **aliasing** effect)



Two different function but their digitization is same.

From here, it is very difficult to recover back the original signal because digitization of Two different signal is same.

Solution

Anti Aliasing :

Aliasing can be reduced by smoothening (Low Pass filter) on the input function or image to attenuate the higher frequencies.

This process has to be done before the function is sampled because aliasing is an sampling issue that cannot be undone.

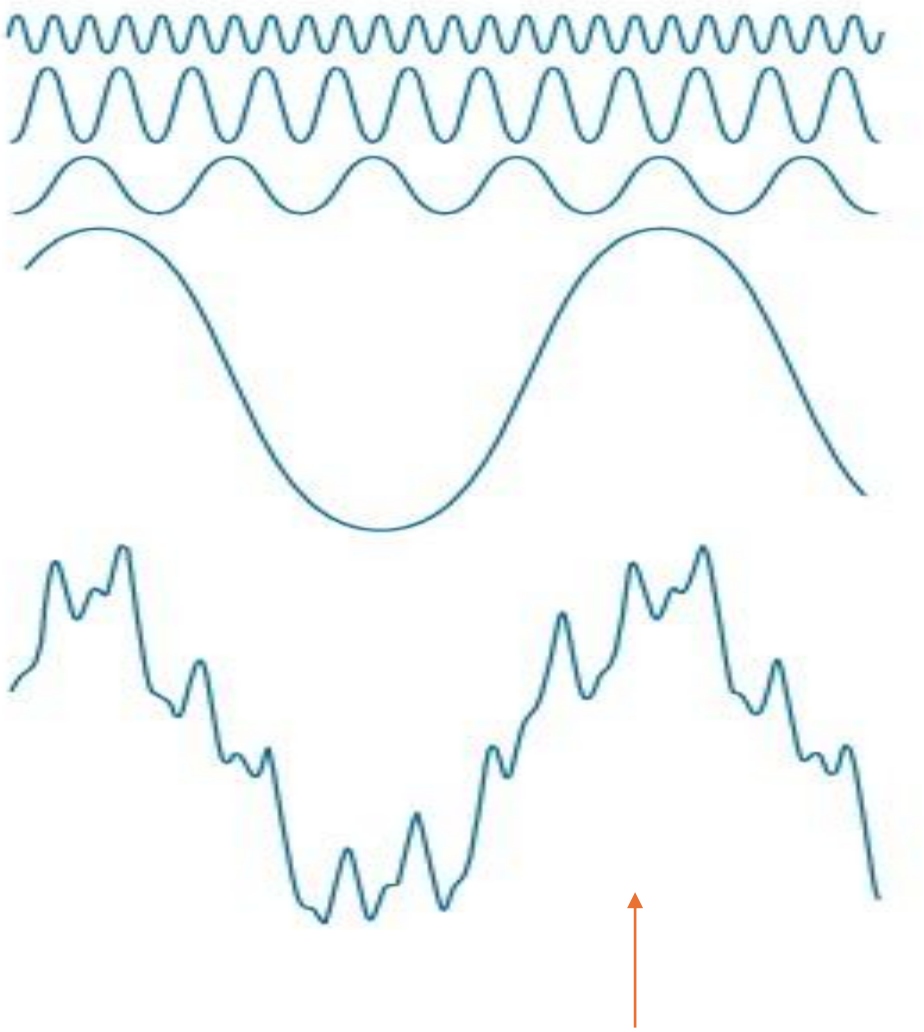
Frequency Domain Fundamentals: Fourier series

1. **Fourier Series:** Any periodic function can be expressed as a sum of sines and/or cosines of different frequencies, each multiplied by a different co-efficient.

$$f(t) = \sum_{n=-\infty}^{\infty} c_n e^{j\frac{2\pi n}{T}t}$$

$$c_n = \frac{1}{T}\int_{-T/2}^{T/2} f(t) e^{-j\frac{2\pi n}{T}t} dt \qquad \text{for } n = 0, \pm 1, \pm 2, \ldots$$

Where –
A function f(t) of a continuous variable, t that is periodic with period T.



This function is the sum of all the four above functions

Note: Here, Sine and cosines are the basis functions.

Frequency Domain Fundamentals: Fourier Transform

2. **Fourier Transform** : Functions those are not periodic (But whose area under the curve is finite) can be expressed as the integrals of sines and/or cosines of different frequencies each multiplied by a weighting function.

Fourier Transform Formula

$$F(\mu) = \int_{-\infty}^{\infty} f(t) e^{-j2\pi\mu t} dt$$

f(t) is a continuous function.

Inverse Fourier Transform Formula

$$f(t) = \int_{-\infty}^{\infty} F(\mu) e^{j2\pi\mu t} d\mu$$

$$f(t) \Leftrightarrow F(\mu)$$

It's a Fourier transform pair which indicate forward and Inverse Fourier transform is possible.

$$e^{i\theta} = \cos\theta + i\sin\theta \qquad i = \sqrt{-1}$$

Expand $e^{i\theta}$ using Taylor Series:

$$e^{i\theta} = 1 + i\theta + \frac{(i\theta)^2}{2!} + \frac{(i\theta)^3}{3!} + \frac{(i\theta)^4}{4!} + \frac{(i\theta)^5}{5!} + \frac{(i\theta)^6}{6!} + \dots$$

$$e^{i\theta} = \underbrace{\left(1 - \frac{\theta^2}{2!} + \frac{\theta^4}{4!} - \frac{\theta^6}{6!} + \cdots\right)}_{\cos\theta} + i\underbrace{\left(\theta - \frac{\theta^3}{3!} + \frac{\theta^5}{5!} - \frac{\theta^7}{7!} + \cdots\right)}_{\sin\theta}$$

Note: Here, Sine and cosines are the basis functions.

Fourier Transform Working Example

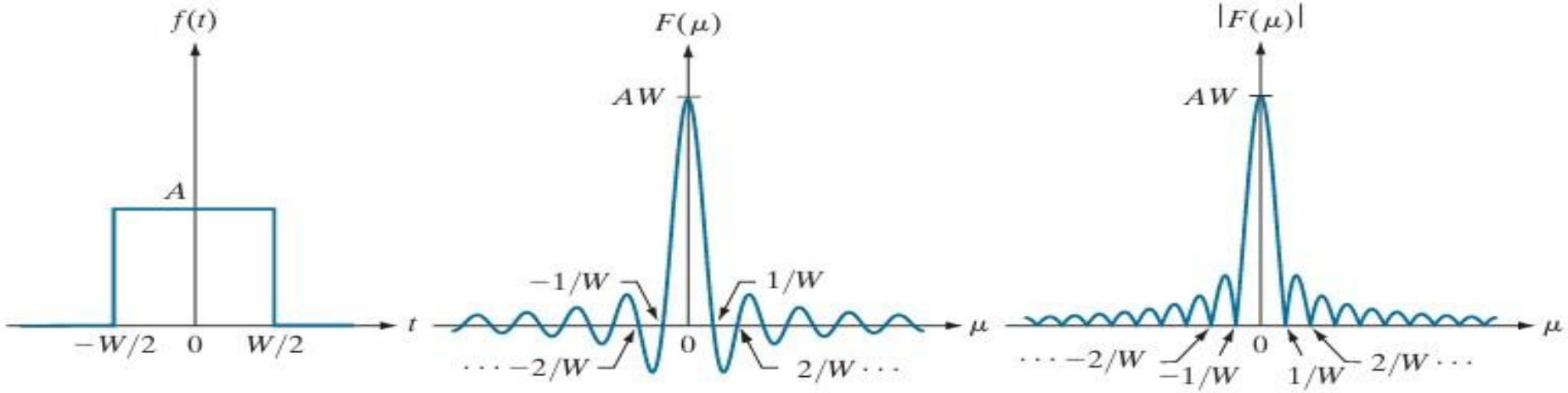
$$F(\mu) = \int_{-\infty}^{\infty} f(t)e^{-j2\pi\mu t}dt = \int_{-W/2}^{W/2} Ae^{-j2\pi\mu t}dt$$

$$= \frac{-A}{j2\pi\mu}\left[e^{-j2\pi\mu t}\right]_{-W/2}^{W/2} = \frac{-A}{j2\pi\mu}\left[e^{-j\pi\mu W} - e^{j\pi\mu W}\right]$$

$$= \frac{A}{j2\pi\mu}\left[e^{j\pi\mu W} - e^{-j\pi\mu W}\right]$$

$$= AW\frac{\sin(\pi\mu W)}{(\pi\mu W)}$$ (Signum function)

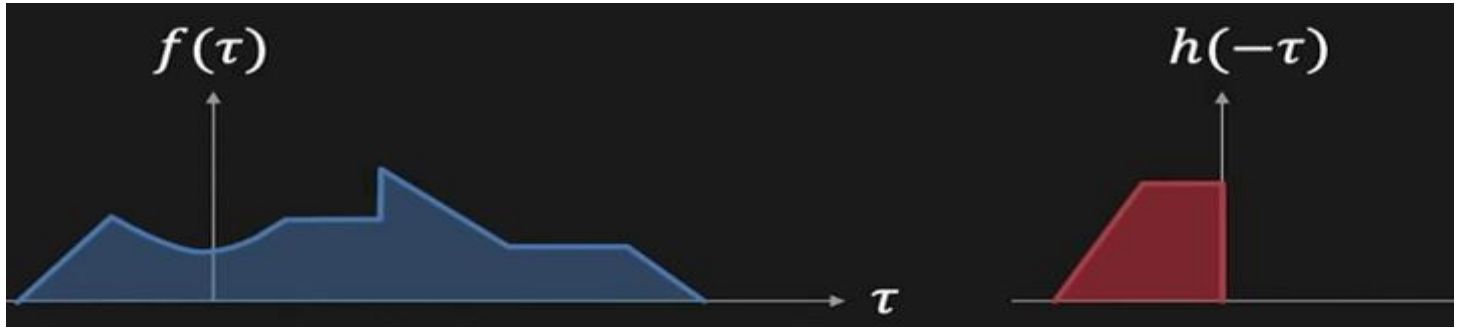
Note: A Functions that is expressed by either Fourier series or Fourier Transform could be reconstructed or recovered completely via an inverse process.



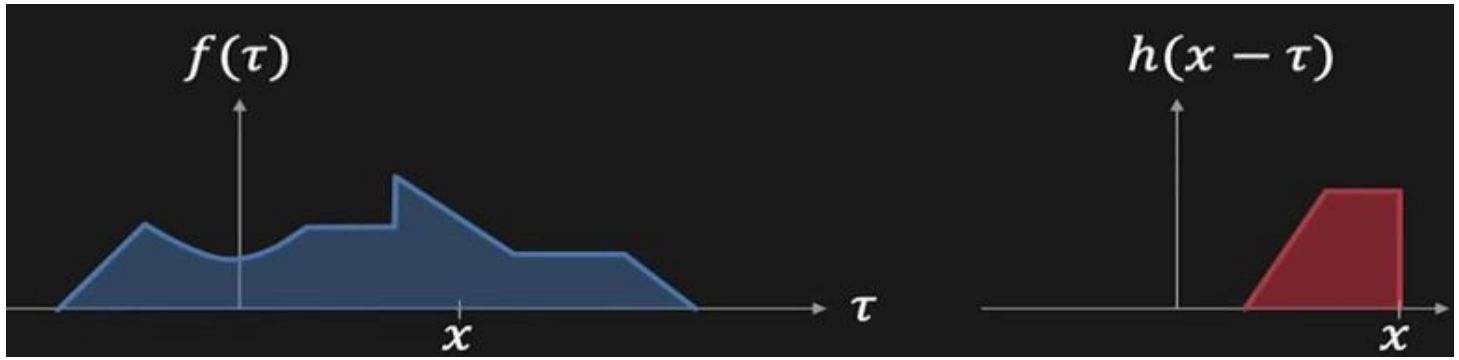
(a) A Box function (b) Fourier Transform (c) Spectrum

Convolution Theorem:

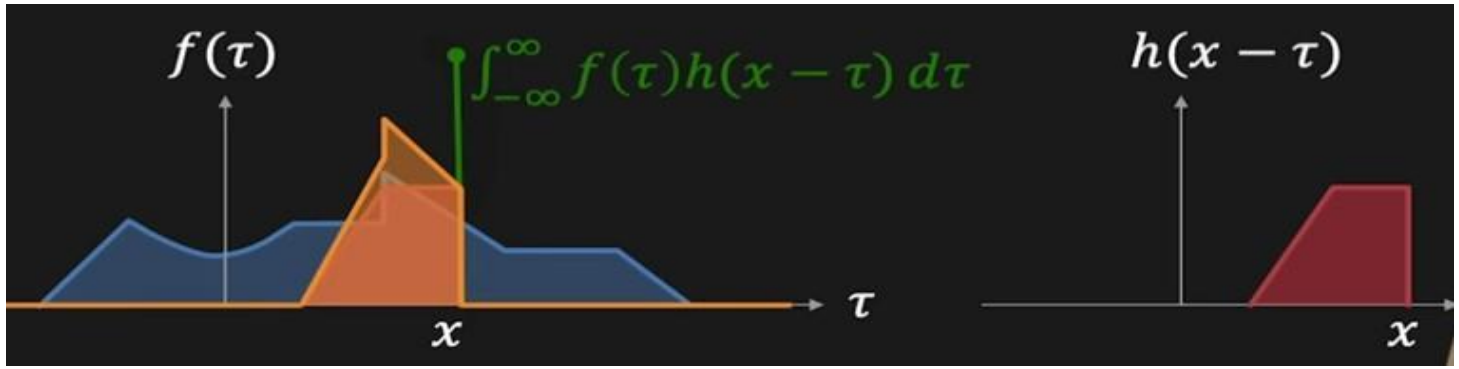
Revisit, what is the convolution.



Flip h(x)



shift h(x)



Overlay it on the f(x). Multiply f(x) and h(x)and perform integration, it will provide a single value to a particular point. This is know as convolution.

Convolution Theorem

Convolution Theorem: Convolution of two signal or image in time domain will become the multiplication in the frequency domain.

Convolution: $g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x-\tau)\,d\tau$

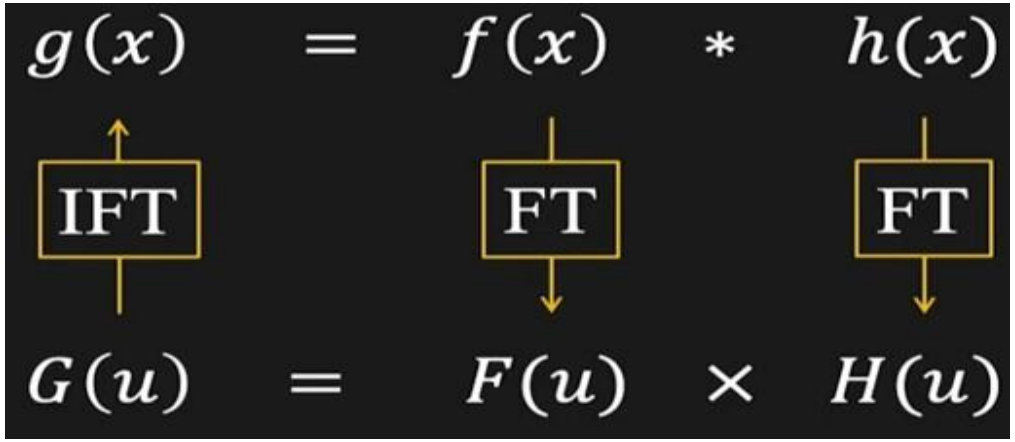
Fourier Transform of $g(x)$:

$$G(u) = \int_{-\infty}^{\infty} g(x)e^{-i2\pi ux}dx$$

$$G(u) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(\tau)h(x-\tau)e^{-i2\pi ux}d\tau dx$$

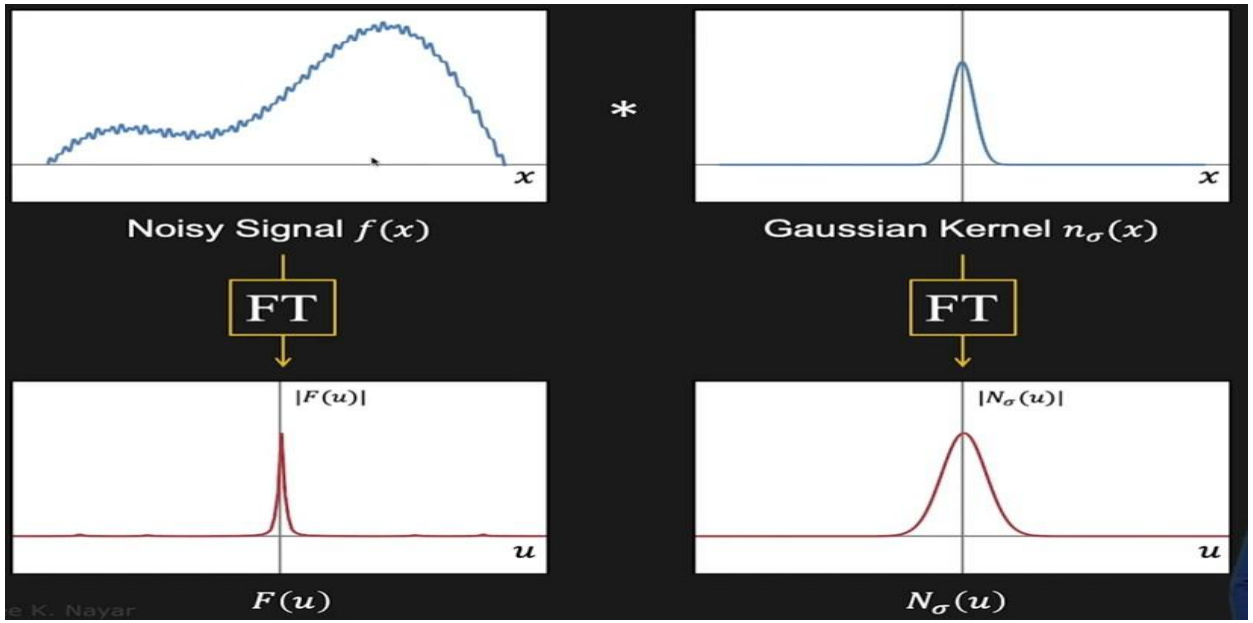
$$G(u) = \underbrace{\int_{-\infty}^{\infty} f(\tau)e^{-i2\pi u\tau}d\tau}_{F(u)} \underbrace{\int_{-\infty}^{\infty} h(x-\tau)e^{-i2\pi u(x-\tau)}dx}_{H(u)}$$

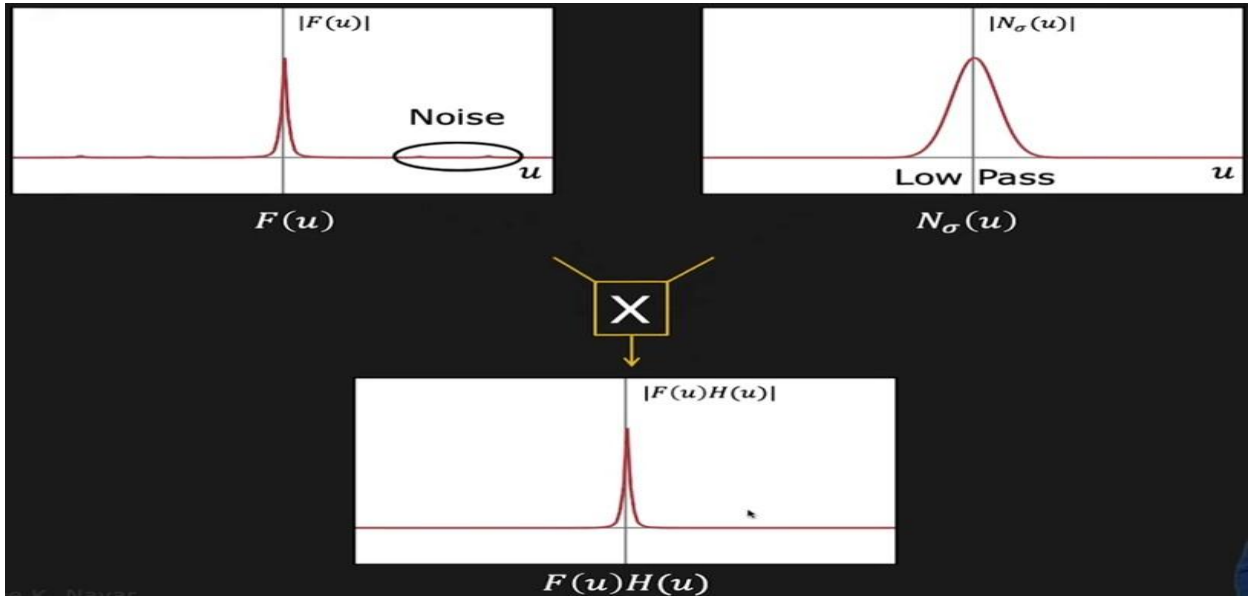
Spatial Domain		Frequency Domain
$g(x) = f(x) * h(x)$ Convolution	$\longleftrightarrow$	$G(u) = F(u)\,H(u)$ Multiplication
$g(x) = f(x)\,h(x)$ Multiplication	$\longleftrightarrow$	$G(u) = F(u) * H(u)$ Convolution

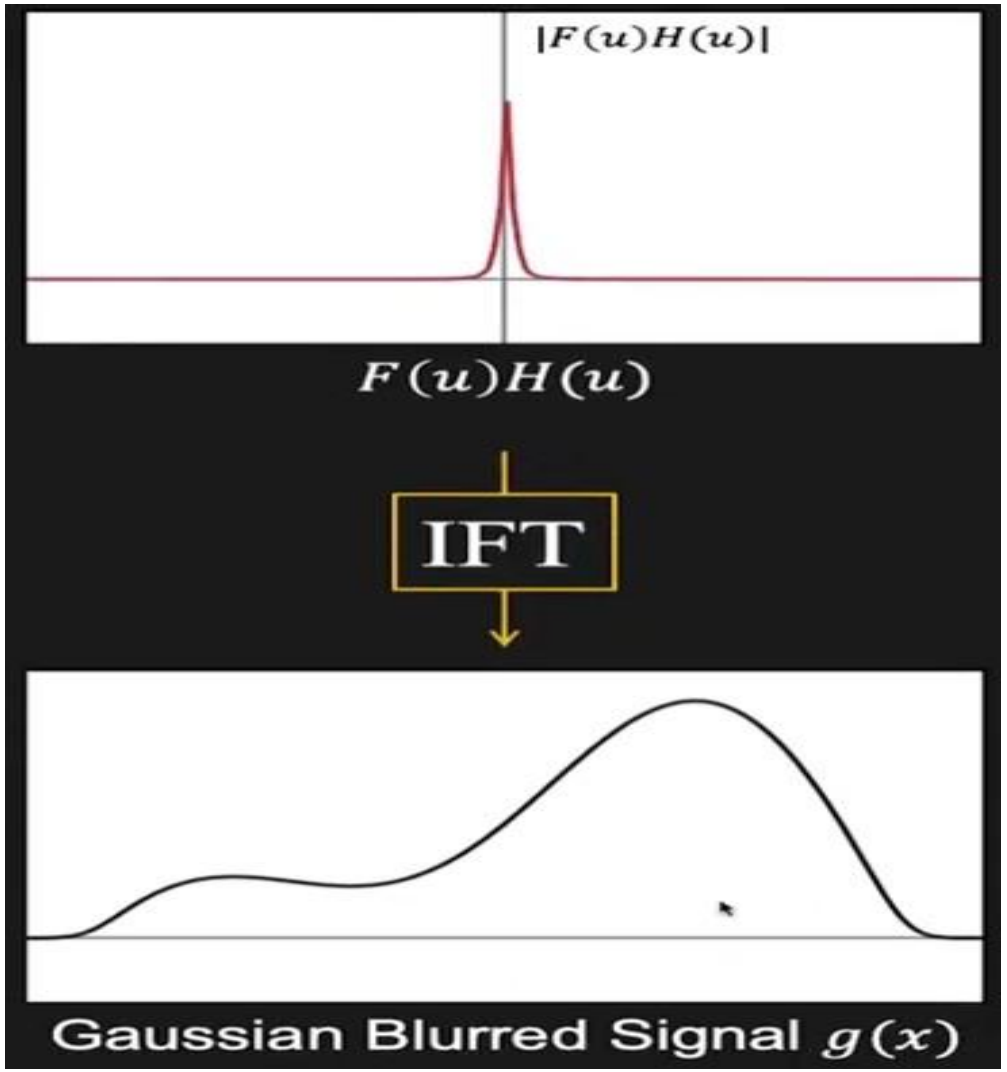


Convolution Theorem

Fourier Transform Working Example





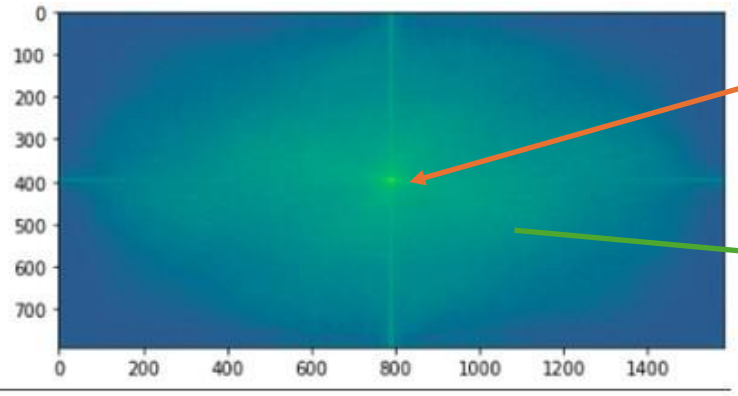


Interpretation image in the Frequency Domain: Fourier Transform

Fourier Transform: It's an image enhancement tool, which is used to decompose an image into its sine and cosine components.

Low frequencies : It represent smooth transition in the image.

High frequencies : It capture the rapid changes in the image. Like edges and texture.



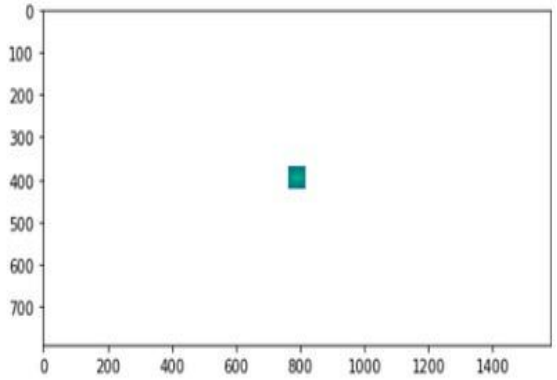
Center shows low Frequency component

High Frequency component

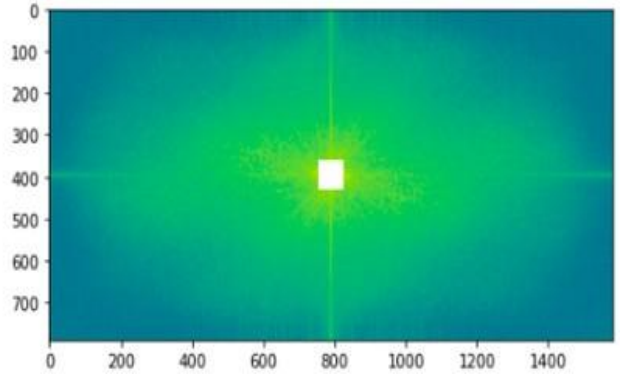
Input Image

F.T. of input Image with heatmap

Keep only low frequency components: image is blurred

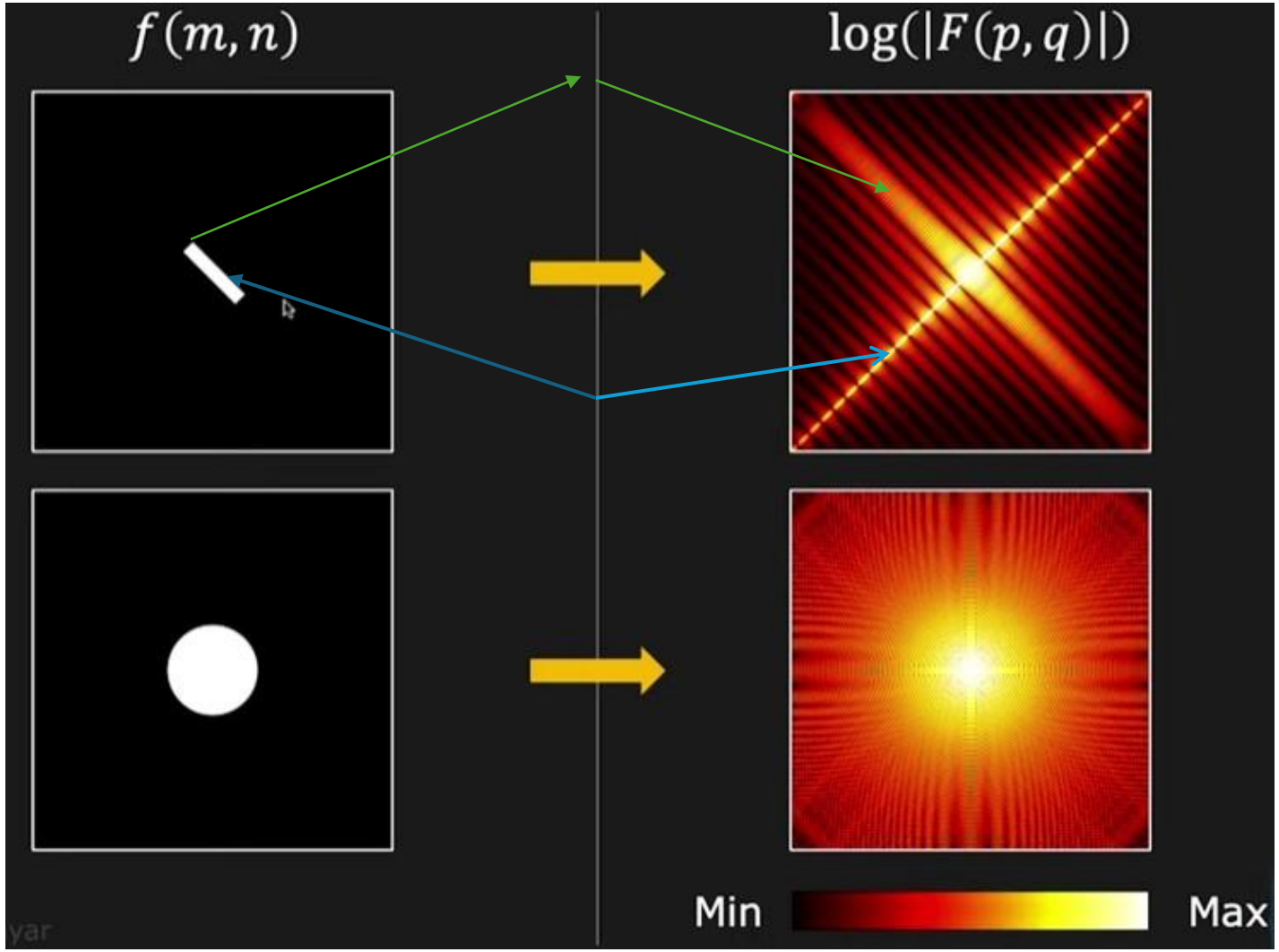


Keep only high frequency components

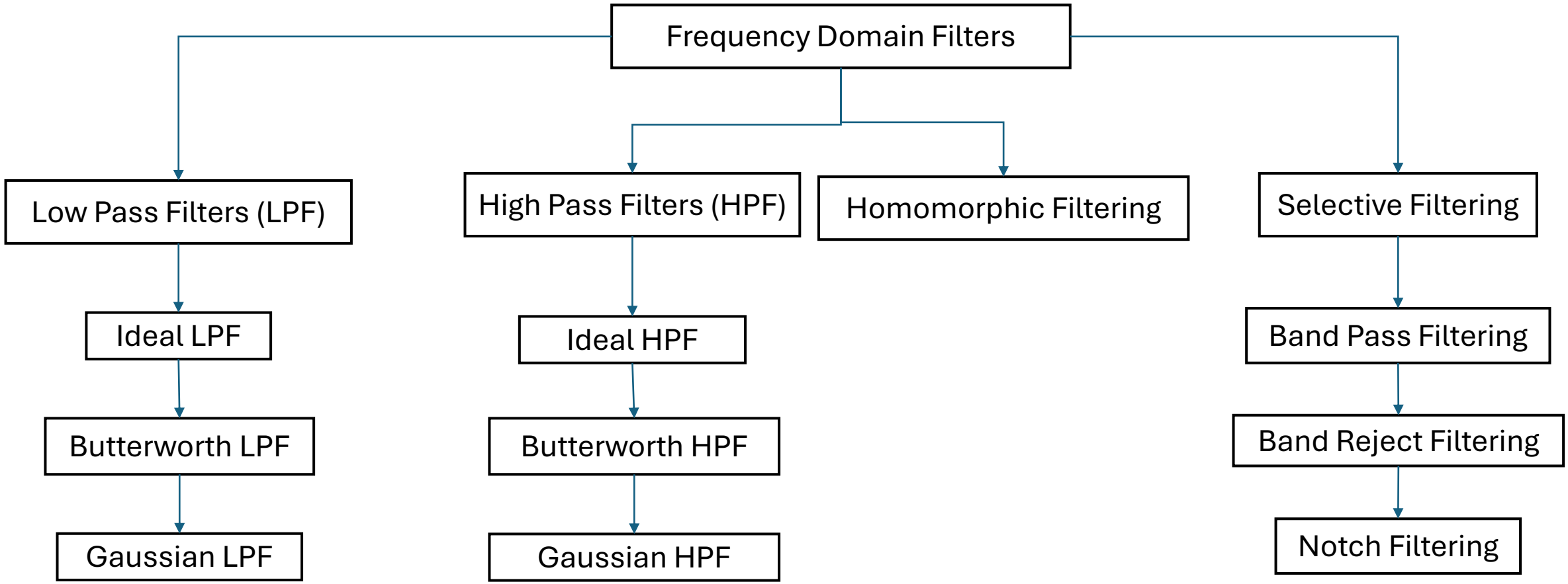


Frequency Domain Filtering: Application

Heatmap of original image and its fourier transform



Frequency Domain Filters



Frequency Domain Filters: Ideal Low Pass Filter (ILPF)

A 2D filter which passes all the frequencies within a circle of radius from the origin and cut off or attenuate all the frequencies which are outside to this circle.

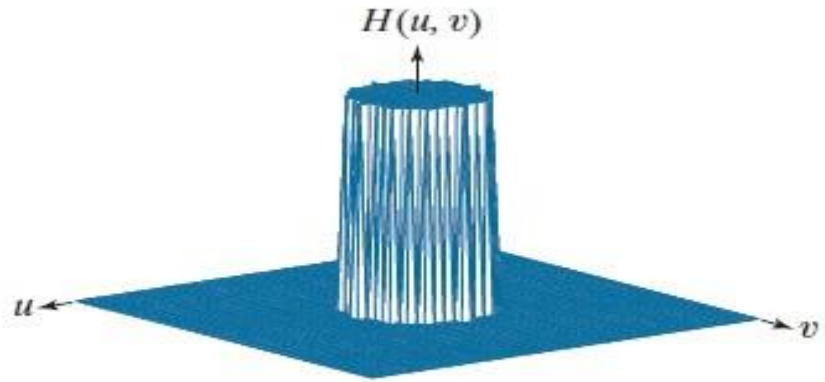
It is specified by the transfer function H(u,v)

$$H(u,v) = \begin{cases} 1 & \text{if } D(u,v) \le D_0 \\ 0 & \text{if } D(u,v) > D_0 \end{cases}$$

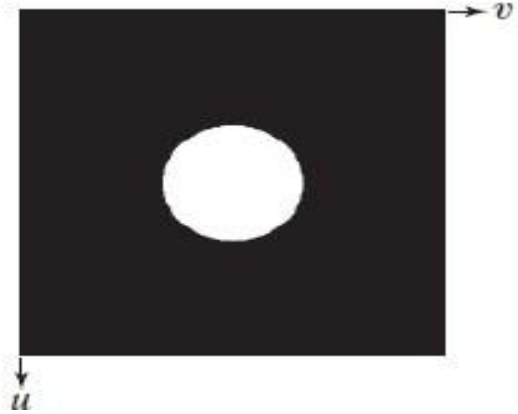
Where - Do = Positive Constant

D(u,v) = distance between a point (u , v) in the frequency domain and the center.

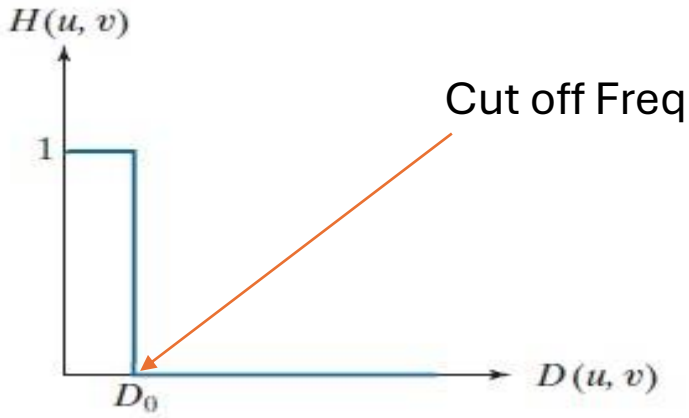
Frequency Domain Filters: Ideal Low Pass Filter (ILPF)



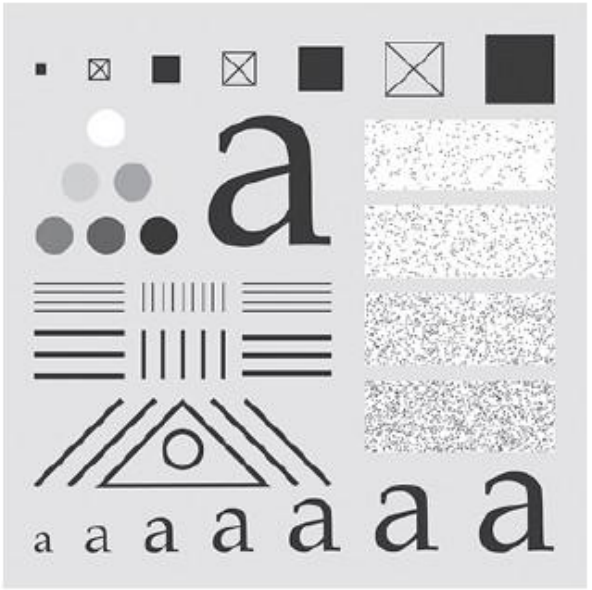
(a) Ideal LPF Transfer function Plot



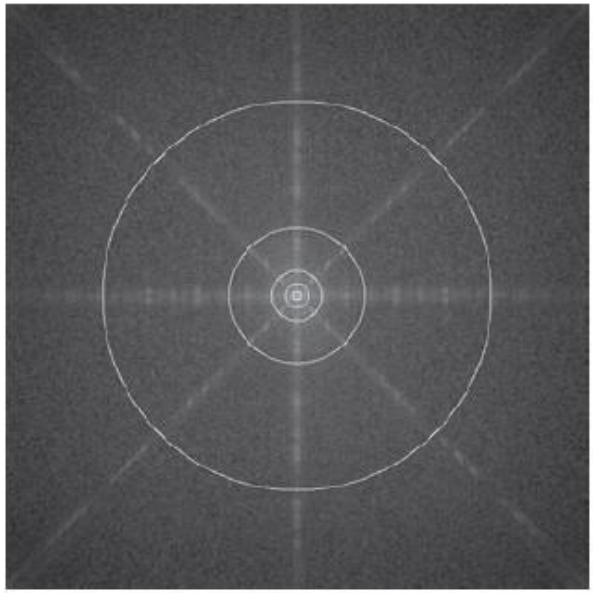
(b) Function displayed as an image



(c) Radial Cross Section

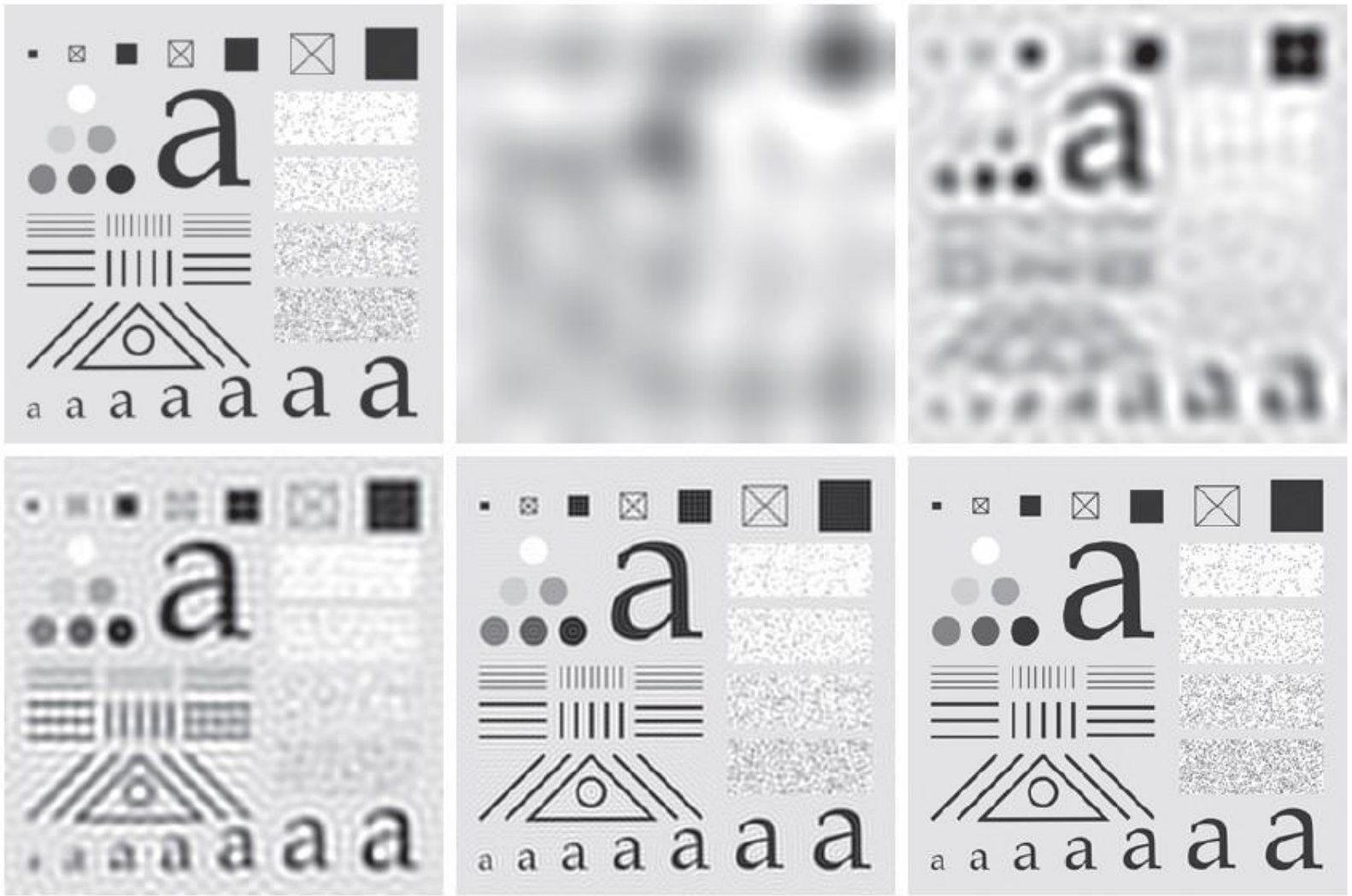


Test Pattern image



Circle with radi 10,30,60,160,460

Frequency Domain Filters: Ideal Low Pass Filter (ILPF)



a – Original Image

b – ILPF with cut off Frequency set at radii value - **10**

c – ILPF with cut off Frequency set at radii value - **30**

d – ILPF with cut off Frequency set at radii value - **60**

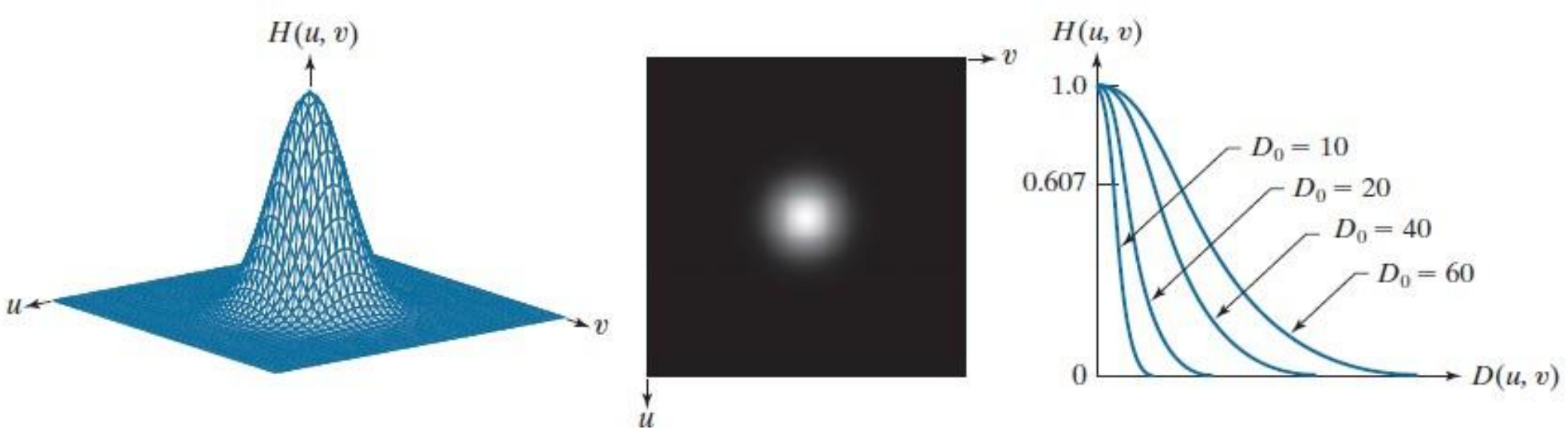
e – ILPF with cut off Frequency set at radii value - **160**

f – ILPF with cut off Frequency set at radii value - **460**

Frequency Domain Filters: Gaussian Low Pass Filter (GLPF)

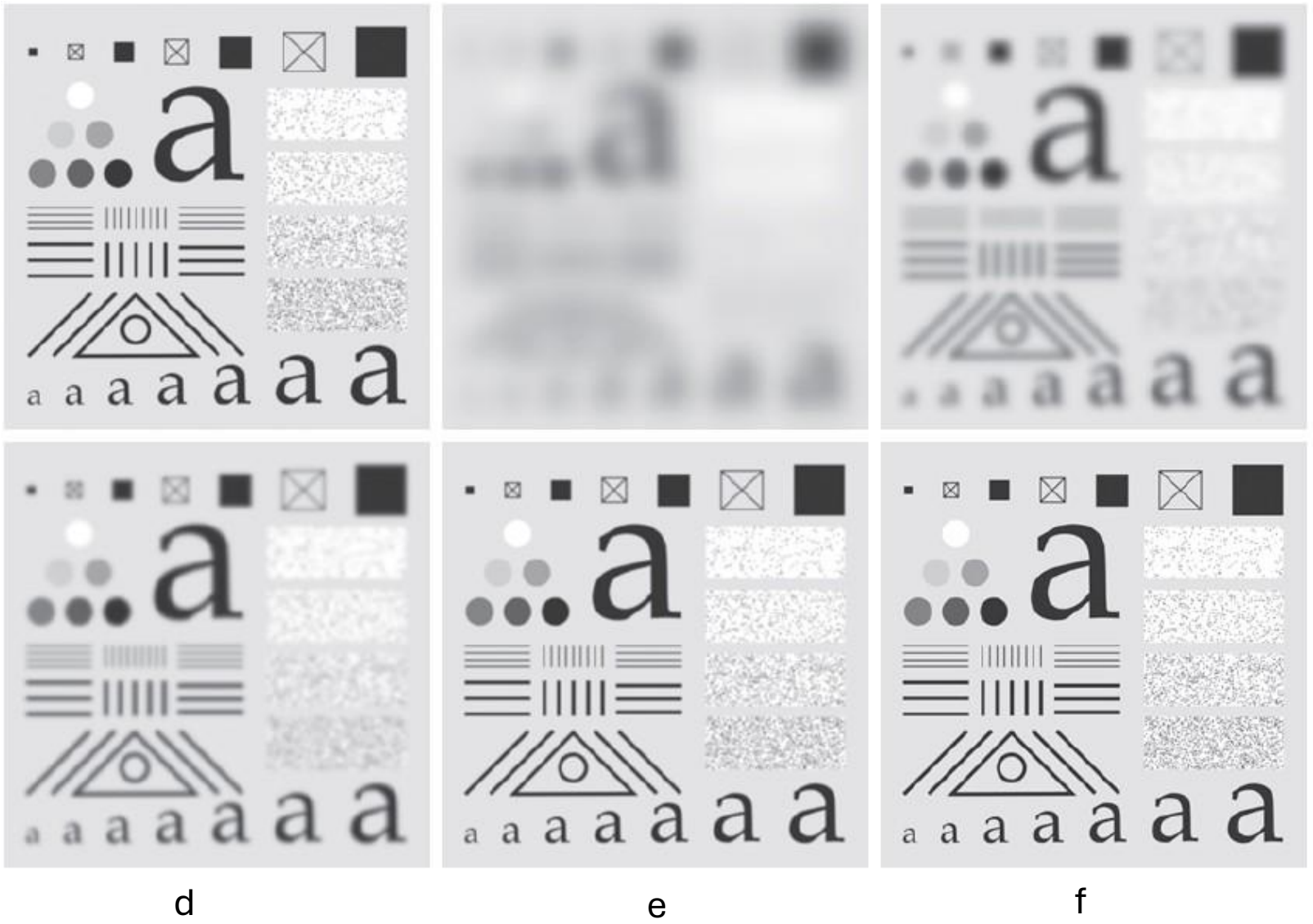
It is specified by the transfer function H(u,v)

$$H(u,v) = e^{-D^2(u,v)/2D_0^2}$$



(a) Gaussian LPF Transfer function Plot (b) Function displayed as an image (c) Radial Cross Section with various value of D0

Frequency Domain Filters: Gaussian Low Pass Filter (GLPF)



a – Original Image

b – GLPF with cut off Frequency set at radii value - **10**

c – GLPF with cut off Frequency set at radii value - **30**

d – GLPF with cut off Frequency set at radii value - **60**

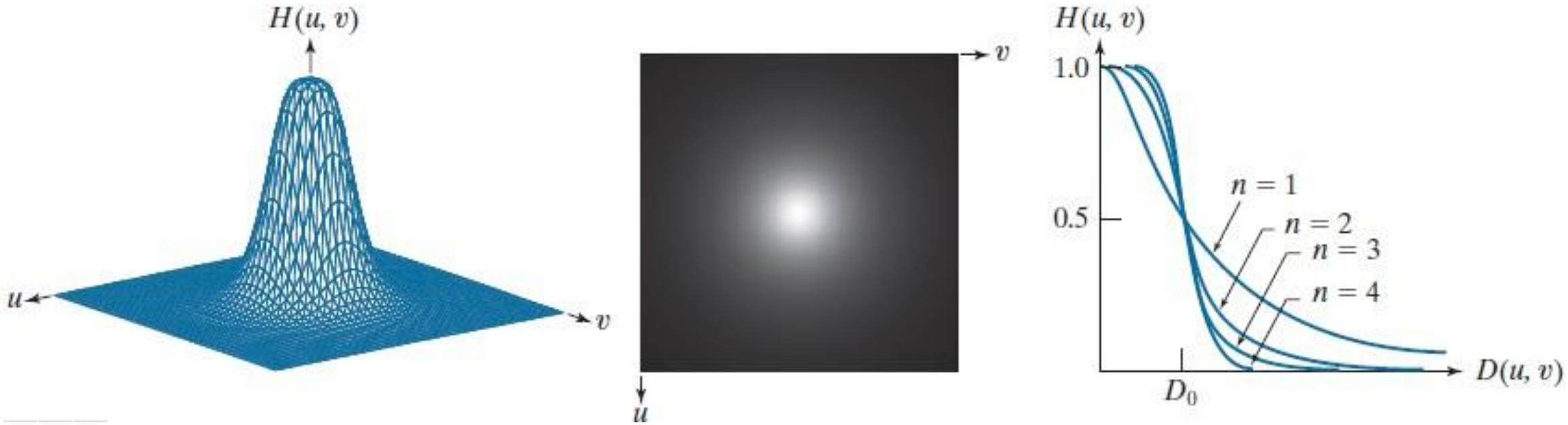
e – GLPF with cut off Frequency set at radii value - **160**

f – GLPF with cut off Frequency set at radii value - **460**

Frequency Domain Filters: Butterworth Low Pass Filter (BLPF)

It is specified by the transfer function H(u,v)

$$H(u,v) = \frac{1}{1 + [D(u,v)/D_0]^{2n}}$$

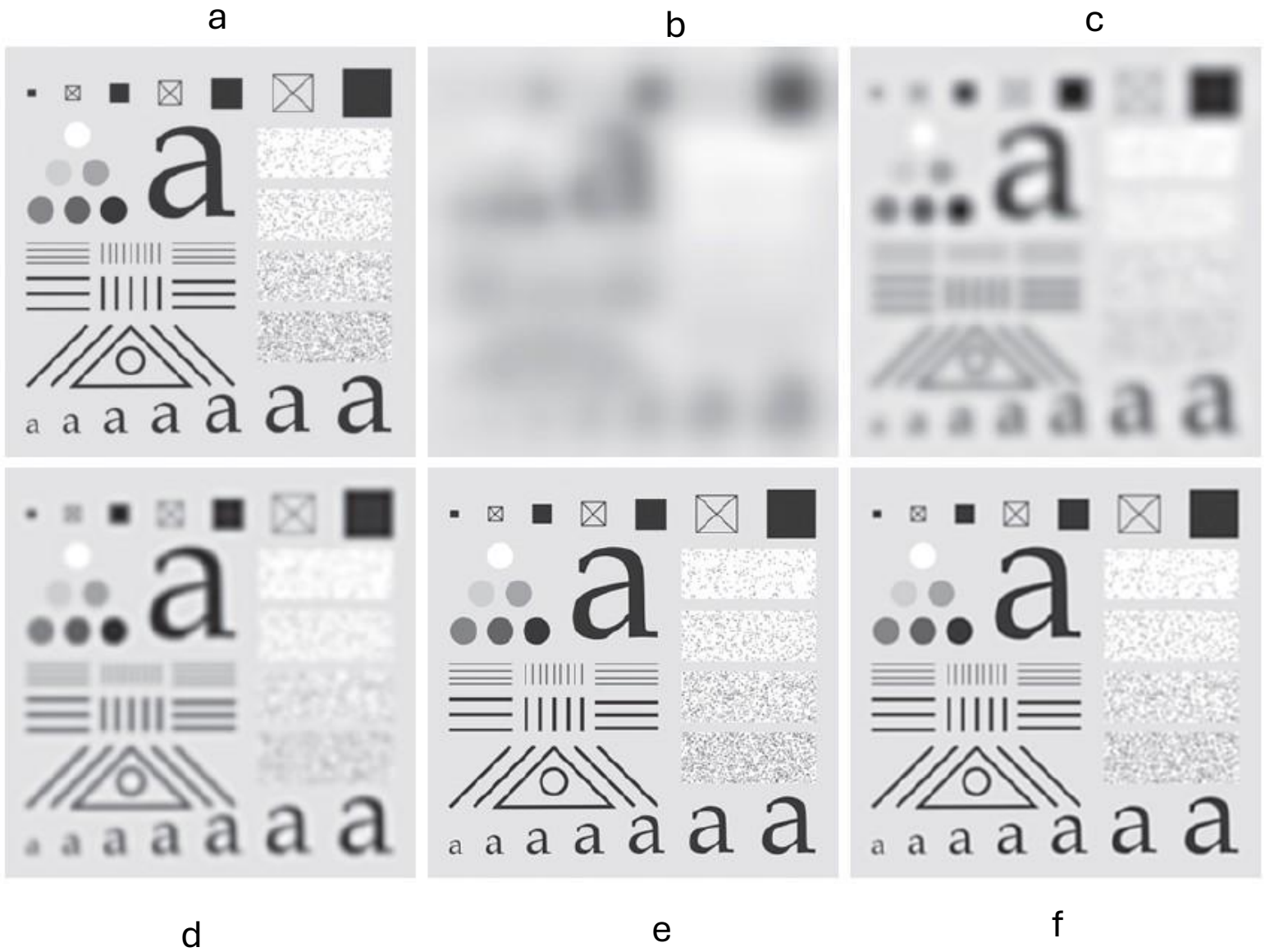


(a) Gaussian LPF Transfer function Plot

(b) Function displayed as an image

(c) Radial Cross Section with orders 1 to 4.

Frequency Domain Filters: Butterworth Low Pass Filter (BLPF)



a – Original Image

b – BLPF with cut off Frequency set at radii value - **10**

c – BLPF with cut off Frequency set at radii value - **30**

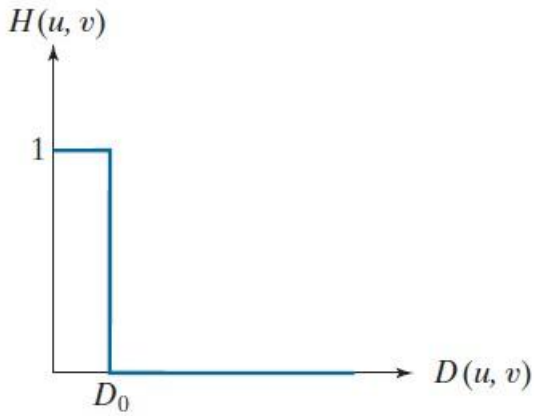
d – BLPF with cut off Frequency set at radii value - **60**

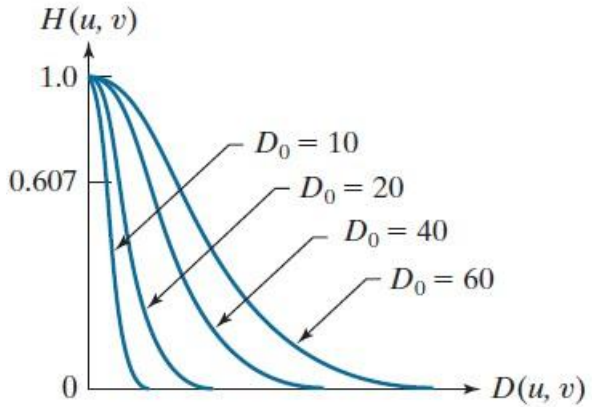
e – BLPF with cut off Frequency set at radii value - **160**

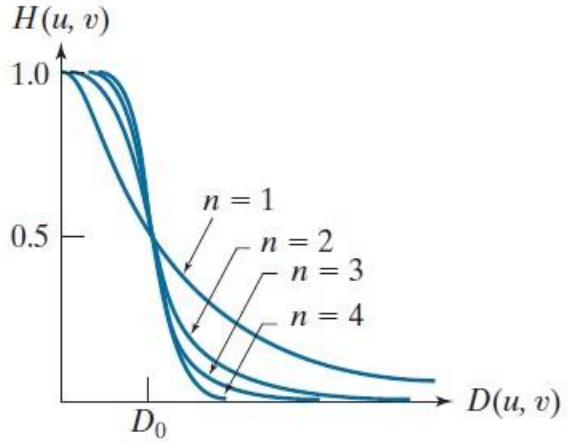
f – BLPF with cut off Frequency set at radii value - **460**

Comparative Analysis Between ILPF, GLBF and BLPF

Ideal	Gaussian	Butterworth
$H(u,v)=\begin{cases}1 & \text{if } D(u,v)\le D_0\\ 0 & \text{if } D(u,v)>D_0\end{cases}$	$H(u,v)=e^{-D^2(u,v)/2D_0^2}$	$H(u,v)=\dfrac{1}{1+\left[D(u,v)/D_0\right]^{2n}}$







Note:

The shape of the Butterworth filter is controlled by the filter order.

If n is large then Butterworth filter approaches to ILPF.

if n is small then Butterworth filter approaches to GLPF.

Frequency Domain Filters: Image Sharpening using High Pass Filter

High Pass Filter

Image Sharpening can be achieved in the frequency domain by passing the high frequency components (i.e Edges or other sharp transitions) and attenuate the low frequency components.

Ideal	Gaussian	Butterworth
$H(u,v) = \begin{cases} 0 & \text{if } D(u,v) \le D_0 \\ 1 & \text{if } D(u,v) > D_0 \end{cases}$	$H(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}$	$H(u,v) = \frac{1}{1 + [D_0/D(u,v)]^{2n}}$

Where

D_0 = Cut off frequency

n = order

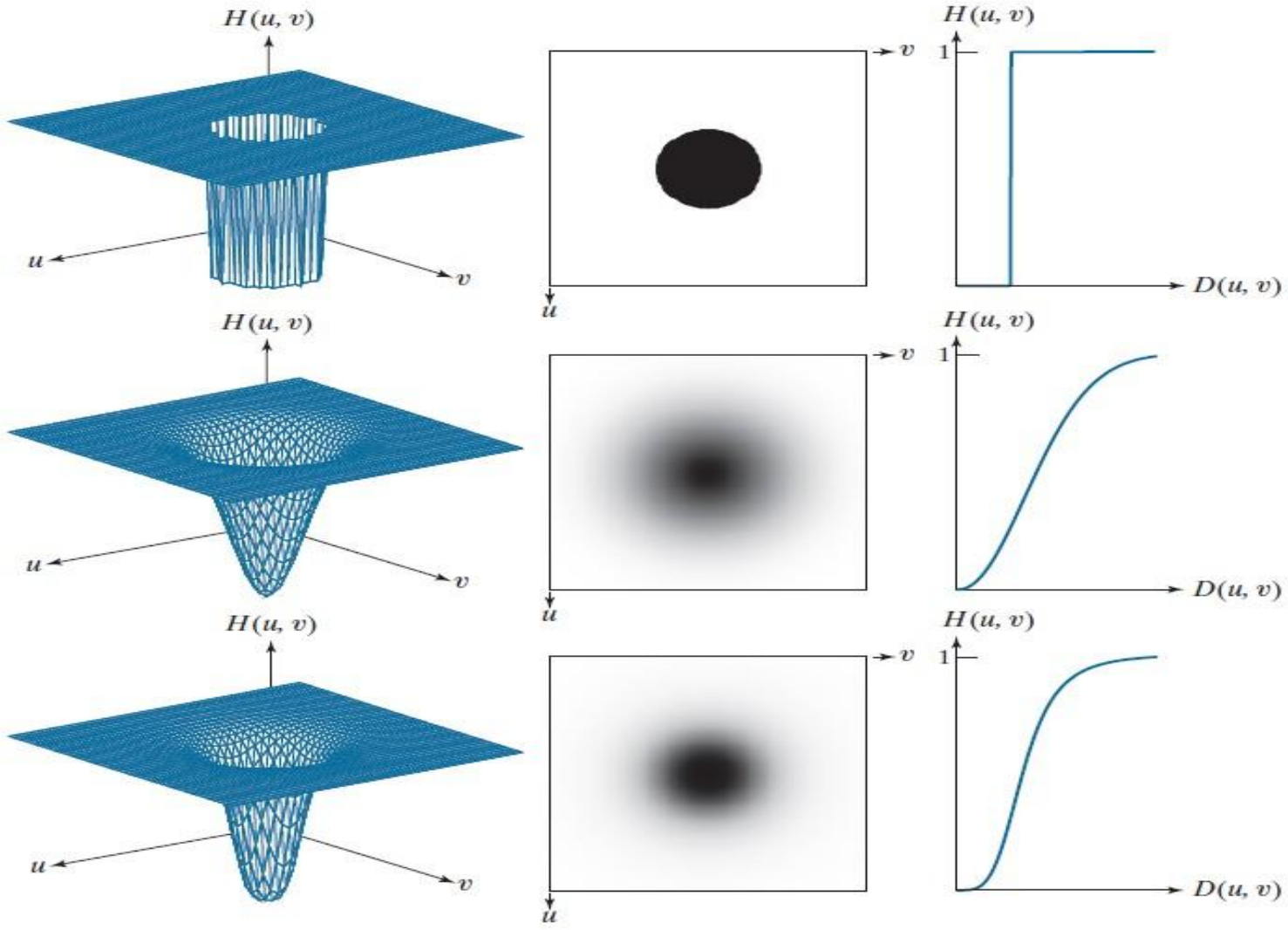
Frequency Domain Filters: Image Sharpening using High PF

$$H(u,v) = \begin{cases} 0 & \text{if } D(u,v) \le D_0 \\ 1 & \text{if } D(u,v) > D_0 \end{cases}$$

$$H(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}$$

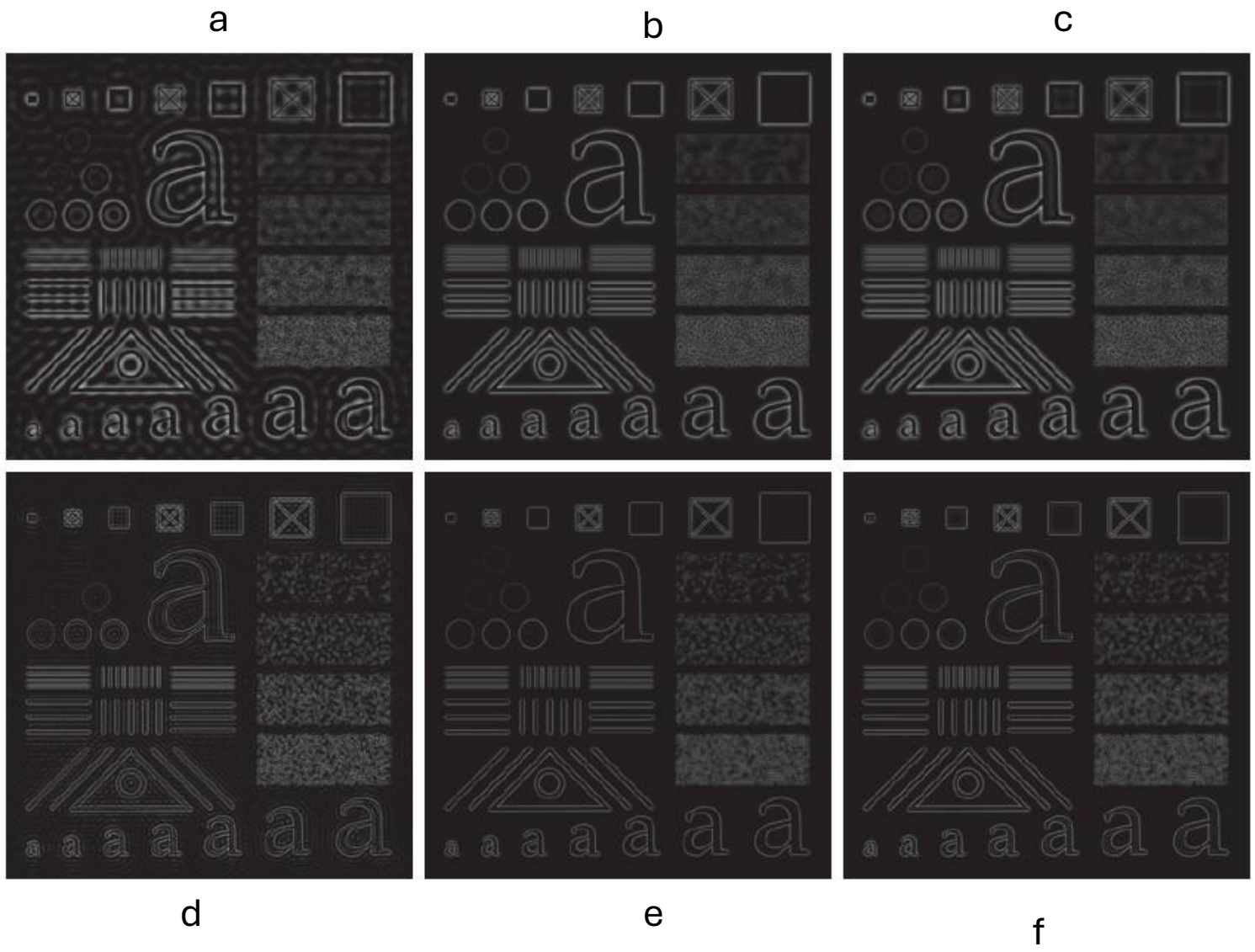
$$H(u,v) = \frac{1}{1 + [D_0/D(u,v)]^{2n}}$$

Transfer Function



(a) HPF Transfer function Plot (b) Function displayed as an image (c) Radial Cross Section

Frequency Domain Filters: Image Sharpening using High PF



Filtered with (a) IHPF, (b) GHPF, (c) BHPF with D0 =60

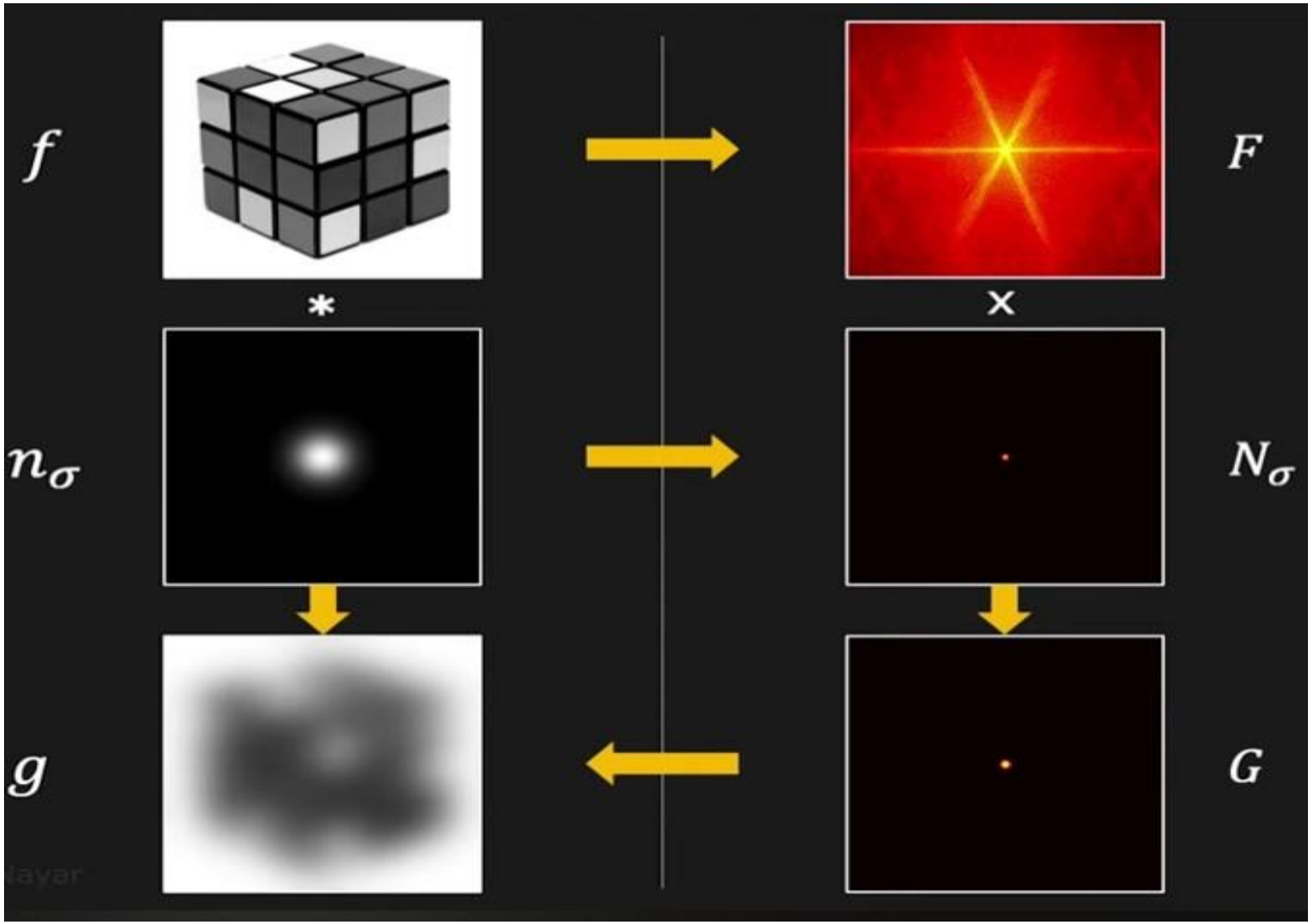
Filtered with (d) IHPF, (e) GHPF, (f) BHPF with D0 =160

Gaussian Smoothening

Original Image

Gaussian Smoothening kernel

Inverse FT



Original Image FT

Gaussian Smoothening kernel FT

Multiplication of Original image FT and Kernel FT.
Here output only contains low frequency terms.

Frequency Domain Filters: Working Examples

1-D Discrete Fourier Transform (DFT) and Inverse Discrete Fourier Transform (IDFT)

DFT

$$F(u) = \sum_{x=0}^{M-1} f(x) e^{-j2\pi ux/M} \qquad u = 0, 1, 2, \ldots, M-1$$

IDFT

$$f(x) = \frac{1}{M} \sum_{u=0}^{M-1} F(u) e^{j2\pi ux/M} \qquad x = 0, 1, 2, \ldots, M-1$$

1-D DFT and Inverse DFT working example

DFT

$$F(u)=\sum_{x=0}^{M-1} f(x)e^{-j2\pi ux/M} \qquad u=0,1,2,\ldots,M-1$$

IDFT

$$f(x)=\frac{1}{M}\sum_{u=0}^{M-1} F(u)e^{j2\pi ux/M} \qquad x=0,1,2,\ldots,M-1$$

$f(k) = \sum_{x=0}^{N-1} f(x)\, e^{-j2\pi kx/N}$

where $k = 0, 1, 2, \ldots N-1$

IDFT

$f(x) = \frac{1}{N}\sum_{k=0}^{N-1} F(k)\, e^{j2\pi kx/N}$

where $x = 0, 1, 2, \ldots N-1$

Q.1 Calculate DFT $f(x) = \{1, 0, 0, 1\}$

↳ 0 1 2 3 → Indices.

$f(k) = \sum_{x=0}^{N-1} f(x)\, e^{-j2\pi kx/N}$

Total 4 values so that $N = 4$

$f[k] = \sum_{x=0}^{3} f(x)\, e^{-j2\pi kx/4}$

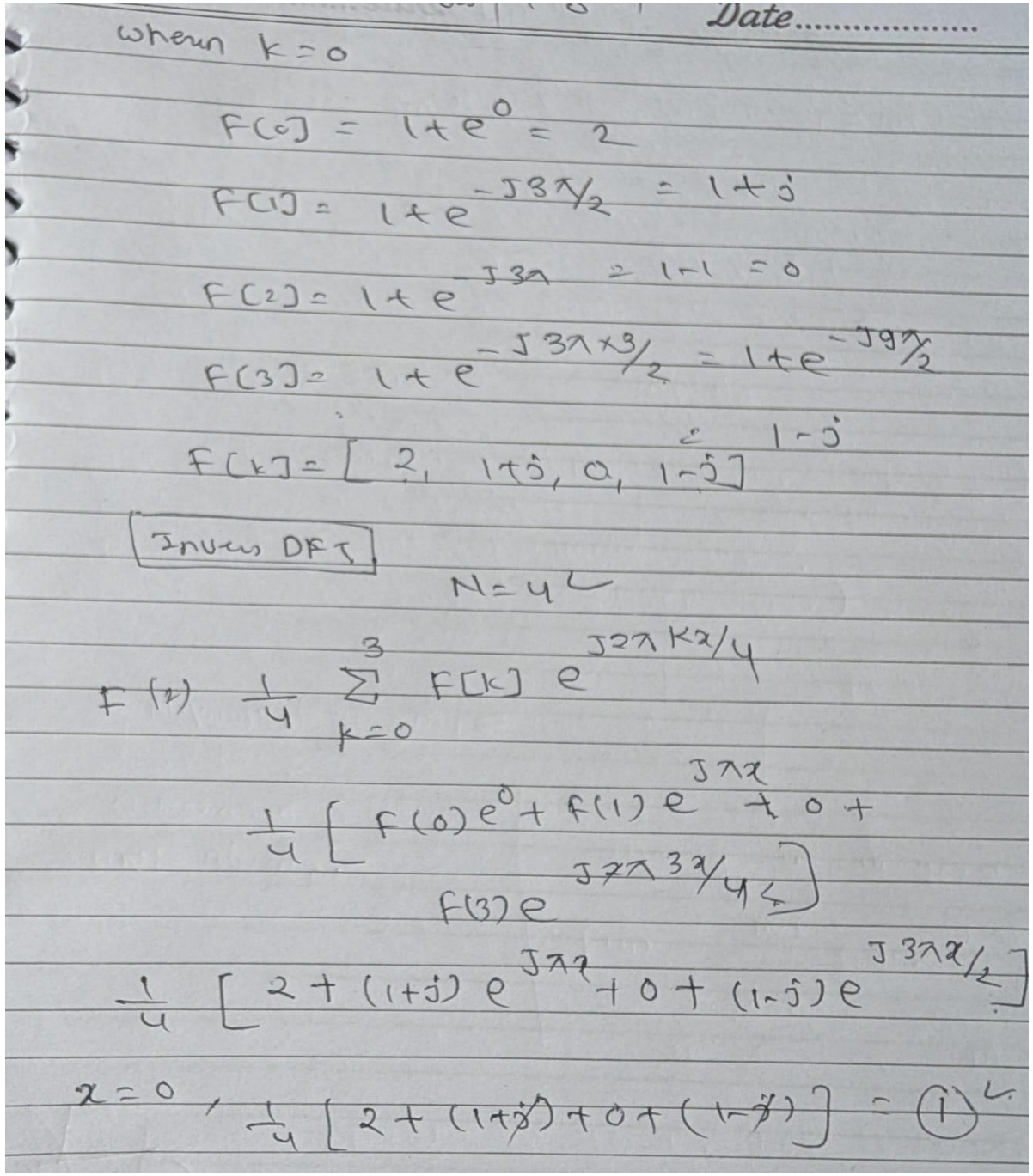
$f(0)\, e^{0} + f(1)\, e^{-j2\pi k/4} + f(2)\, e^{-j2\pi k\times 2/4} + f(3)\, e^{-j2\pi k\times 3/4}$

$1 + 0 + 0 + 1\, e^{-j3\pi k/2}$

$f(k) = 1 + e^{-j3\pi k/2}$

Teacher's Sign .....

1-D DFT and Inverse DFT working example



when k = 0

$F[0] = 1 + e^{0} = 2$

$F[1] = 1 + e^{-J3\pi/2} = 1 + j$

$F[2] = 1 + e^{-J3\pi} = 1 - 1 = 0$

$F[3] = 1 + e^{-J3\pi \times 3/2} = 1 + e^{-J9\pi/2}$

$\therefore 1 - j$

$F[k] = [2,\ 1+j,\ 0,\ 1-j]$

Inverse DFT

$N = 4$

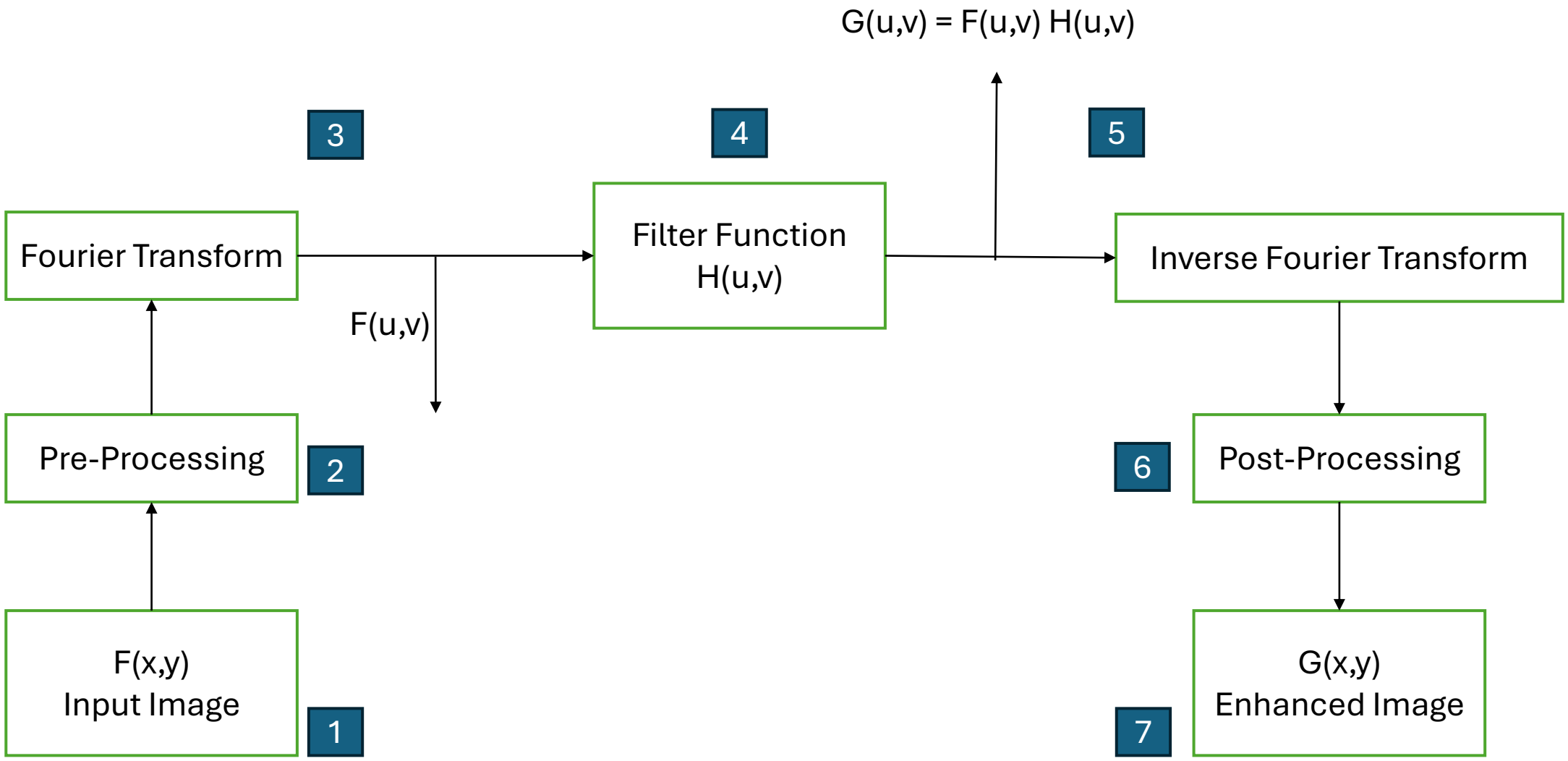
$$f(x) \frac{1}{4} \sum_{k=0}^{3} F[k]\, e^{J2\pi kx/4}$$

$$\frac{1}{4}\left[f(0)e^{0} + f(1)e^{J\pi x} + 0 + f(3)e^{J2\pi 3x/4} \right]$$

$$\frac{1}{4}\left[2 + (1+j)e^{J\pi x} + 0 + (1-j)e^{J3\pi x/2} \right]$$

$x = 0$, $\frac{1}{4}[2 + (1+j) + 0 + (1-j)] = (1)$

Frequency domain filtering : Flow Chart



Frequency Domain Filters: Working Examples

Question

For the spatial domain image, perform the frequency domain filtering using Ideal High pass filter with cut off frequency 0.5.

Input Image

1	0	1	0
1	0	1	0
1	0	1	0
1	0	1	0

Step 1: Multiply the input image by (-1)^x+y to shift the Centre from (0,0) to (2,2).

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3

Note:
Input image is 4x4. so the centre is row/2, column/2 i.e 4/2 = 2, 4/2 = 2.

Frequency Domain Filters: Working Examples

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3

(-1)^0,0 = 1, (-1)^0,1 = -1 (-1)^0,2 = 1 (-1)^0,3 = -1

(-1)^1,0 = -1, (-1)^1,1 = 1 (-1)^1,2 = -1 (-1)^1,3 = 1

(-1)^2,0 = 1, (-1)^2,1 = -1 (-1)^2,2 = 1 (-1)^2,3 = -1

(-1)^3,0 = -1, (-1)^3,1 = 1 (-1)^3,2 = -1 (-1)^3,3 = 1

Input Image

1	0	1	0
1	0	1	0
1	0	1	0
1	0	1	0

X

(-1)^(x+y)

1	-1	1	-1
-1	1	-1	1
1	-1	1	-1
-1	1	-1	1

=

1	0	1	0
-1	0	-1	0
1	0	1	0
-1	0	-1	0

Note: Its pixel-to-pixel multiplication **NOT** a matrix multiplication.

Frequency Domain Filters: Working Examples

Step 2: Compute the DFT of the image.

F(U,V) DFT = Kernel x f(x,y) x Kernel^{T}

Kernel

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

X

Input Image

1	0	1	0
-1	0	-1	0
1	0	1	0
-1	0	-1	0

X

Kernel Transpose

1	1	1	1
1	-j	-1	j
1	-1	1	-1
1	j	-1	-j

=

F(U,V) DFT =

0	0	0	0
0	0	0	0
16	0	16	0
0	0	0	0

Note: Matrix multiplication. Consider only real values.

Frequency Domain Filters: Working Examples

Step 2: Compute the distance between each value and the centre.

$\sqrt{(x-u)^2+(y-v)^2}$ u, v = 2,2

0,0	0,1	0,2	0,3
1,0	1,1	1,2	1,3
2,0	2,1	2,2	2,3
3,0	3,1	3,2	3,3

D(u,v) =

2.82	2.23	2	2.23
2.23	1.14	1	1.41
2	1	0	1
2.23	1.41	1	1.41

Filter Function H(u,v)

H(u,v) =

D0 = 0.5

1	1	1	1
1	1	1	1
1	1	0	1
1	1	1	1

Note: IHPF – Any value greater than 0.5 will be 1 else is 0.

Frequency Domain Filters: Working Examples

Step 3,4: G(u,v) = F(u,v) x H(u,v)

0	0	0	0
0	0	0	0
16	0	16	0
0	0	0	0

X

1	1	1	1
1	1	1	1
1	1	0	1
1	1	1	1

=

0	0	0	0
0	0	0	0
16	0	0	0
0	0	0	0

Note: Its pixel-to-pixel multiplication not a matrix multiplication.

Frequency Domain Filters: Working Examples

Step 5: Compute the IDFT of the image.

DFT = 1/4 (Kernel) x f(x,y) x $1/4(\text{Kernel}^{T})$

1	1	1	1
1	j	-1	-j
1	-1	1	-1
1	-j	-1	j

X

0	0	0	0
0	0	0	0
16	0	0	0
0	0	0	0

X

1	1	1	1
1	j	-1	-j
1	-1	1	-1
1	-j	-1	j

=

1/16

16	16	16	16
-16	-16	-16	-16
16	16	16	16
-16	-16	-16	-16

=

1	1	1	1
-1	-1	-1	-1
1	1	1	1
-1	-1	-1	-1

Note: Matrix multiplication.

Frequency Domain Filters: Working Examples

Step 6: Multiply the output image by (-1)^x+y to shift the Centre from (2,2) to (0,0)

1	1	1	1
-1	-1	-1	-1
1	1	1	1
-1	-1	-1	-1

X

(-1)^(x+y)

1	-1	1	-1
-1	1	-1	1
1	-1	1	-1
-1	1	-1	1

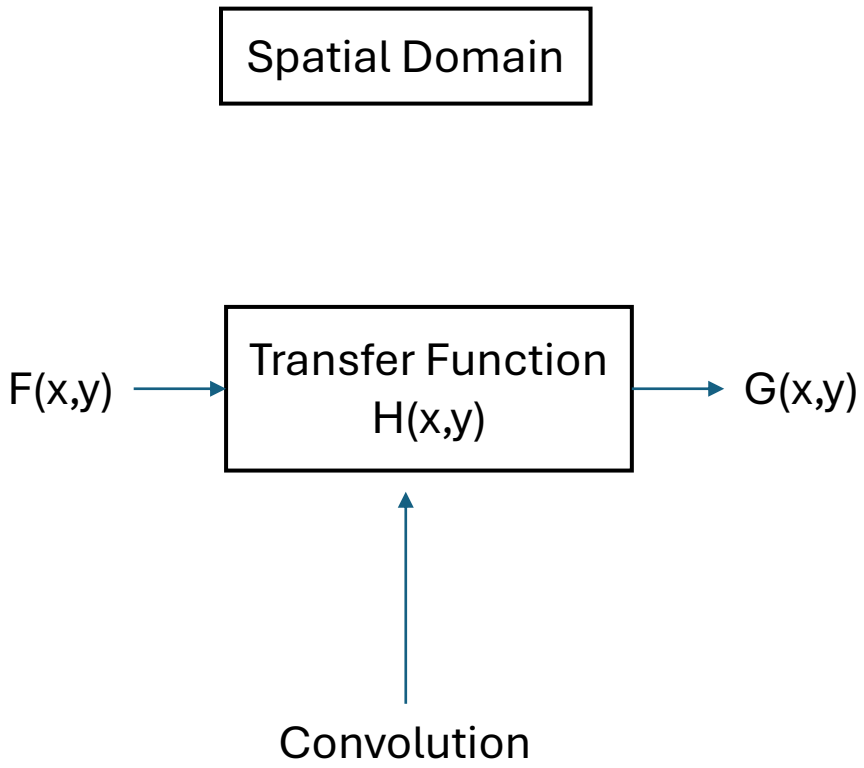
=

Final Output =

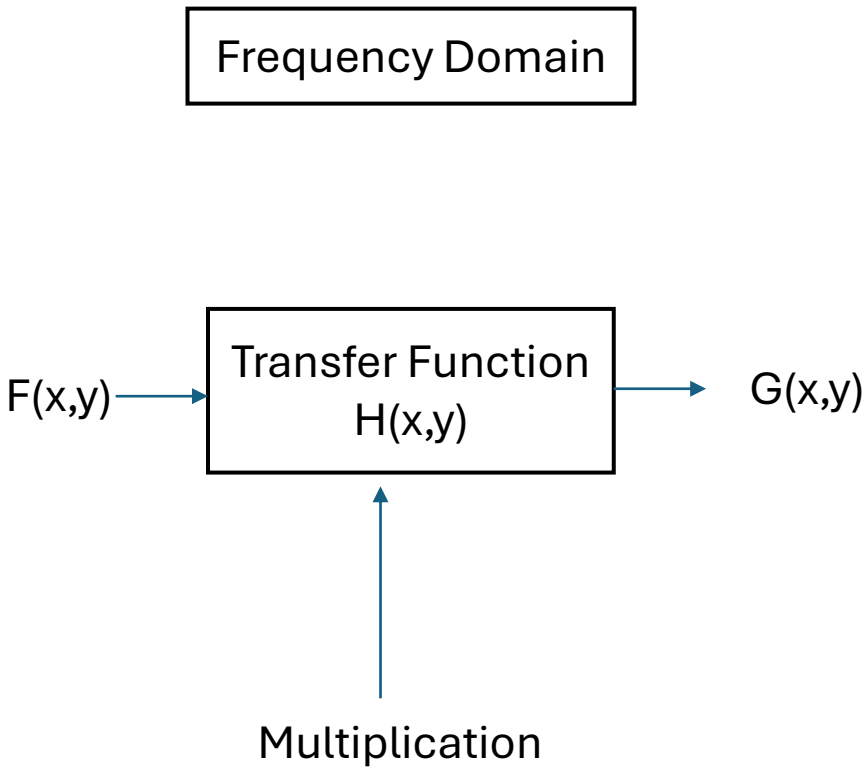
1	-1	1	-1
1	-1	1	-1
1	-1	1	-1
1	-1	1	-1

Note: Its pixel-to-pixel multiplication **NOT** a matrix multiplication.

Spatial domain and Frequency Domain Filters Analogy



G(x,y) = F(x,y) convolution H(x,y)



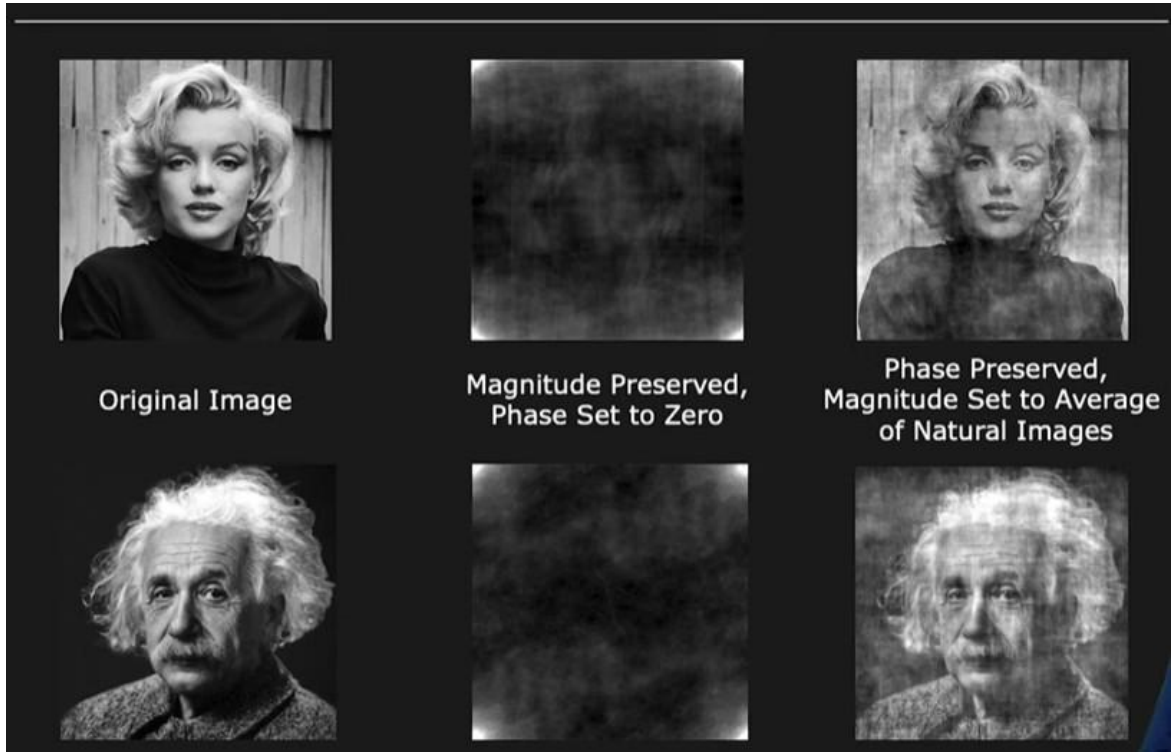
G(x,y) = F(x,y) Multiplication H(x,y)

Difference between spatial domain and frequency domain.

Feature	Spatial Domain Enhancement	Frequency Domain Enhancement
Domain of Operation	Image plane (pixels)	Frequency components (Fourier transform)
Primary Operations	Direct pixel manipulation, neighborhood filtering	Filtering in the frequency spectrum
Common Techniques	Histogram equalization, spatial filtering (smoothing, sharpening)	Low-pass filtering (blurring), high-pass filtering (sharpening), homomorphic filtering
Best For	Localized tasks (e.g., edge detection)	Global tasks (e.g., removing periodic noise)
Computational Speed	Fast for small kernels, slow for large kernels	Slower due to transforms for small kernels, faster for large kernels
Intuition	High (direct pixel manipulation)	Low (abstract frequency components)

Frequency domain: The process requires two computationally intensive steps: a forward Fourier transform and an inverse Fourier transform.

Frequency Domain Filtering: Application

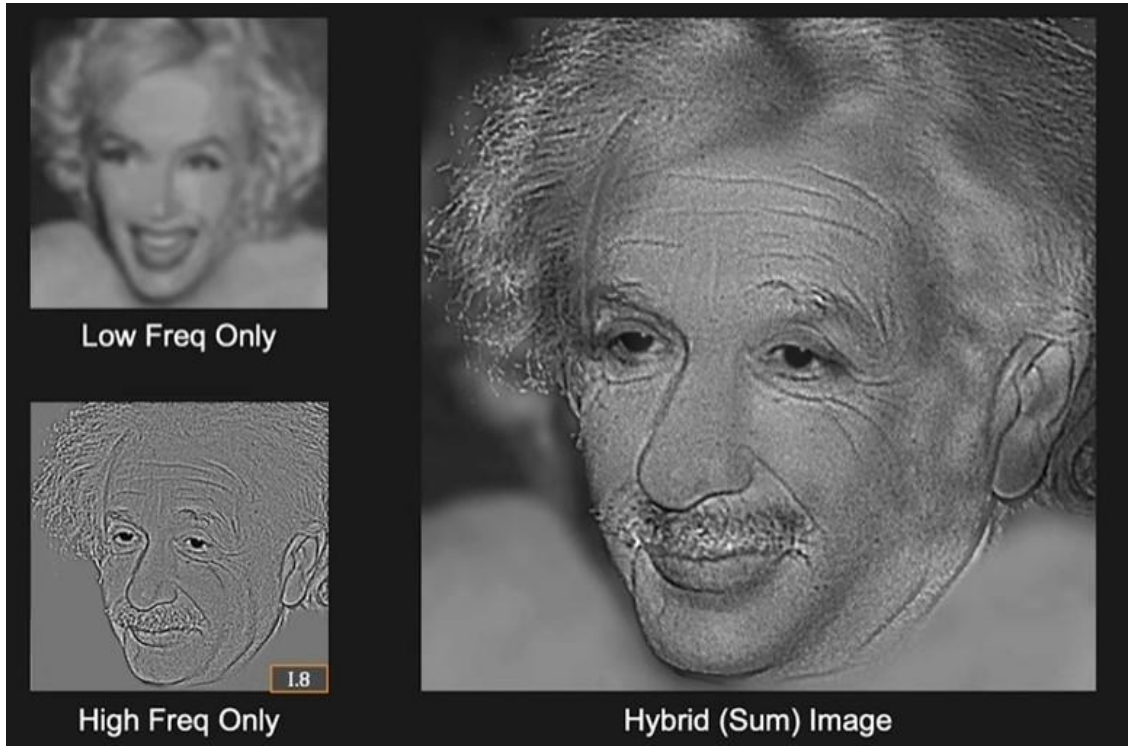


Importance of phase information

Image Morphing

Blur

Edges

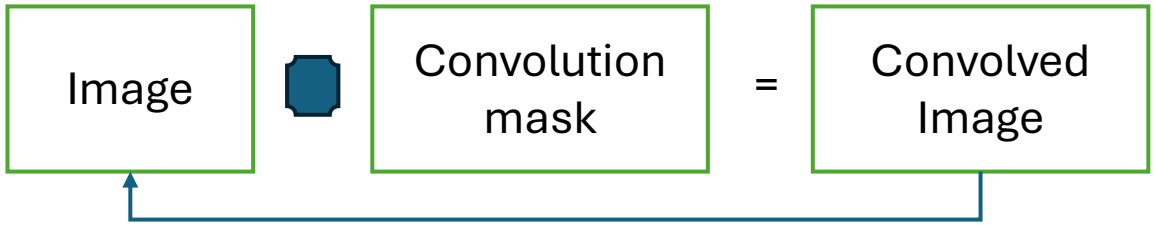


From closer look you will see albert Einstein

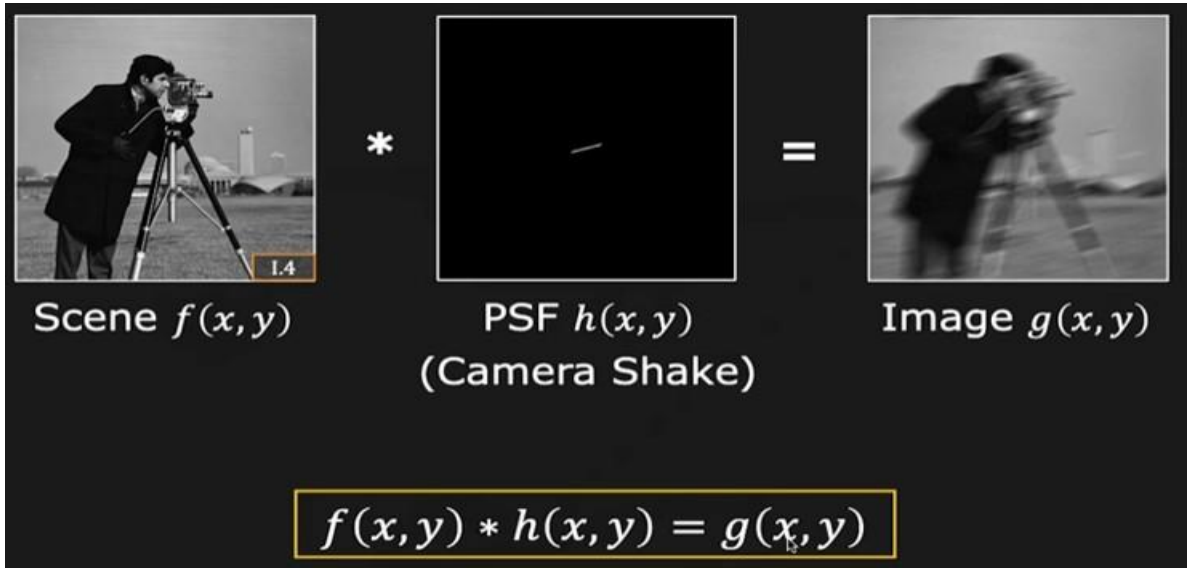
From far away you will see Marilyne.

When you close to the image, you could able to see high Details inside the image but as you see the same image from Far away you may loose high detail and you can see only low Frequency component.

Deconvolution in the frequency domain

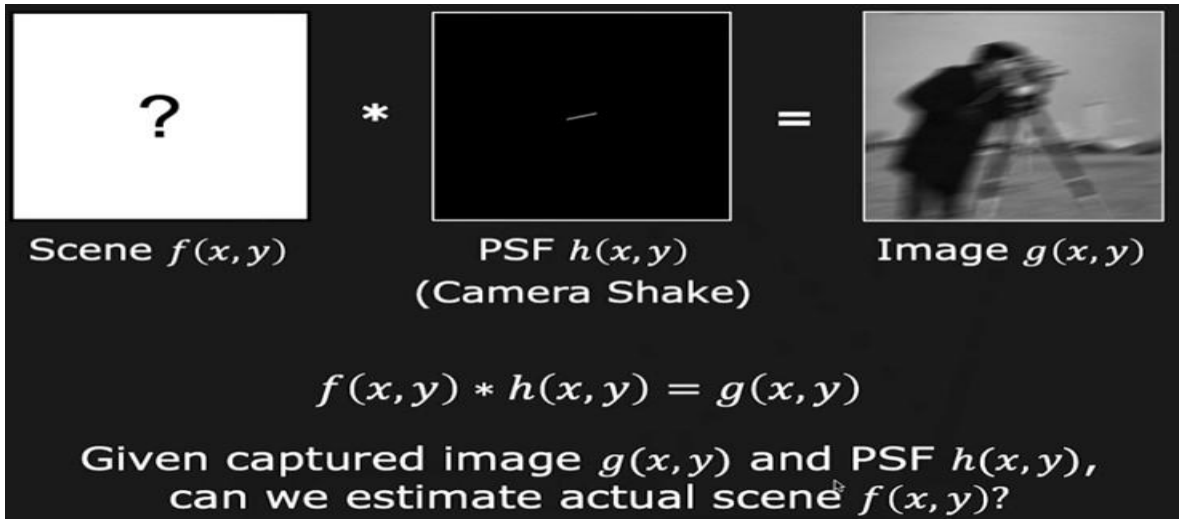


Deconvolution is the process to undo the convolution.



Take an example of the motion blur.

A original image convolved with point Spread function to get the blurred image.



How to find the PSF ?

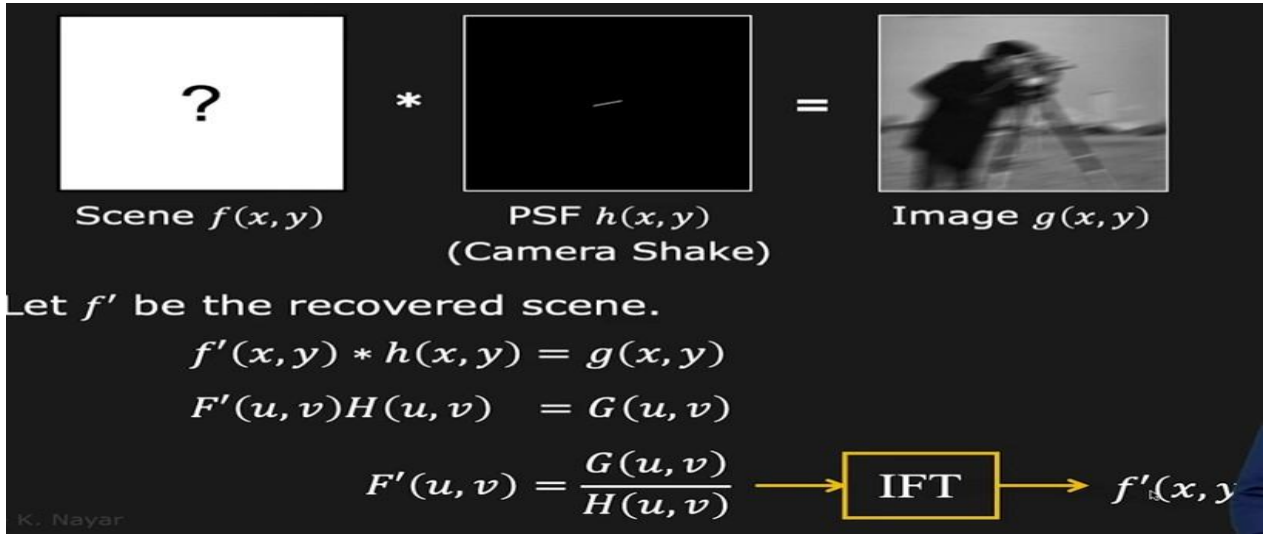
Smartphone camera contains some kind of measurement units -
Like -
Inertial measurement unit (IMU)
Accelerometer.
Gyroscope.
Use all of them to capture motion of the camera and with the help of all these parameters, generate the PSF.

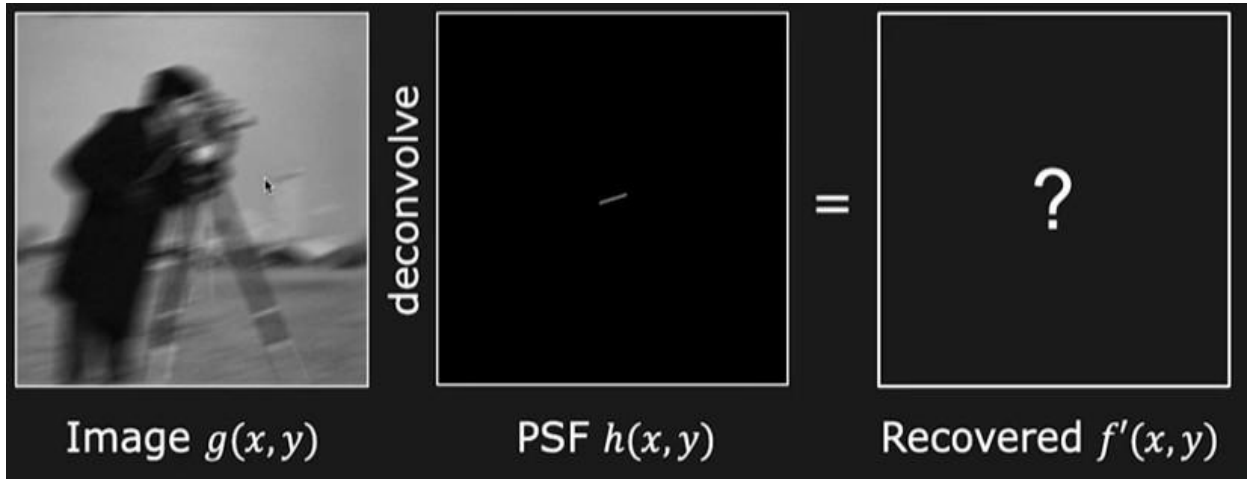
Deconvolution can be performed in better way in the frequency domain.

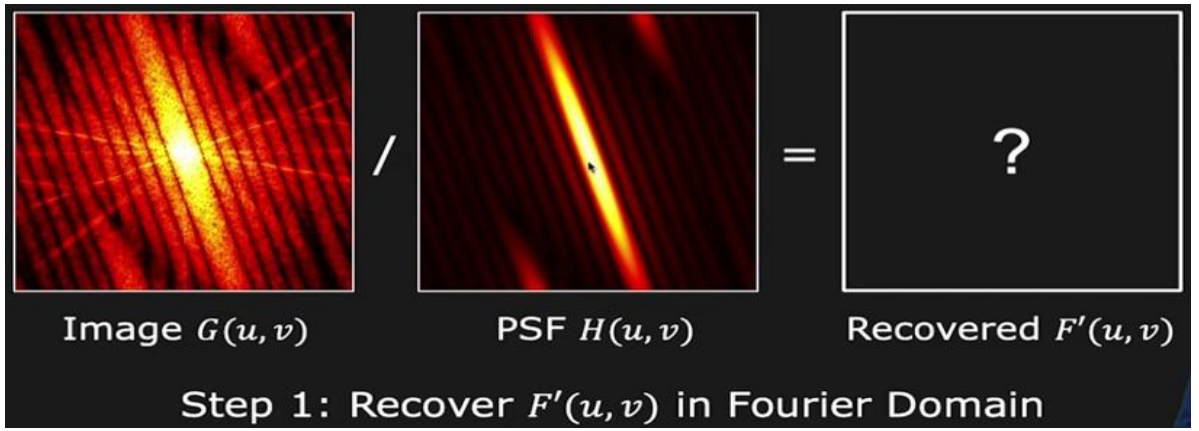
In the frequency domain, Fourier transform can help to recover the original image.

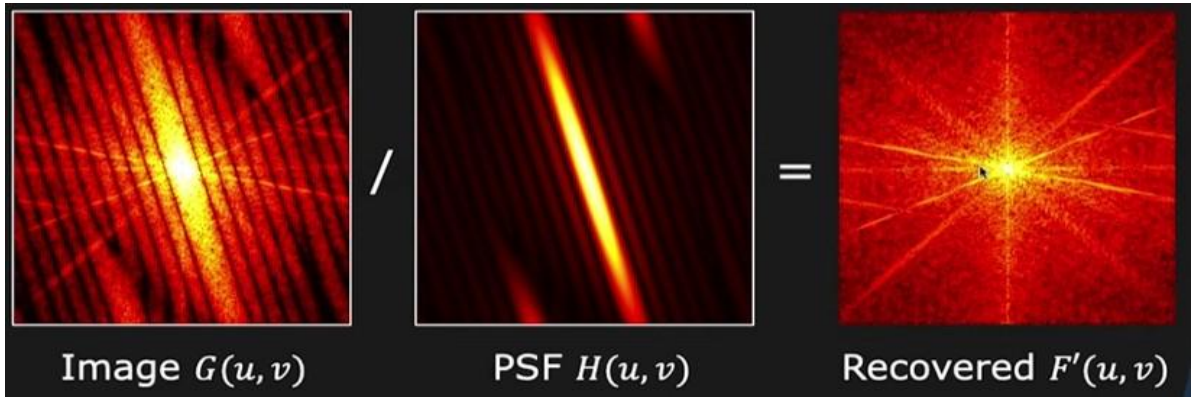
Deconvolution in the frequency domain

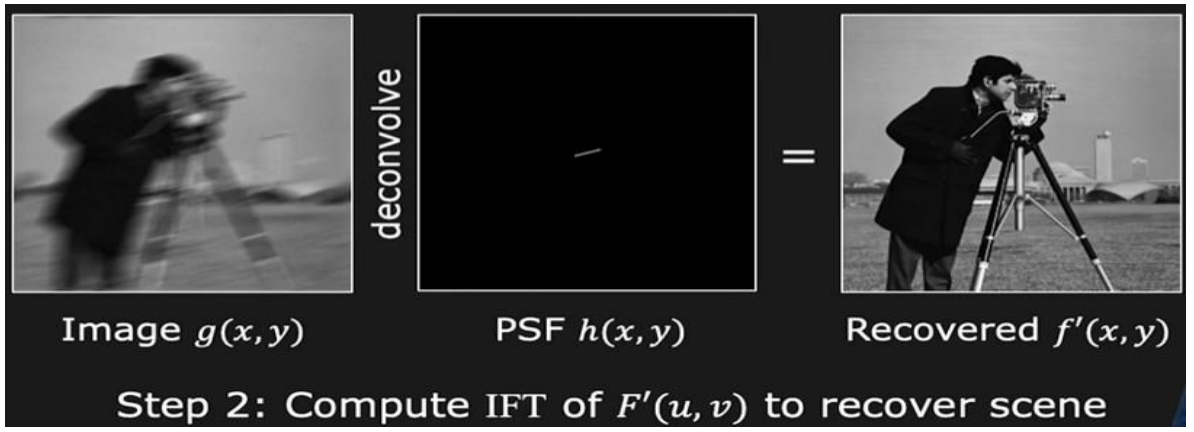
Here, you want to recover f'.
Take the FT and then find the F' and by performing IFT, you will get the original image i.e. (f').





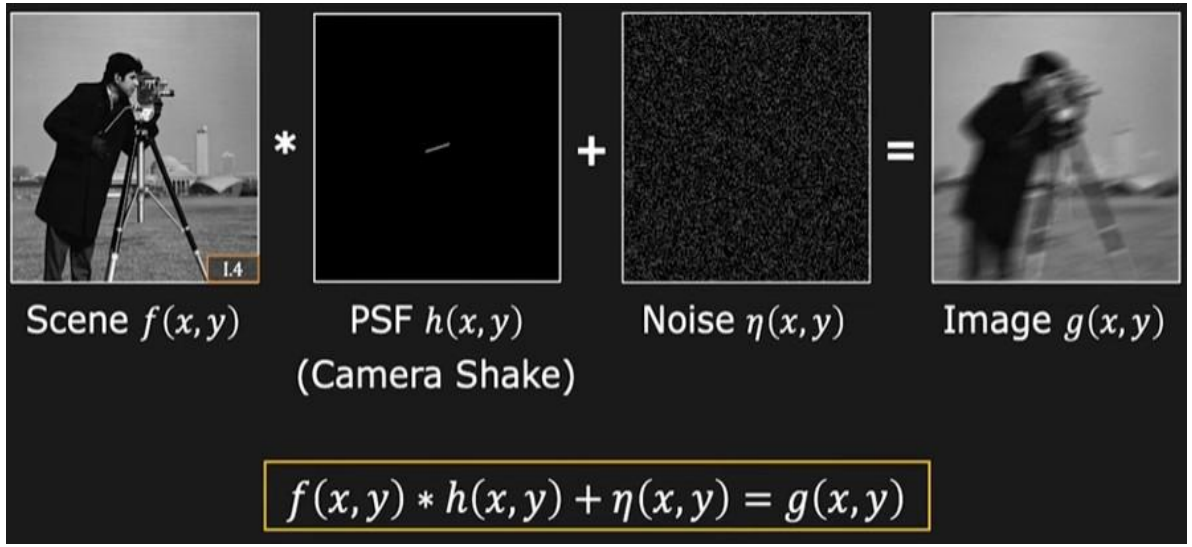




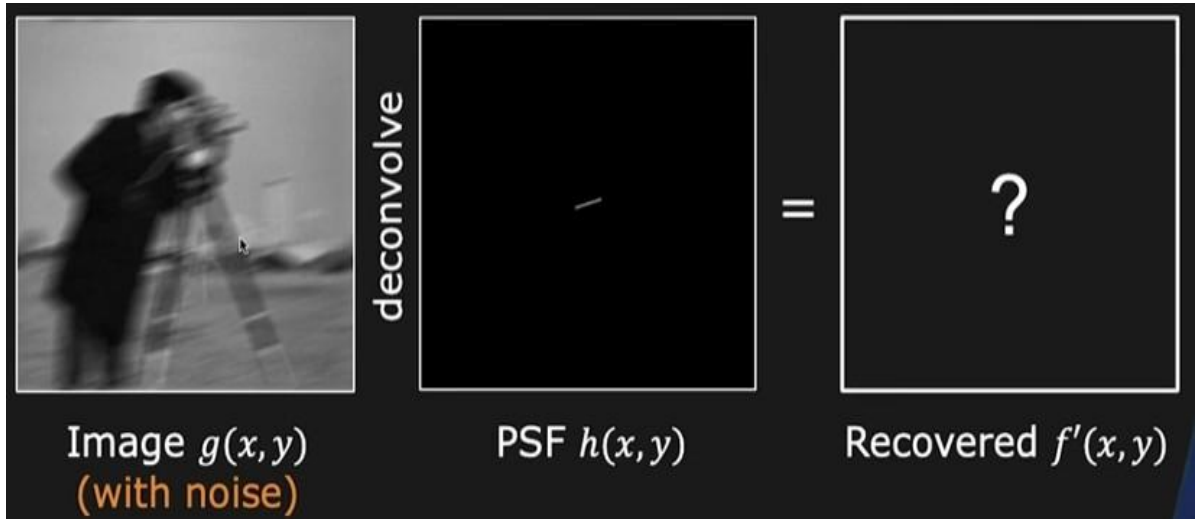


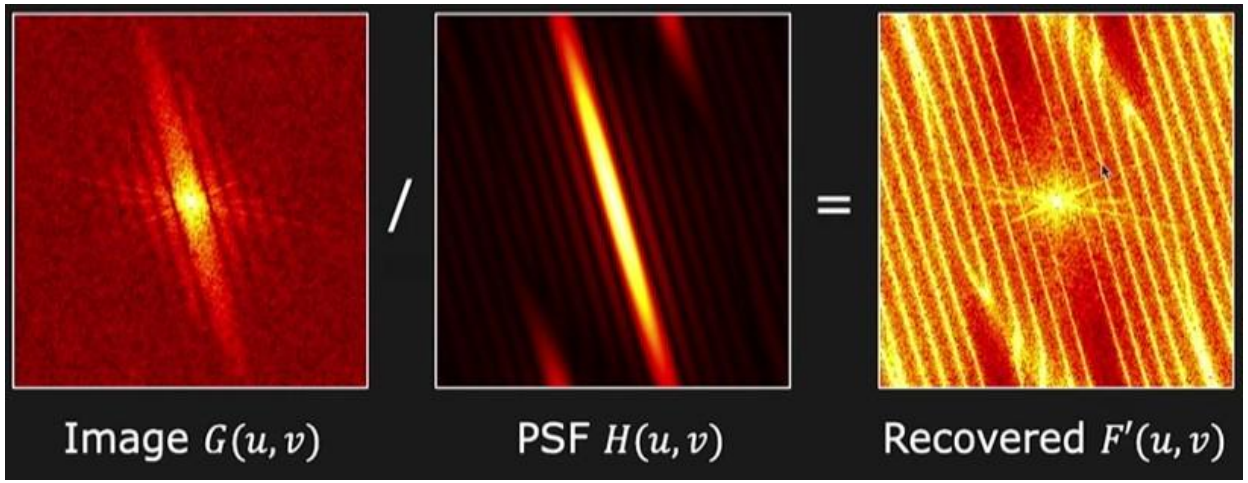
Deconvolution in the frequency domain

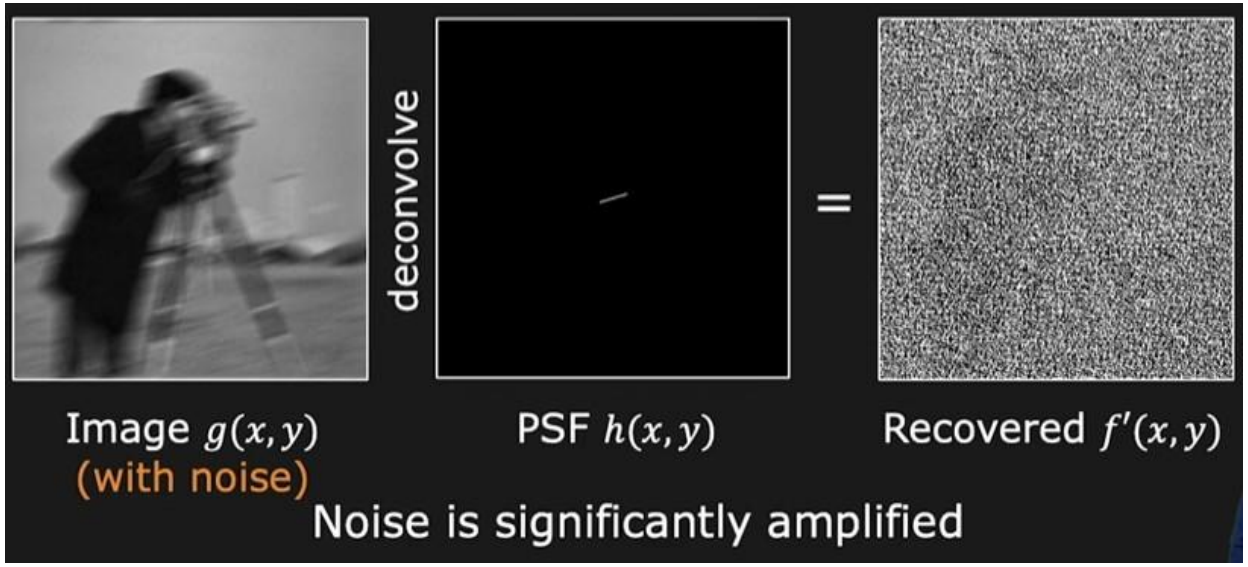
But we have some problem. The problem is related to the noise. After camera shake a noise has been added in the image.



Can we afford to avoid noise ?







Our original image is destroyed because of noise.

Deconvolution: Issues

$$\frac{G(u,v)}{H(u,v)} = F'(u,v) \rightarrow \boxed{\text{IFT}} \rightarrow f'(x,y)$$

1. Where $H(u,v) = 0$, $F'(u,v) = \infty \rightarrow$ Not recoverable

2. Motion blur filter $H(u,v)$ is a low pass filter.

   For high frequencies (u,v):
   - Noise $N(u,v)$ in $G(u,v)$ is high
   - Filter $H(u,v) \approx 0$

   } Noise in $G(u,v)$ is amplified

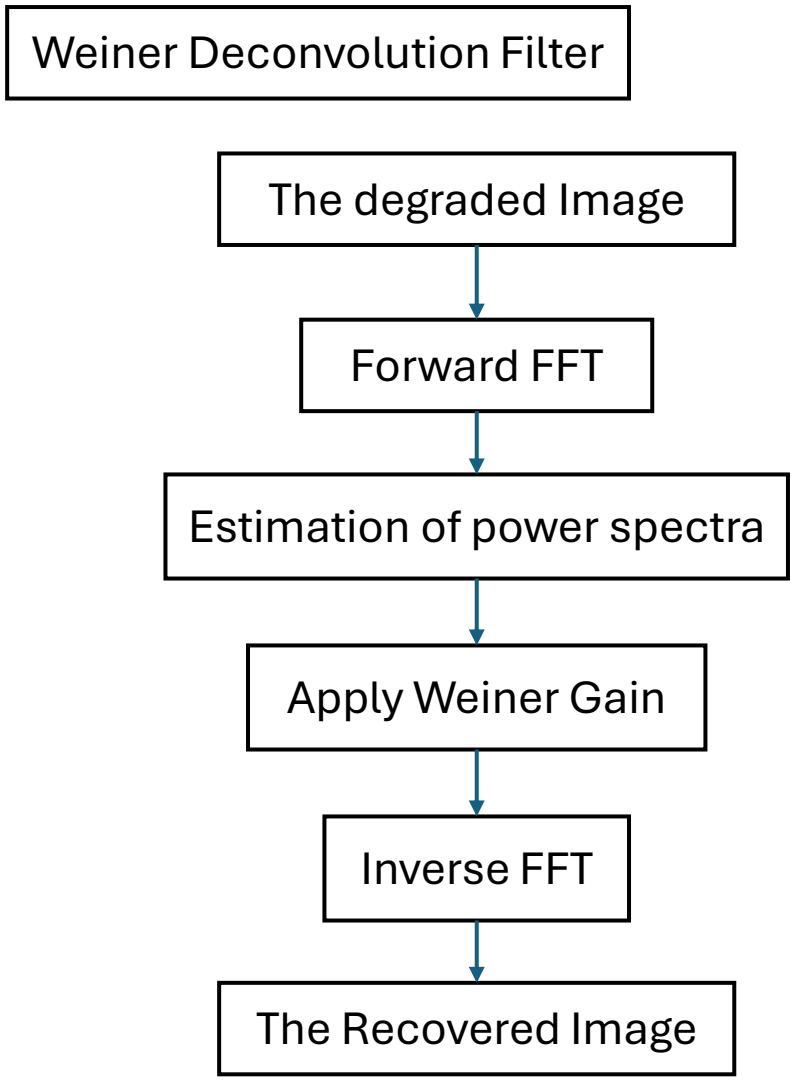
We need some kind of Noise Suppression.

Here, H(u,v) = Fourier transform of point spread function.
Basically H(u,v) is a low pass filter.

Let's say noise is present in the image. If it is high noise, then you will get more high frequencies in the image.

As we mentioned, Motion blur filter H(u.v) is a low pass filter so that output of it is approx. zero. Result in, F'(u,v) = infinite i.e. not recoverable.

Solution : During deconvolution process we need certain kind of noise suppression.



Weiner filter used to minimize the square distance between the Estimated and true image.

CSET340

Advanced Computer Vision and Video Analytics

9th Feb. to 13th Feb. 2026

Overall Course Coordinator-
Dr. Gaurav Kumar Dashondhi
Gaurav.dashondhi@bennett.edu.in

Note : Any query related to course then first connect with overall course coordinator.

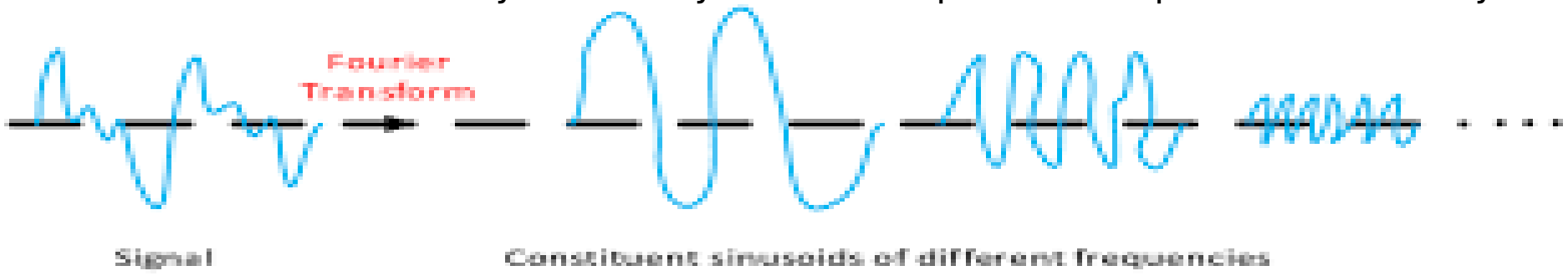
Discrete Wavelet Transform

Wavelet – (Mother Wavelet) Think of it as a small, brief oscillation (a "little wave") that starts at zero, grows, and then dies back down to zero.

Unlike a **Sine Wave** (used in Fourier Transform) which goes on forever, a Wavelet is **localized** in both time and frequency.

The FT tells you which frequencies (notes) are present in the signal, but it has no idea **when** they happened.

The WT provides a multi-resolution analysis. It tells you which frequencies are present **and** exactly where they are located



Discrete Wavelet Transform

A wavelet is a mathematical function used to divide a signal or image into different frequency components.

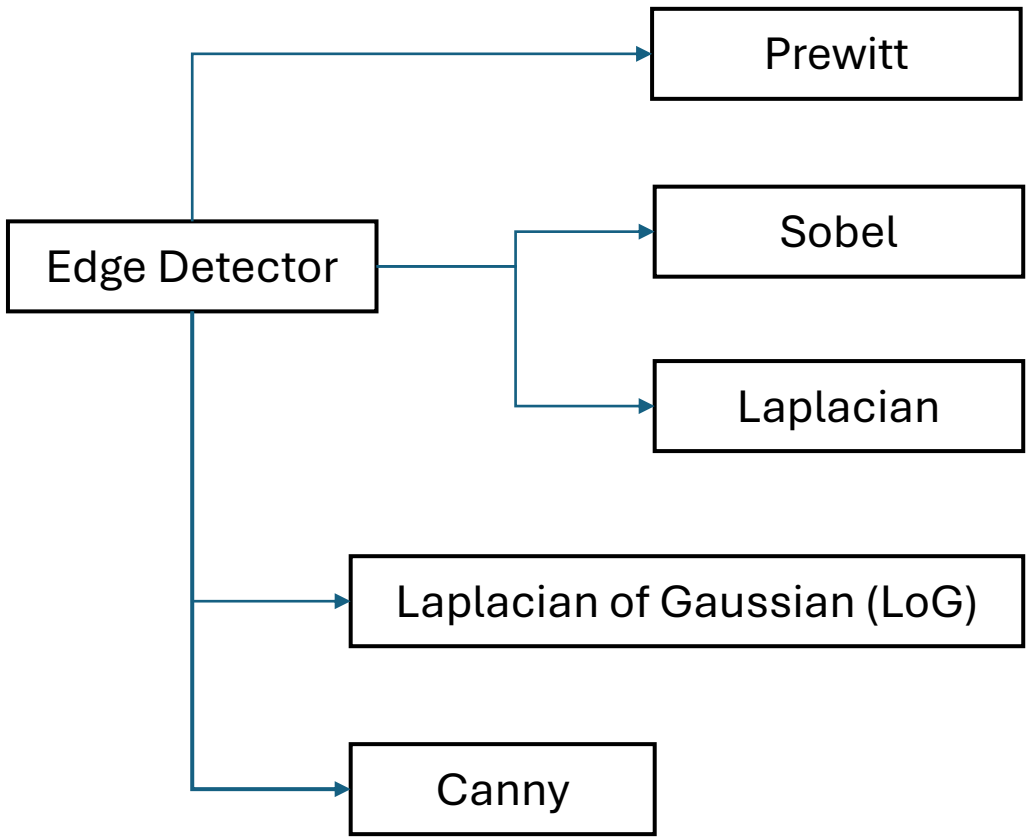
It has two main properties that you can manipulate:

•**Scaling (Width):** Stretching or squishing the wavelet to look for "big" patterns or "tiny" details.

•**Shifting (Position):** Moving the wavelet along the signal to find exactly *where* a feature occurs.

Feature	Fourier Transform (FT)	Wavelet Transform (WT)
Basic Building Block	Sine waves (Infinite length)	Wavelets (Short, finite pulses)
Information Provided	Frequency only	**Time + Frequency**
Resolution	Constant (Global)	**Variable** (Multi-resolution)
Best For...	Stationary signals (like a constant hum)	Non-stationary signals (spikes, edges, pulses)
Image Use	General frequency filtering	**Compression (JPEG 2000) and Denoising**

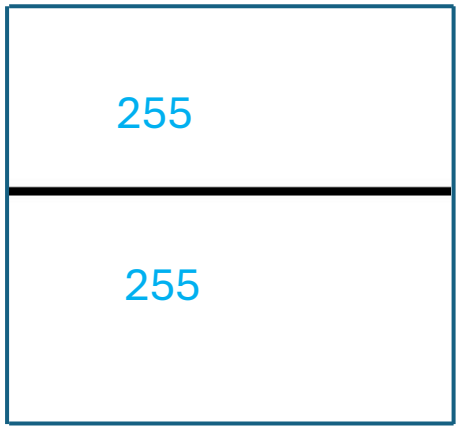
Edge Detection



What is an Edge, Line and Point

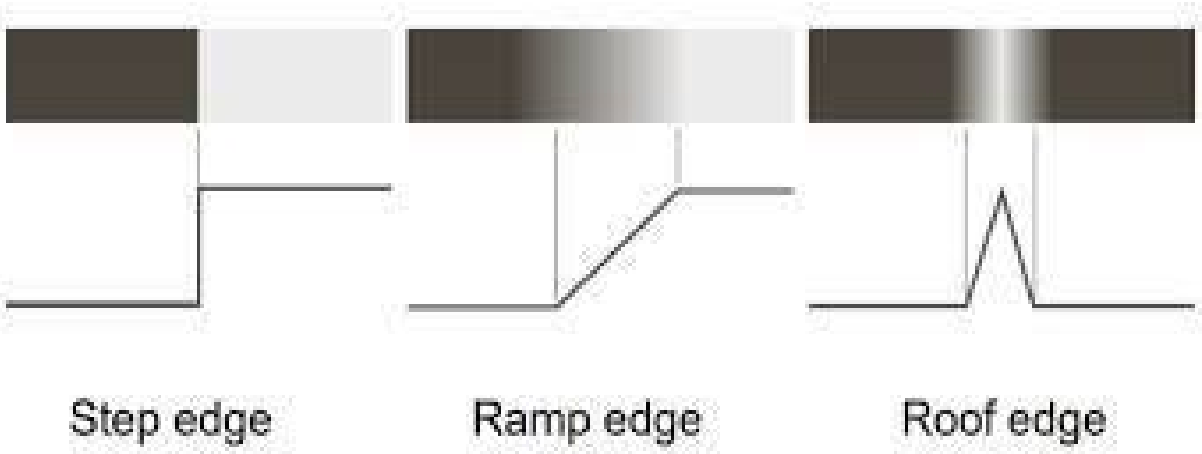
Edge: Edge pixels are the pixels at which intensity of an image changes abruptly.

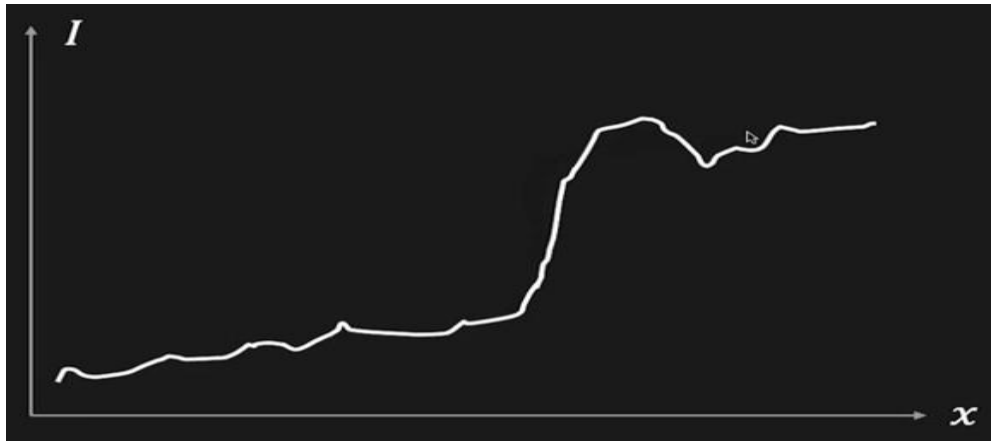
Line: it may be viewed as a thin edge segment where intensity of background on either side of line is either much higher or much lower.



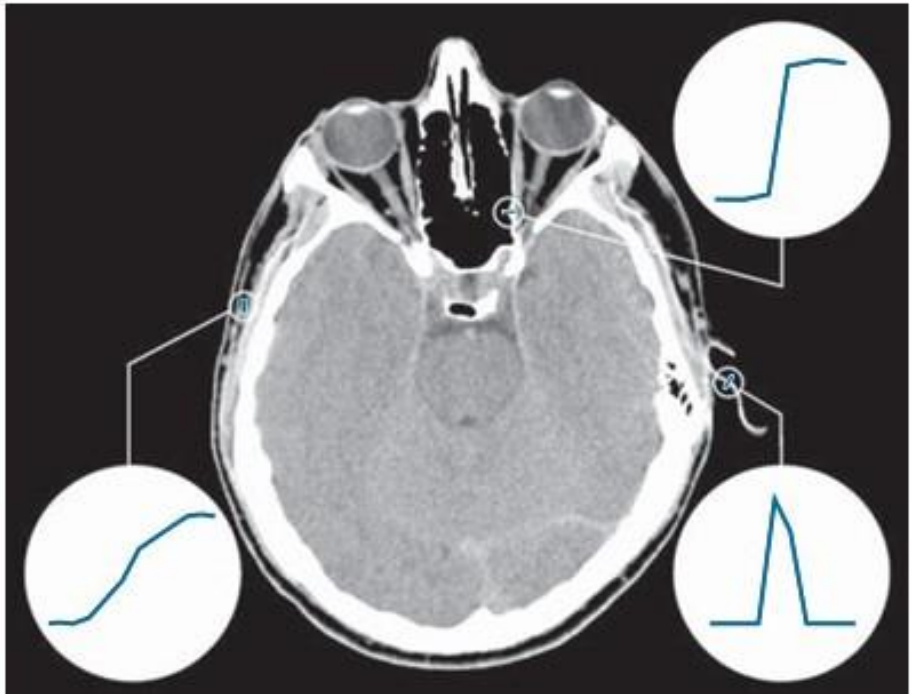
Point: It may be viewed as foreground pixel surrounded by background and Vice-Versa.

Types of Edges





Real edge or Real step edge –
As analog signal need to pass through **sampling, quantization and then Noise** is also attached.

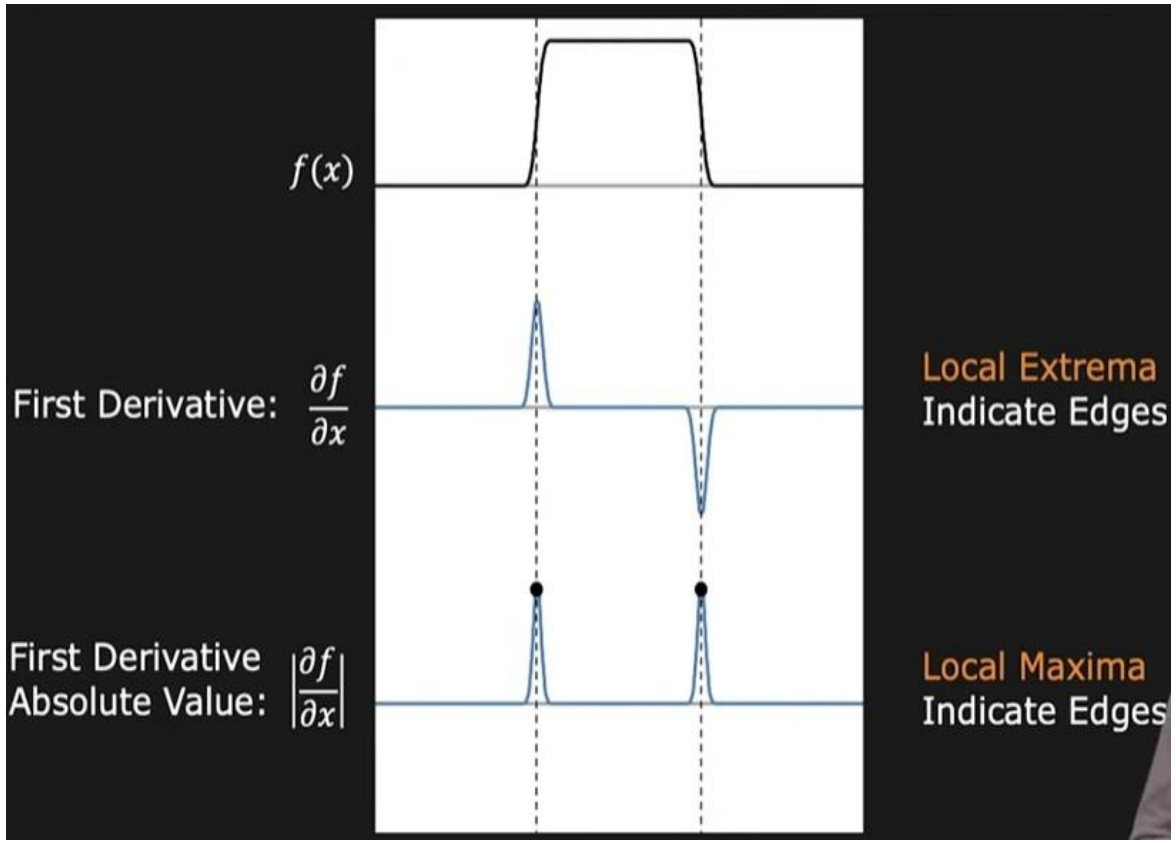


Averaging or smoothening is analogous to Integration.

Derivatives could be able to detect the abrupt change in intensity.

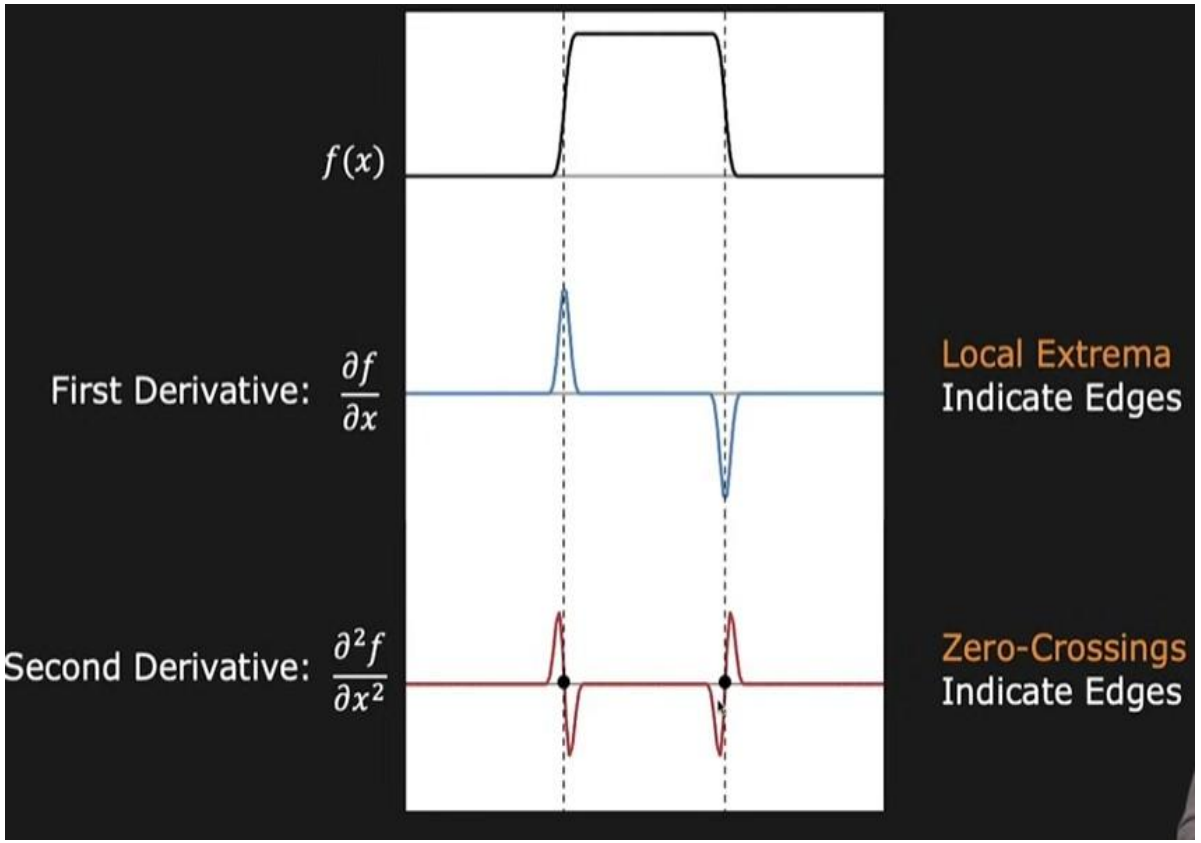
Edge detection using first and second derivative

Edge Detection using first derivative.



Local extrema will talk about the position of an edge and its amplitude helps to understand the magnitude of the edge.

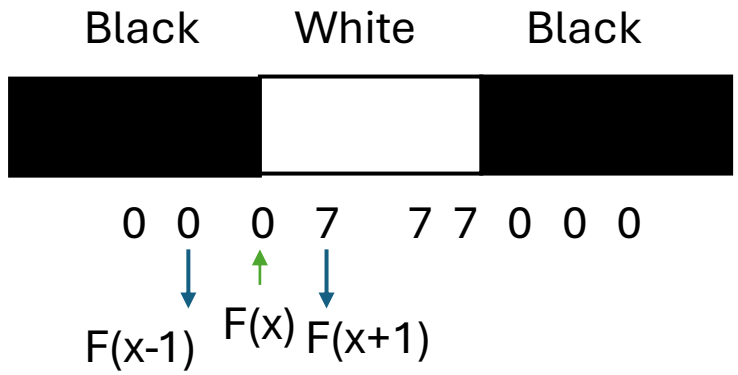
Edge Detection using second derivative.



Instead of Local extrema, **zero crossing** will talk about the position of an edge,

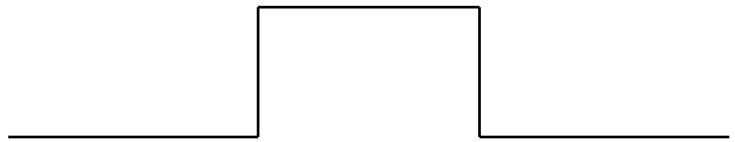
Edge

Sample Image



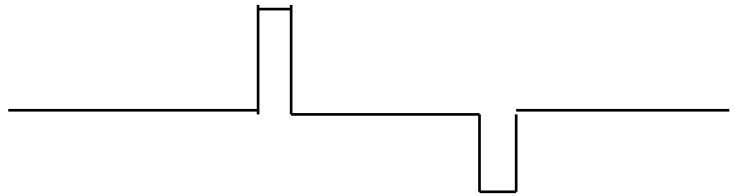
Pixel intensity values

Step Edge



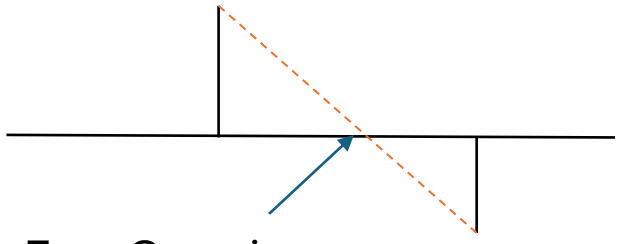
First Derivative

$$\frac{\partial f(x)}{\partial x} = f'(x) = \frac{f(x+1) - f(x-1)}{2}$$



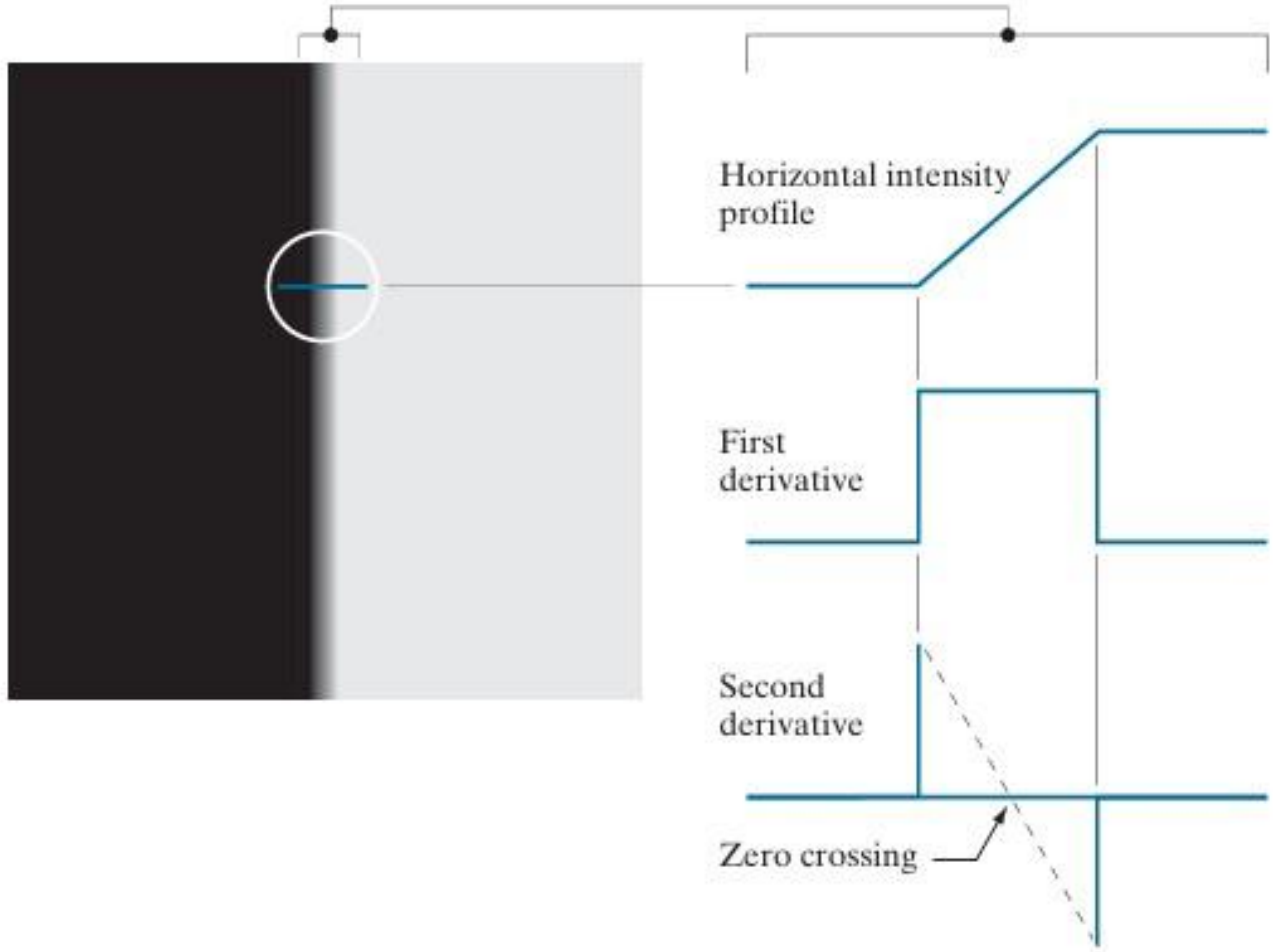
Second Derivative

$$\frac{\partial^2 f(x)}{\partial x^2} = f''(x) = f(x+1) - 2f(x) + f(x-1)$$



Zero Crossing used to identify the location of an edge.

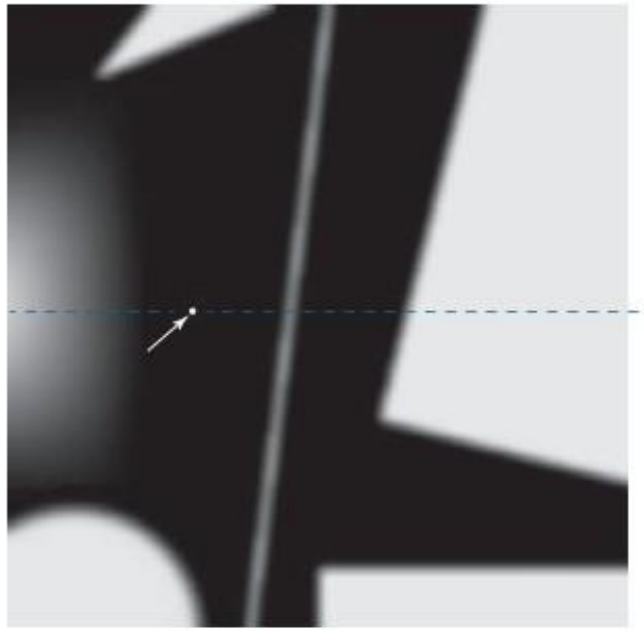
Edge



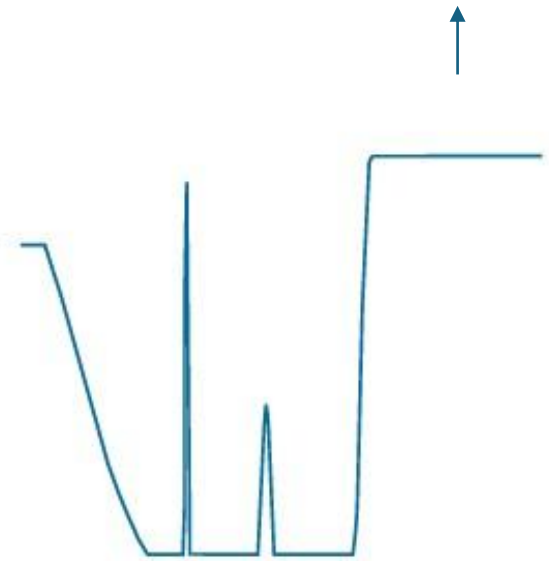
Horizontal intensity profile of the blurred edge.

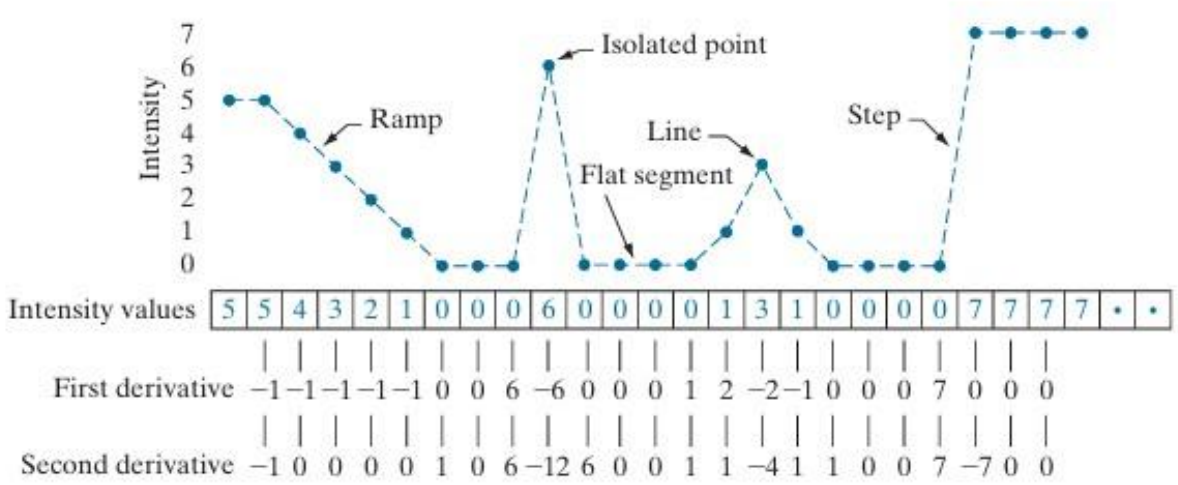
Second derivative will help to identify the location of an edge.

Edge



Intensity profile of the different edges.





Calculation of first and second derivative

$$\frac{\partial f(x)}{\partial x} = f'(x) = \frac{f(x+1) - f(x-1)}{2}$$

$$\frac{\partial^2 f(x)}{\partial x^2} = f''(x) = f(x+1) - 2f(x) + f(x-1)\,1)$$

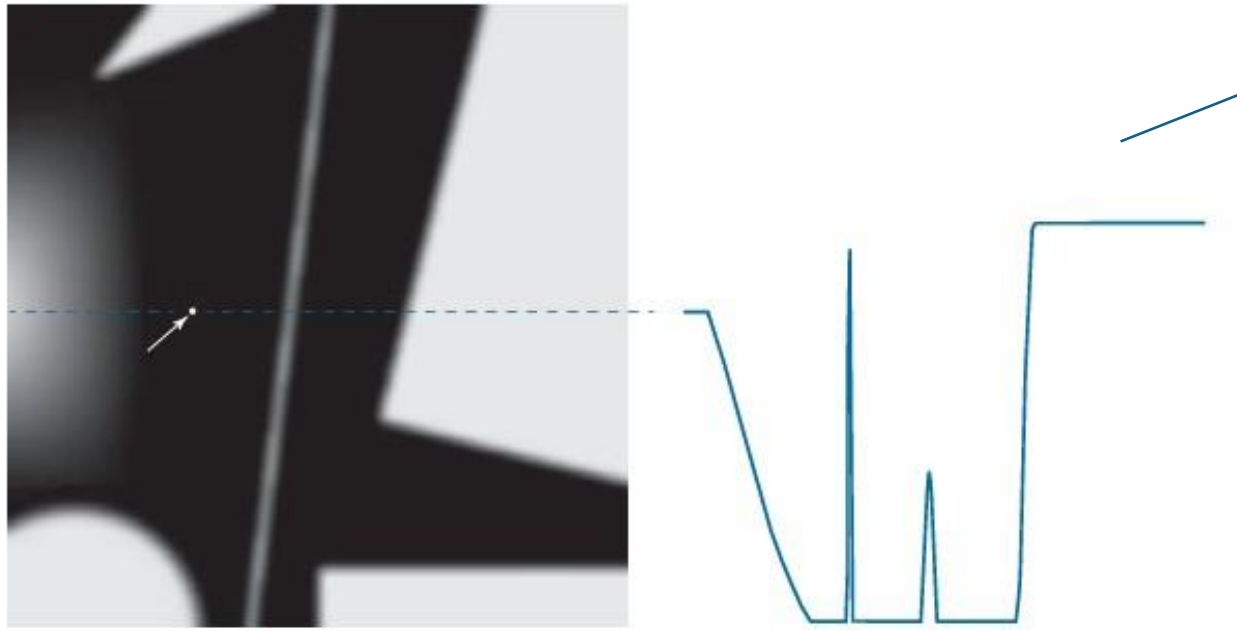
Take a sequence - 5 5 4 3 2 1

F(x-1) F(x) F(x+1)

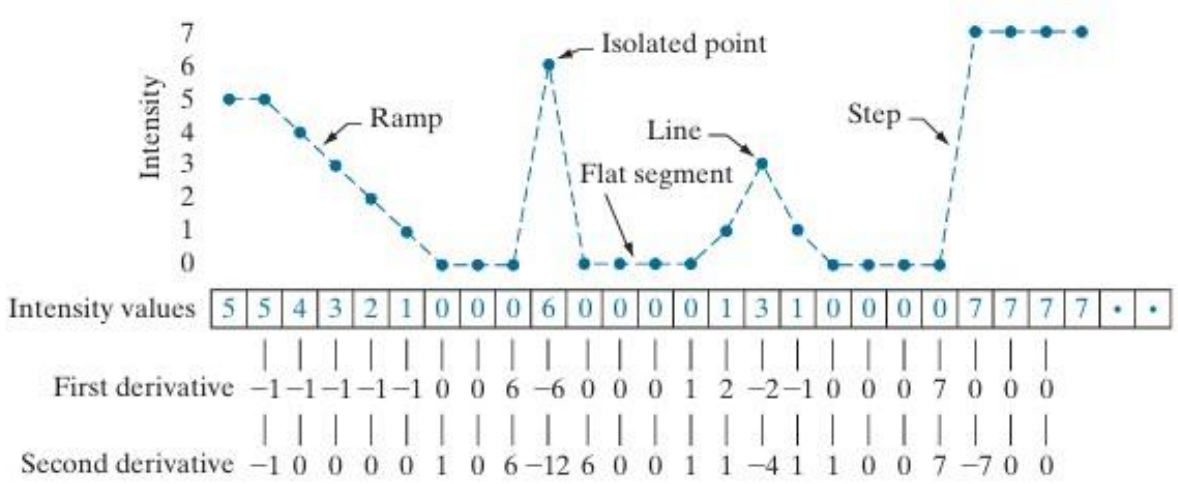
First derivative = (3-5)/2 = -1

Second derivative = 3-(2x4)+5 = 0

Edge



Intensity profile of the different edges.



Edge	First Order	Second Order
Ramp	Non Zero (Thick Edge)	Non zero at onset and end of the ramp (Thin Edge)
Point	Magnitude Less	Magnitude High
Line	Magnitude Less	Magnitude High

1. In case of ramp and step edge both the second derivative has opposite sign.
2. If second derivative negative : Transition from light to dark.
3. If second derivative positive : Transition from dark to light.

Edge

Observations

1. First order derivative produces thicker edges.
2. Second order derivative have the stronger response for point, thin lines and noise.
3. Second order derivative produces double edge response at ramp and step transition in intensity.
4. The sign of second derivative used to determine whether a transition into an edge from light to dark or dark to light.

What exactly we want from an edge detector ?

We want an Edge Operator that produces:

- Edge Position
- Edge Magnitude (Strength)
- Edge Orientation (Direction)

Performance Requirements:

- High Detection Rate
- Good Localization
- Low Noise Sensitivity

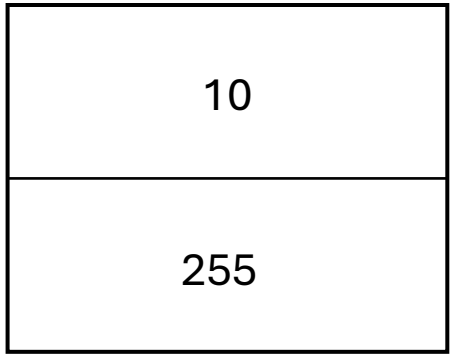
Edge Detection

Image Gradient and its properties

Edge : An Abrupt or Sudden change in intensity which will help to identify the edges.

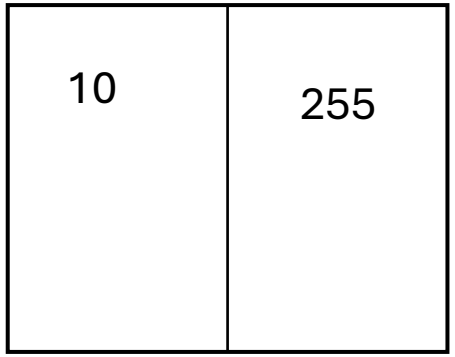
Derivatives could be able to detect the abrupt change in the intensity.

Horizontal Edge



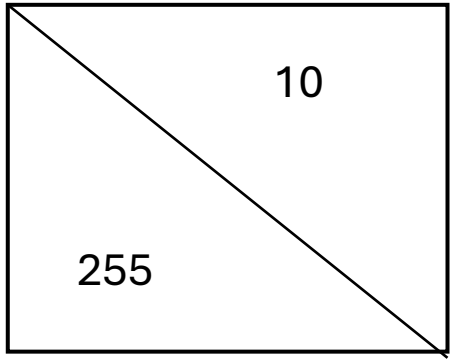
10	10	10
0	0	0
255	255	255

Vertical Edge



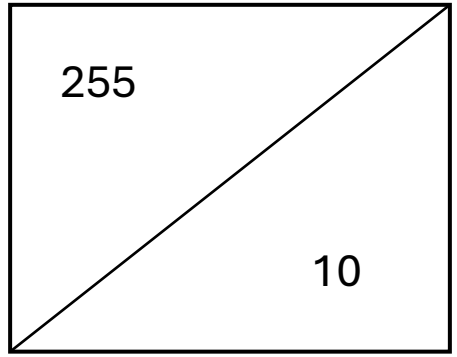
10	0	255
10	0	255
10	0	255

Right or Principal Diagonal



0	10	10
255	0	10
255	255	0

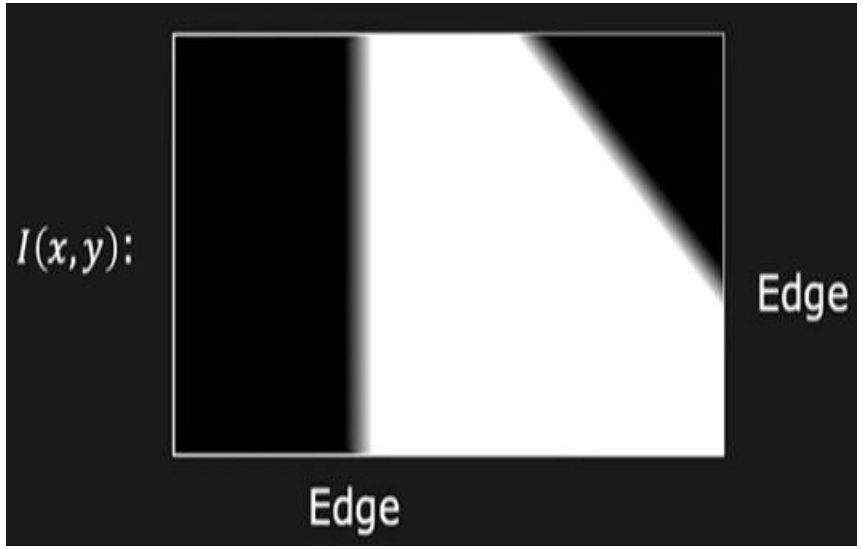
Left Diagonal



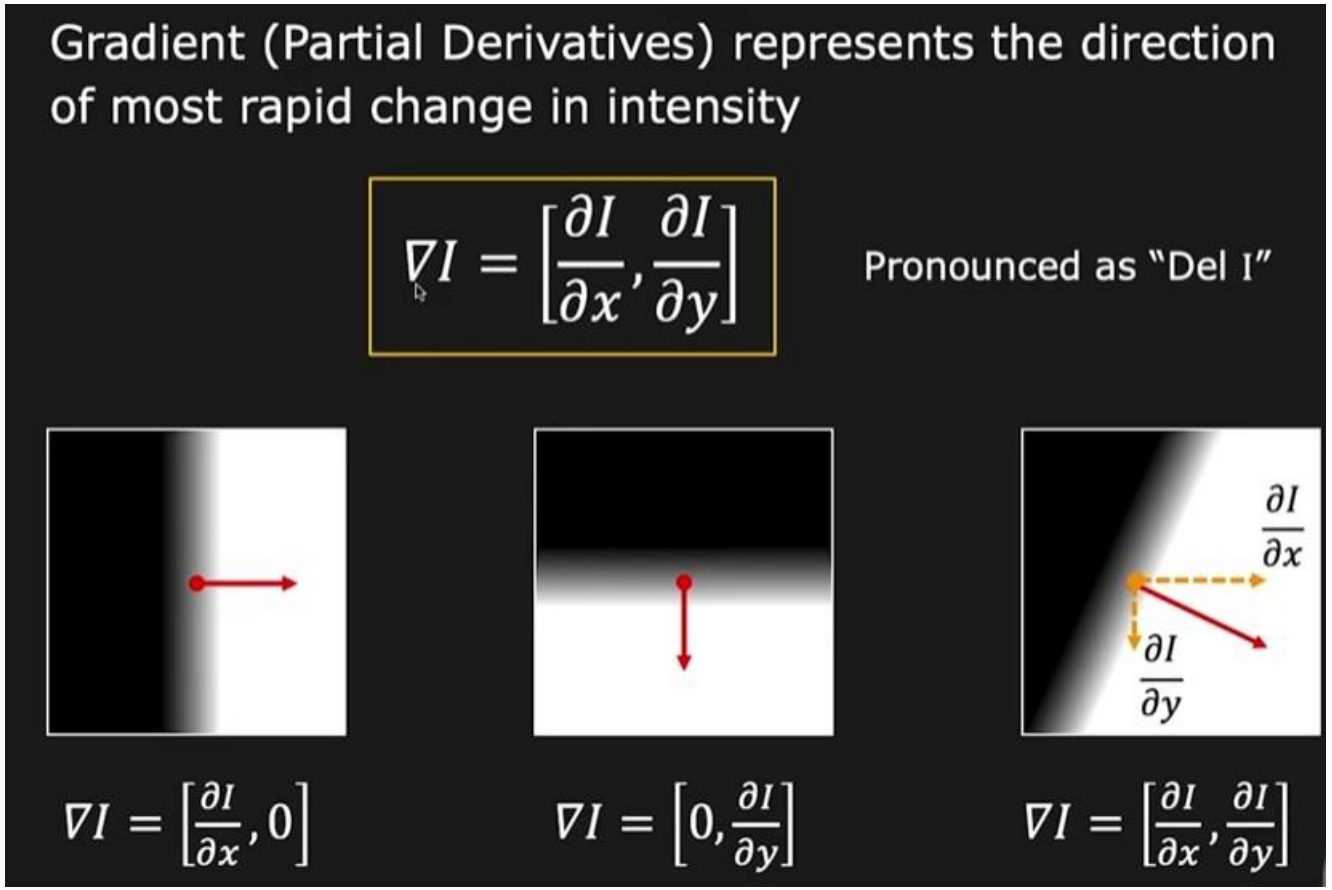
255	255	0
255	0	10
0	10	10

Edge Detection in 2D

2D Edge detection



Partial derivatives of 2D continuous function represent the amount of change along each dimension.



It shows change in x direction.

It shows change in y direction.

It shows change in x and y both the direction.

Edge : Magnitude and Direction.

Image Gradient is a tool to identify edge Magnitude and Direction.

Gradient Vector

$$\nabla f(x,y) \equiv \text{grad}[f(x,y)] \equiv \begin{bmatrix} g_x(x,y) \\ g_y(x,y) \end{bmatrix} = \begin{bmatrix} \frac{\partial f(x,y)}{\partial x} \\ \frac{\partial f(x,y)}{\partial y} \end{bmatrix}$$

Magnitude of Gradient Vector

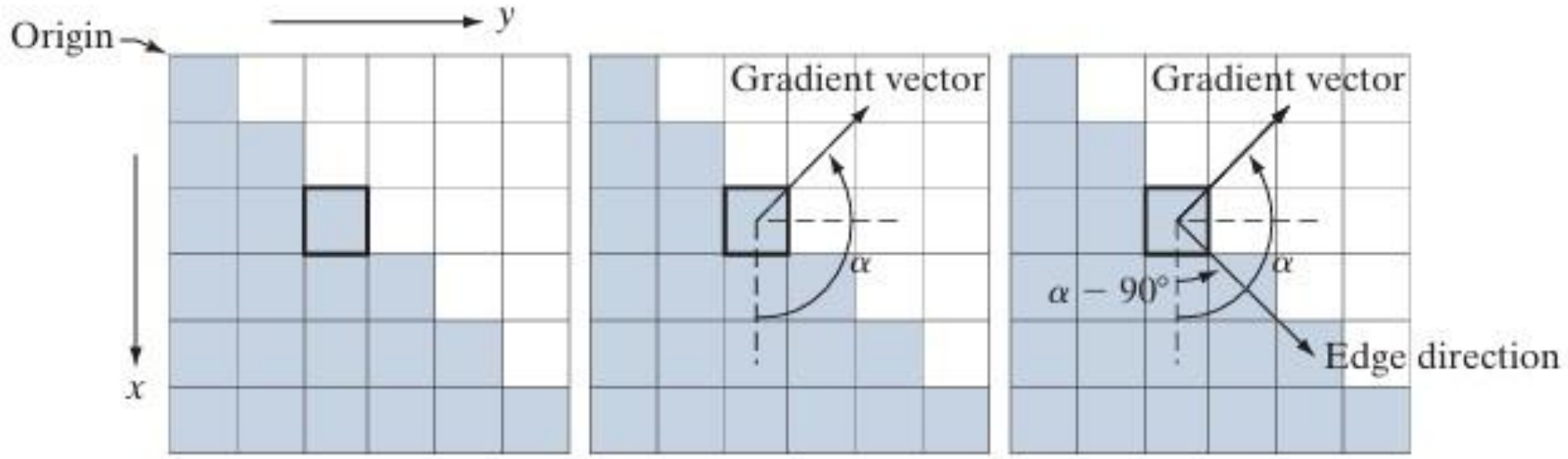
$$M(x,y) = \|\nabla f(x,y)\| = \sqrt{g_x^2(x,y) + g_y^2(x,y)}$$

Direction of Gradient Vector

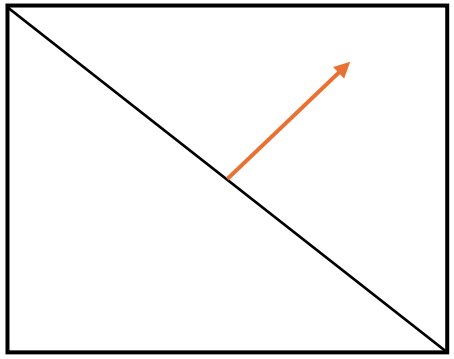
$$\alpha(x,y) = \tan^{-1}\left[\frac{g_y(x,y)}{g_x(x,y)}\right]$$

Edge Detection

Image Gradient and its properties



0	1	1
0	0	1
0	0	0



Z1	Z2	Z3
Z4	Z5	Z6
Z7	Z8	Z9

y

x

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Prewitt and Sobel Edge Detectors

Prewitt Edge Detector

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Sobel Edge Detector

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

Note: Sum of all kernel coefficient is equal to zero.

Prewitt Edge Detectors example

Prewitt Edge Detector

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Input Image

50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50

Find the edge for green and red pixels

Z1	Z2	Z3
Z4	Z5	Z6
Z7	Z8	Z9

$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Calculation for all green pixels

Gx = (50+50+50) – (50+50+50) = 0
Gy = (50+50+50) – (50+50+50) = 0

Note: If gradient magnitude is zero it means it's a Constant intensity region.

Prewitt Edge Detectors example

Prewitt Edge Detector

-1	-1	-1
0	0	0
1	1	1

-1	0	1
-1	0	1
-1	0	1

Input Image

50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50

Find the edge for green and red pixels

Z1	Z2	Z3
Z4	Z5	Z6
Z7	Z8	Z9

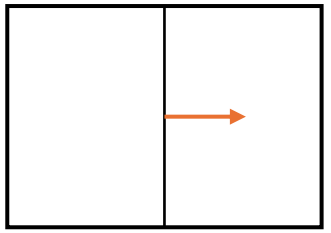
$$g_x = \frac{\partial f}{\partial x} = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Calculation for all red pixels

Gx = (50+50+150) – (50+50+150) = 0
Gy = (150+150+150) – (50+50+50) = 300

Note: change in horizontal direction it means there is any edge in vertical direction.



$$\alpha(x,y) = \tan^{-1}\left[\frac{g_y(x,y)}{g_x(x,y)}\right]$$ = tan^-1(300/0) = 90degree

Vertical edge, gradient in horizontal direction.

Sobel Edge Detectors example

Sobel Edge Detector

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

Input Image

50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50

Find the edge for green and red pixels

Z1	Z2	Z3
Z4	Z5	Z6
Z7	Z8	Z9

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

Calculation for all green pixels

Gx = (50+100+50) – (50+100+50) = 0
Gy = (50+100+50) – (50+1000+50) = 0

Note: If gradient magnitude is zero it means it's a Constant intensity region.

Sobel Edge Detectors example

Sobel Edge Detector

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

Input Image

50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50
50	50	50	150	50	50	50

Find the edge for green and red pixels

Z1	Z2	Z3
Z4	Z5	Z6
Z7	Z8	Z9

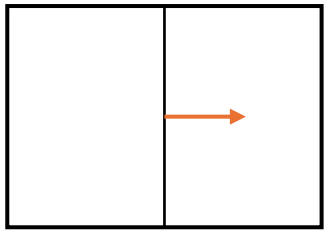
$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

Calculation for all red pixels

Gx = (50+100+150) – (50+100+150) = 0
Gy = (150+300+150) – (50+50+50) = 450

Note: change in horizontal direction it means there is any edge in vertical direction.



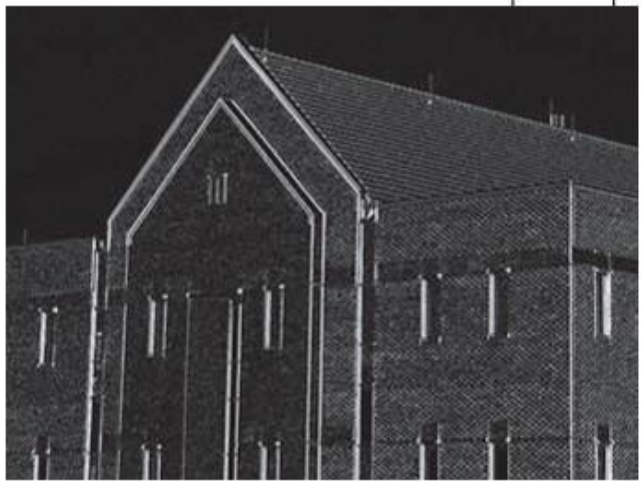
$$\alpha(x,y) = \tan^{-1}\left[\frac{g_y(x,y)}{g_x(x,y)}\right]$$ = tan^-1(450/0) = 90degree

Vertical edge, gradient in horizontal direction.

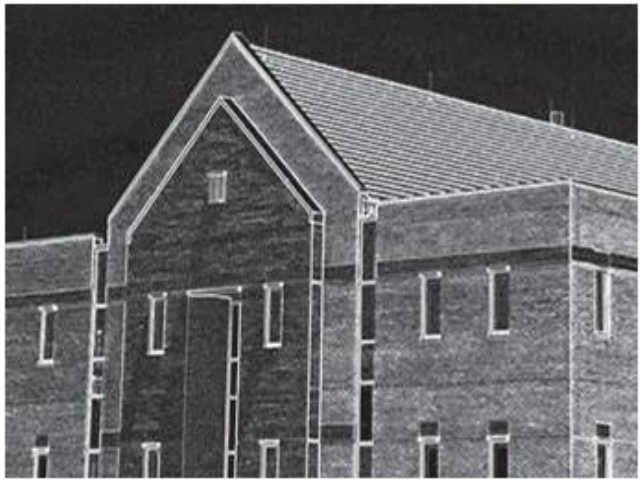
Sobel Edge Detectors

Original Image

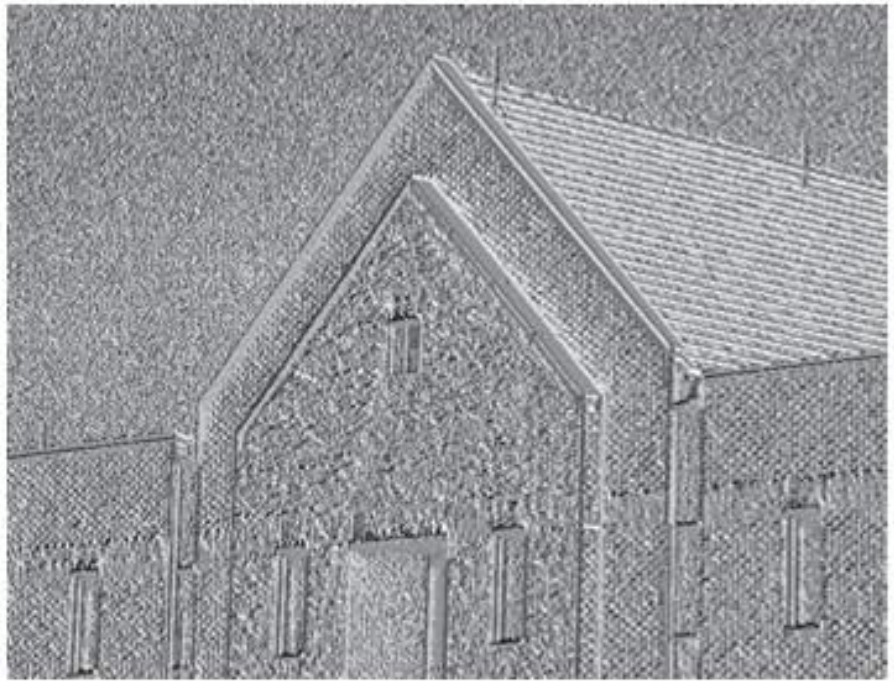
Edge in x direction



Edge in y direction



Gradient Image



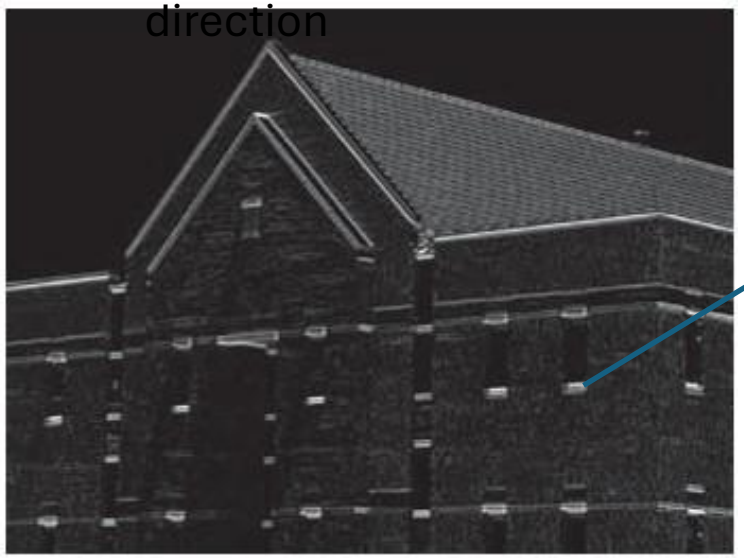
Gradient Angle Image

Note: It generally not contain too much information but it complement the information extracted from image gradient.

Smooth image by 5 by 5 kernel and then apply Sobel Edge Detectors

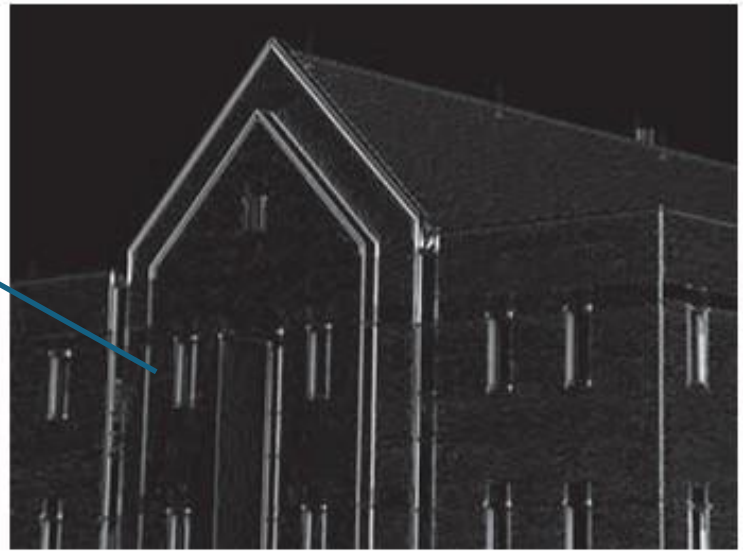
Original Image

Gradient in x direction



Horizontal edge

Vertical edge



Gradient Image = Gradient in x direction + Gradient in y direction

Gradient in y direction

Comparing Different Gradient Operators

Gradient	Roberts	Prewitt	Sobel (3x3)	Sobel (5x5)
$\frac{\partial I}{\partial x}$	$\begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & -2 & 0 & 2 & 1 \\ -2 & -3 & 0 & 3 & 2 \\ -3 & -5 & 0 & 5 & 3 \\ -2 & -3 & 0 & 3 & 2 \\ -1 & -2 & 0 & 2 & 1 \end{bmatrix}$
$\frac{\partial I}{\partial y}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$	$\begin{bmatrix} 1 & 2 & 3 & 2 & 1 \\ 2 & 3 & 5 & 3 & 2 \\ 0 & 0 & 0 & 0 & 0 \\ -2 & -3 & -5 & -3 & -2 \\ -1 & -2 & -3 & -2 & -1 \end{bmatrix}$

Good Edge Localization
High Noise Sensitivity
Poor Detection

Poor Edge Localization
Less Noise Sensitivity
Good Detection

As the kernel size increases…

Combining Gradient with Thresholding

if pixel value > = 33% of the maximum value of gradient image

Output : White pixel

else:

Output: Black Pixel

Gradient Image

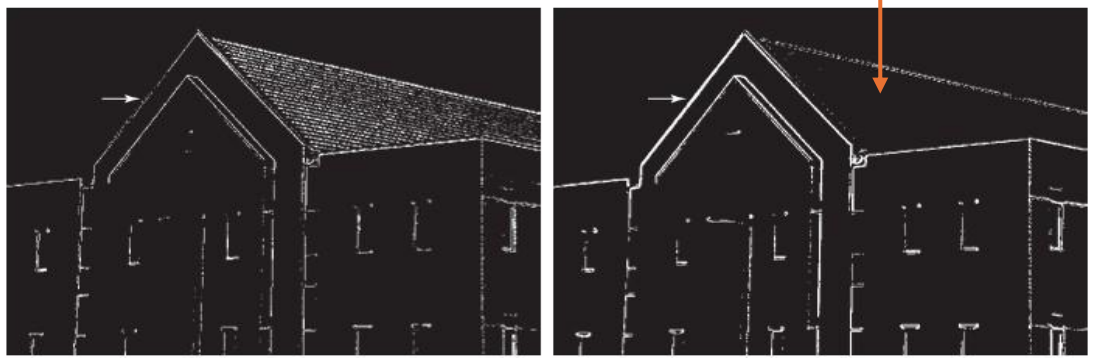
10	02	72
80	40	42
15	30	50
30	9	100

Thresholded output Image

0	0	1
1	1	1
0	0	1
0	0	1

Note: Maximum value is 100, 33% of maximum value is 33.

Thresholding helps to avoid the Minor Edges.



Threshold applied on Gradient image

Threshold applied on **Smoothed** Gradient image

Note: When interest lies in high-lightening the principal edge and maintaining as much connectivity as possible, it is common practice to use **both smoothening and thresholding**.

Sharpening (High-pass) Spatial filters

Image Blurring or averaging or smoothening or low pass filtering in the spatial domain is analogous to the integration.

Sharpening or highpass filtering can be accomplished as spatial differentiation.

Image differentiation enhances the edges other discontinuities (like noise) and de-emphasize the area with slowly varying intensities.

Derivatives of digital function are define in terms of differences -

1. Averaging or smoothening is analogous to Integration.
2. Derivatives could able to detect the abrupt change in the intensity.
3. In case of digital Images: Derivative is defined as finite difference
4. Approximation used for the **first** derivatives:
    - 4.1 It must be zero in the area of constant intensity.
    - 4.2 It must be non zero at the onset and end of an intensity step or ramp.
    - 4.3 It must be **non zero** at points along an intensity ramp.
5. Approximation used for the **Second** derivatives:
    - 5.1 It must be zero in the area of constant intensity.
    - 5.2 It must be non zero at the onset and end of an intensity step or ramp.
    - 5.3 It must be **zero** at points along an intensity ramp.

Use of second derivative for image sharpening: The Laplacian

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\frac{\partial^2 f}{\partial x^2} = f(x+1,y) + f(x-1,y) - 2f(x,y)$$

1 -2 1 In x direction

$$\frac{\partial^2 f}{\partial y^2} = f(x,y+1) + f(x,y-1) - 2f(x,y)$$

1 -2 1 In y direction

combine

$$\nabla^2 f(x,y) = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$$

$$g(x,y) = f(x,y) + c\left[\nabla^2 f(x,y)\right]$$

Where
F(x,y) is the original image
g(x,y) is sharped image
C = -1 when kernel centre is negative
C = 1 when kernel centre is positive

kernel

0	1	0
1	−4	1
0	1	0

Or

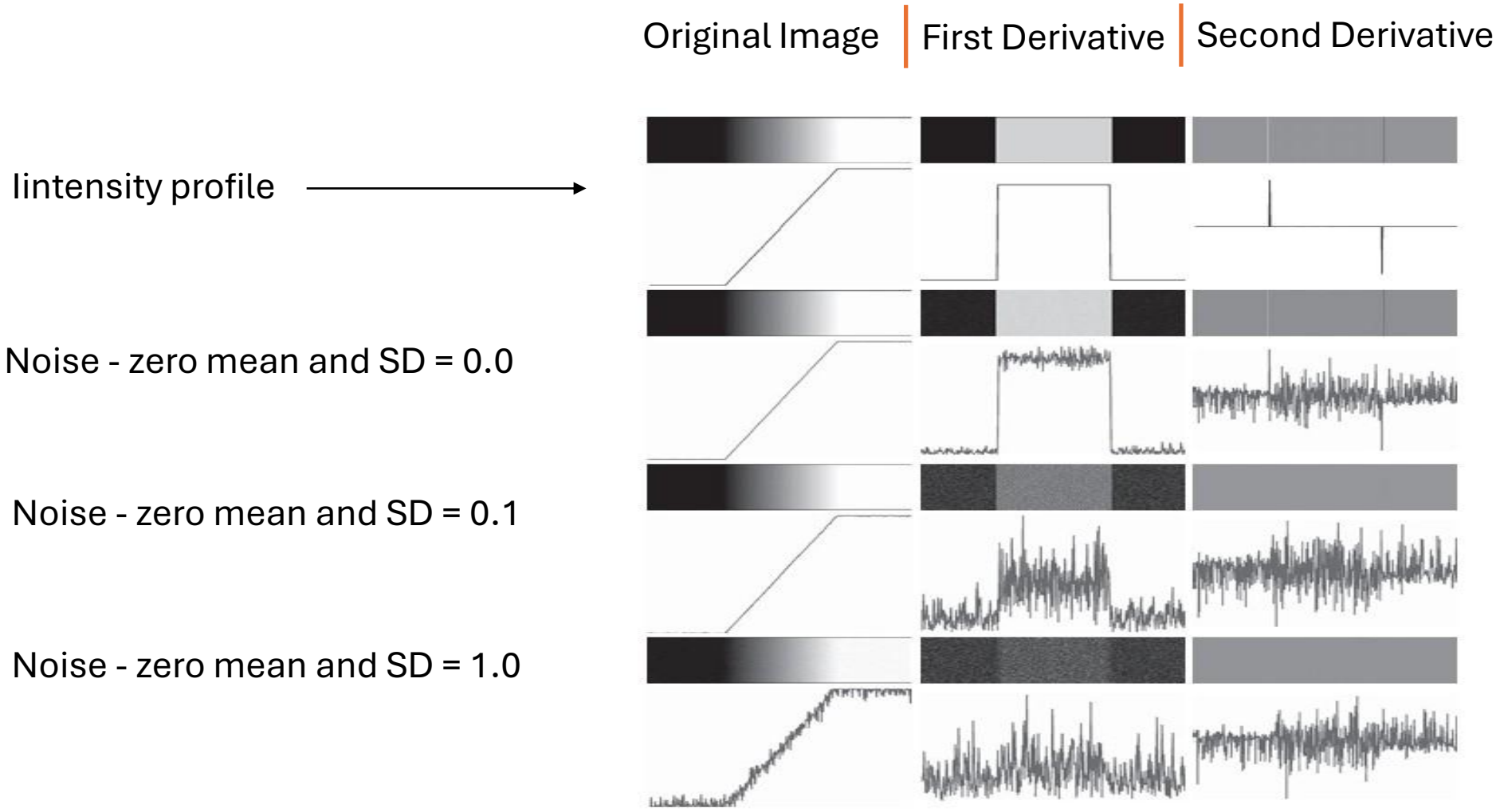
kernel

$$\nabla^2 \approx \frac{1}{6\varepsilon^2}$$

1	4	1
4	−20	4
1	4	1

Note: Here, this kernel is not taking care of all the directions.
Laplacian only provide location of the edge, no magnitude and direction information

Edge: How first and second derivative changes as noise increases.



Note: As noise increases then second derivative enhances the edge as well as noise also. Preprocessing is required.

Laplacian of Gaussian (LoG)

Limitation of Prewitt and Sobel Operators

1. They only able to detect the edges in horizontal and vertical direction.
2. For different type of images, kernel size will be different.

Silent Features required for any Edge Detectors

1.Second derivative could able to localize edges in the better way as compared to the first derivative.

2.Preprocessing is required before applying edge detector.

3. Edge detector should be isotropic i.e. it could detect edges in all the directions.

4.Larger Kernel should able to detect blurry edges and small kernel could able to detect the sharply focused fine details.

LoG : Gaussian function first smooth the image and Laplacian will find the second derivative of gaussian function which directly helps to locate the edge.

Laplacian of Gaussian (LoG)

LoG : Gaussian function first smooth the image and Laplacian will find the second derivative of gaussian function which directly helps to locate the edge.

Laplacian Function

$$G(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Laplacian of Gaussian

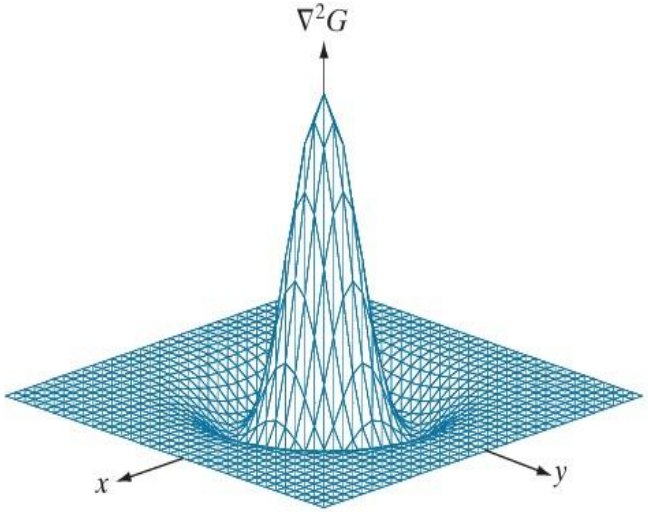
$$\nabla^2 G(x,y) = \frac{\partial^2 G(x,y)}{\partial x^2} + \frac{\partial^2 G(x,y)}{\partial y^2}$$

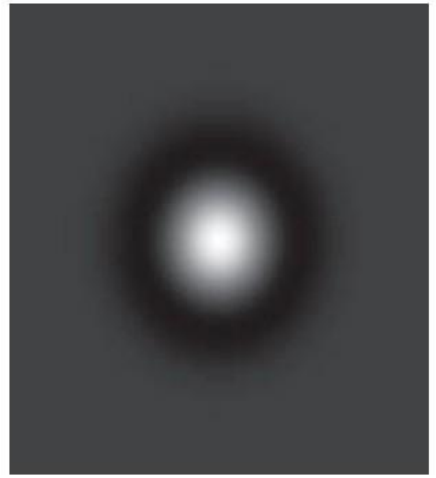
$$= \frac{\partial}{\partial x}\left(\frac{-x}{\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}\right) + \frac{\partial}{\partial y}\left(\frac{-y}{\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}\right)$$

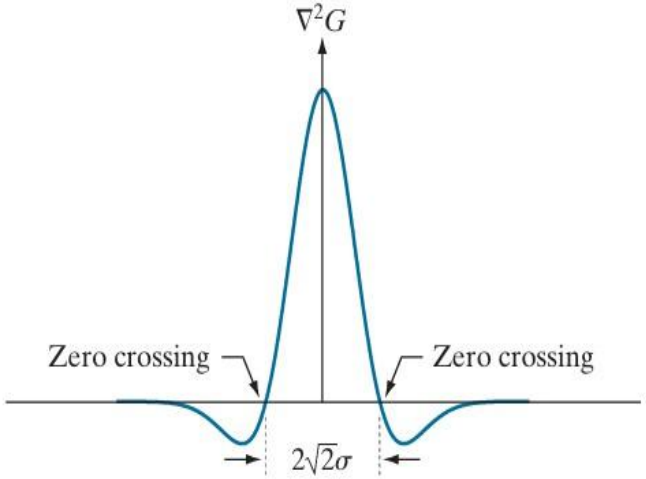
$$= \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) e^{-\frac{x^2+y^2}{2\sigma^2}} + \left(\frac{y^2}{\sigma^4} - \frac{1}{\sigma^2}\right) e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Collecting terms, we obtain

$$\nabla^2 G(x,y) = \left(\frac{x^2+y^2-2\sigma^2}{\sigma^4}\right) e^{-\frac{x^2+y^2}{2\sigma^2}}$$







0	0	−1	0	0
0	−1	−2	−1	0
−1	−2	16	−2	−1
0	−1	−2	−1	0
0	0	−1	0	0

Laplacian of Gaussian (LoG)

10	10	10	10	10
10	10	10	10	10
100	100	100	100	100
100	100	100	100	100
10	10	10	10	10

Image

-1	-1	-1
-1	8	-1
-1	-1	-1

3 by 3 Kernel

Output Image

Laplacian of Gaussian (LoG)

10	10	10	10	10
10	10	10	10	10
100	100	100	100	100
100	100	100	100	100
10	10	10	10	10

Image

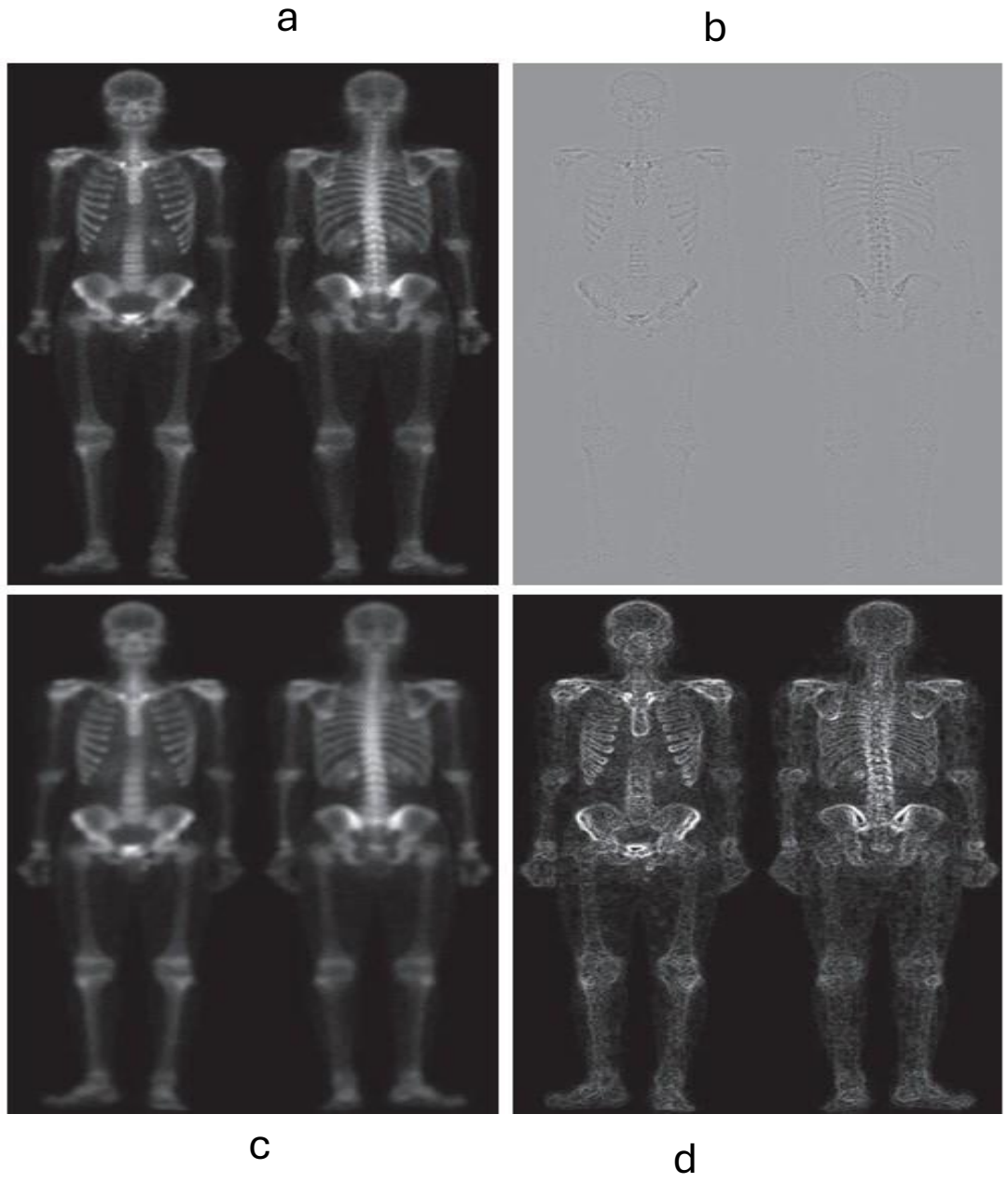
-1	-1	-1
-1	8	-1
-1	-1	-1

3 by 3 Kernel

	-220	-220	-220	
	270	270	270	
	270	270	270	

Transition from –ve to +ve, it directly shows the zero crossing i.e. Edge Location.

Combining Spatial Enhancement Methods



a – Image of whole body bone scan

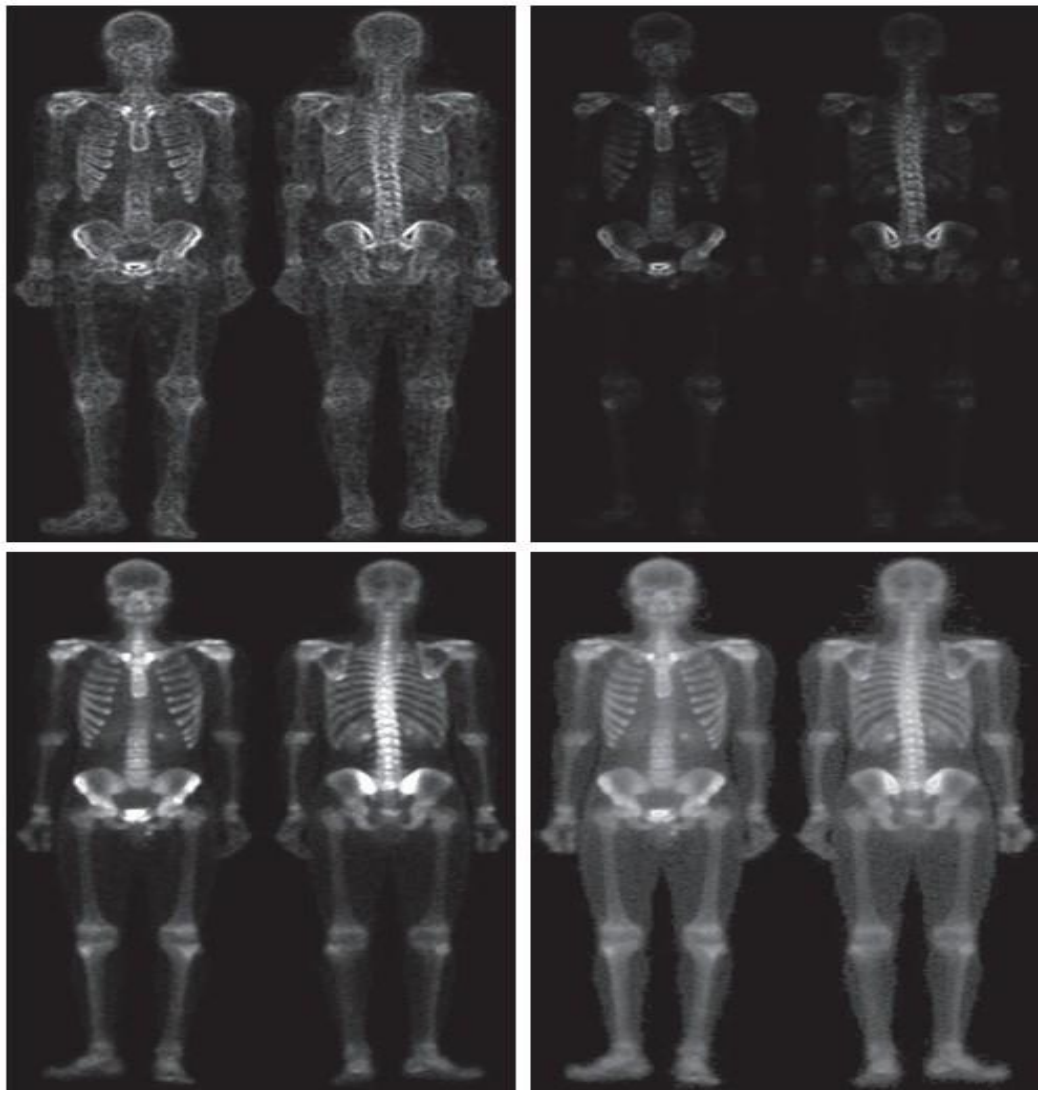
b – Laplacian of a

c – Sharpened image obtain by adding a and b

d- Sobel gradient of image (a)

Combining Spatial Enhancement Methods

e f



g h

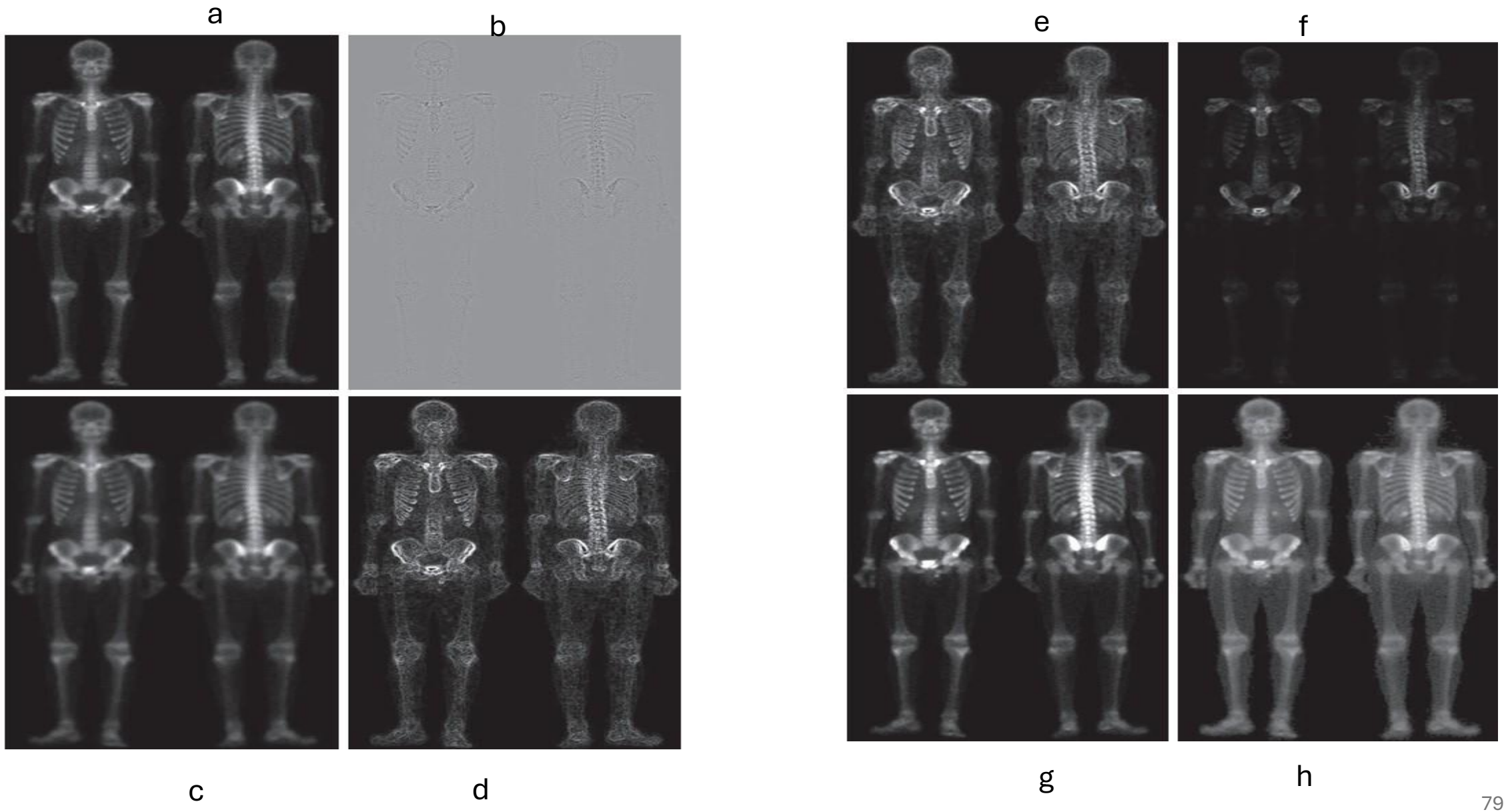
e– Sobel image smoothed with a 5 x 5 box filter

f – Mask image formed by the product of b and e

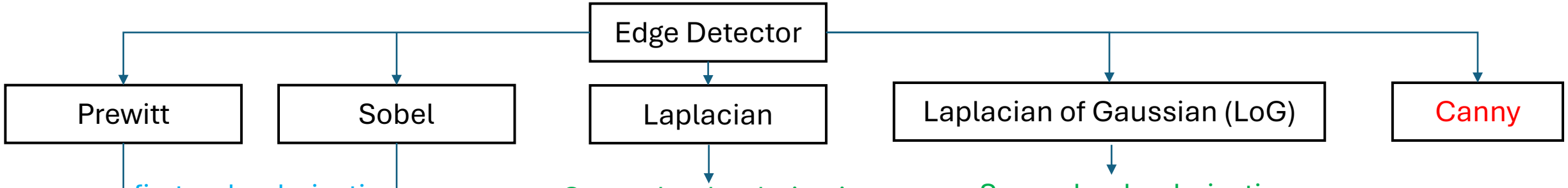
g – Sharpened image obtain by adding a and f

h- Final result obtained by applying a power law transformation to g.

Combining Spatial Enhancement Methods



Edge Detection (Recap of all edge detectors)



Requirements of a good edge detector

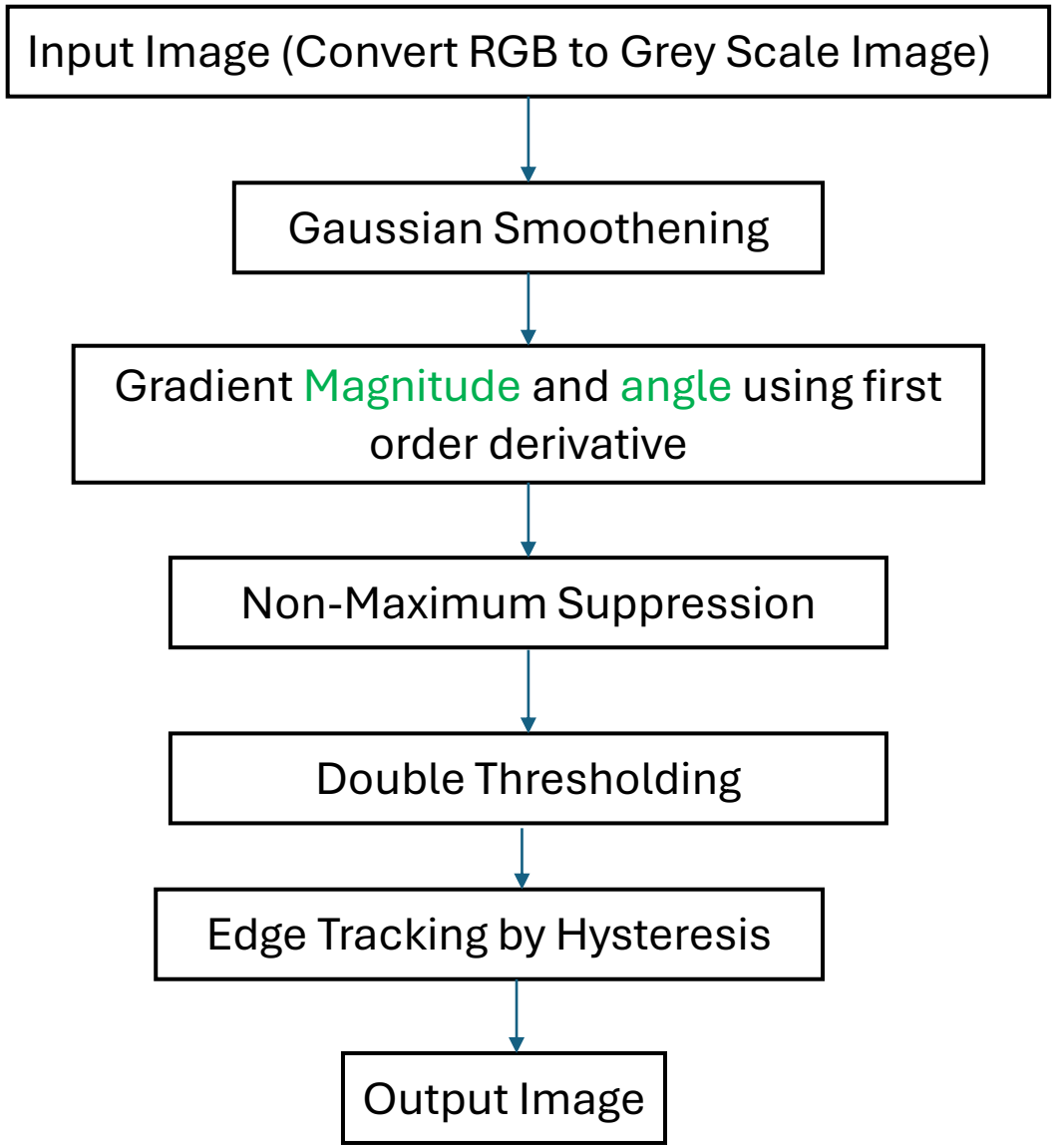
1. **Good Detection:** Find all the real edges and ignoring noise or artifacts (should not detect fake edges.)

2. **Good Localization:** Detect edges as close as possible to true edges.

3. **Single Response:** Return one point only for each true edge point. (No double edge response)

Canny Edge Detector

Flow Diagram



Steps in Canny Edge Detector.

1. Convert RGB to Grey scale image

2. Smooth the image to reduce the noise. (Gaussian Smoothening)

3. Gradient or Magnitude (**M**) Calculation. Angle calculation.

4. Apply Non-Maximum Suppression. (NMS)

5. Double Thresholding.

   If M > H:
   Edge
   elif L< M<H:
   Check is it connected with strong or weak edge
   else:
   No edge

6. Edge Tracking by Hysteresis

7. Output (Cleaned Image)

Canny Edge Detector

Step-1 Gaussian Smoothening

By reducing noise, you can prevent false edge detection.

$$w(s,t) = G(s,t) = Ke^{-\frac{s^2+t^2}{2\sigma^2}}$$

or

$$G(r) = Ke^{-\frac{r^2}{2\sigma^2}}$$

Consider r = (s^2 + t^2)^1/2

Gaussian kernel when Sigma =1

0.0780	0.1286	0.0780
0.1286	0.2121	0.1286
0.0780	0.1286	0.0780

Step-2 Detect edge i.e. gradient or magnitude calculation

First order derivative, Sobel used to detect the edge.

-1	-2	-1
0	0	0
1	2	1

-1	0	1
-2	0	2
-1	0	1

Z1	Z2	Z3
Z4	Z5	Z6
Z7	Z8	Z9

$$g_x = \frac{\partial f}{\partial x} = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$g_y = \frac{\partial f}{\partial y} = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

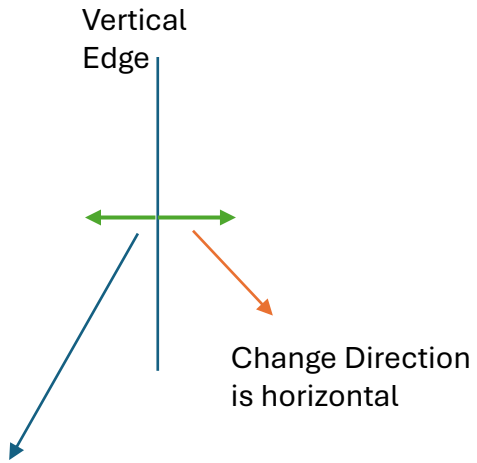
$$M(x,y) = \|\nabla f(x,y)\| = \sqrt{g_x^2(x,y) + g_y^2(x,y)}$$

Canny Edge Detector

Step-3 Non- Maximum Suppression

It used to find out the maximum pixel value on an edge in the gradient direction.

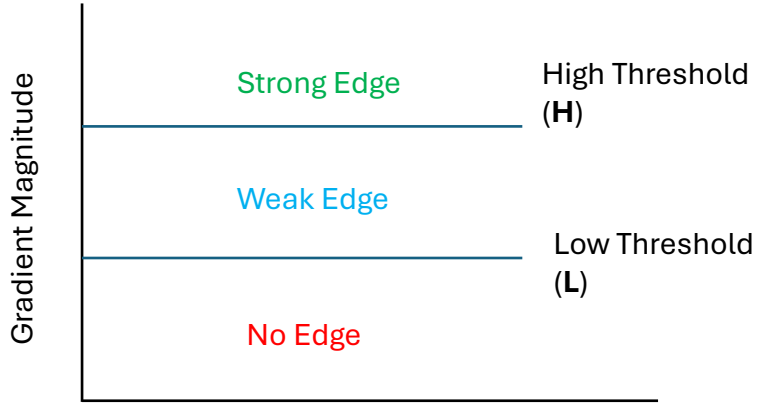
It retain the maximum value and suppress the non maximum values.



Suppress all the values those are not maximum in this particular direction to get the thin edges.

Step-4 Double Thresholding

Classify edges into 3 categories based on user defined threshold.



Step -5 Edge Tracking by Hysteresis

This step is used to check which weak edge is actual edge or false edge.

Pseudo code:

If weak edge connected to strong edge:

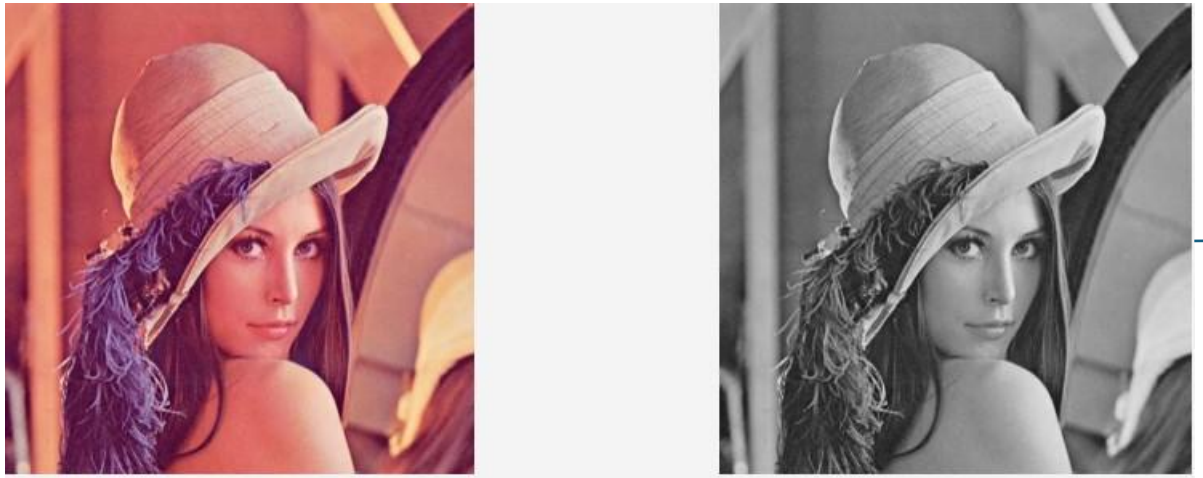
its an actual edge retain it as an edge

Else:

its an false edge, reject it.

Canny Edge Detector: Output of different steps

Color to Grey Scale image conversion



Gaussian Blur output

Gradient in x direction

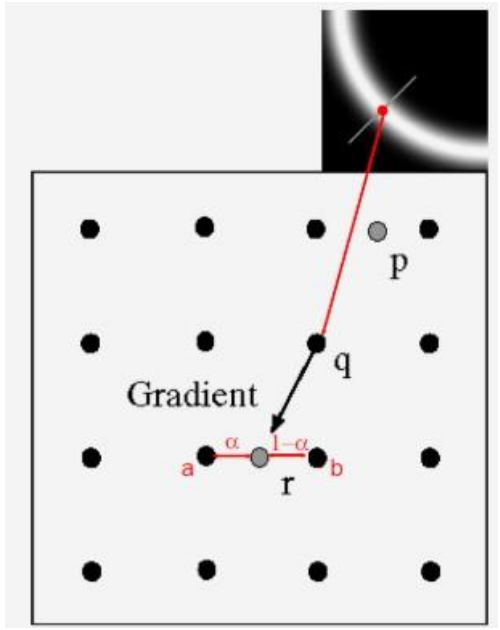
Gradient in y direction

Gradient Magnitude

Canny Edge Detector: Output of different steps

Non Maximum Suppression Output

Double thresholding Output



Hysteresis Edge Tracking Output

p,r are the pixels in the gradient direction.
If q value is grater than p and r than suppress p and r.

Canny Edge detector: Working example

Input Image

1	1	10	10	10
1	1	10	10	10
1	1	10	10	10

Padded (duplicate) Input Image

1	1	1	10	10	10	10
1	1	1	10	10	10	10
1	1	1	10	10	10	10
1	1	1	10	10	10	10
1	1	1	10	10	10	10

Here all the green pixels are padded pixels

Step-1 Gaussian Smoothening

Gaussian Kernel with sigma = 1

0.0780	0.1286	0.0780
0.1286	0.2121	0.1286
0.0780	0.1286	0.0780

Gaussian Smoothening output

Canny Edge detector: Working example

Input Image

1	1	10	10	10
1	1	10	10	10
1	1	10	10	10

Gaussian Kernel with sigma = 1

0.0780	0.1286	0.0780
0.1286	0.2121	0.1286
0.0780	0.1286	0.0780

Padded (duplicate) Input Image

1	1	1	10	10	10	10
1	1	1	10	10	10	10
1	1	1	10	10	10	10
1	1	1	10	10	10	10
1	1	1	10	10	10	10

Here all the green pixels are padded pixels

Gaussian Smoothening output

1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38

After smoothening, green pixels are padded pixels

Canny Edge detector: Working example

1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38
1	1	3.56	7.823	10.38	10.38	10.38

Sobel edge detector

-1	-2	-1
0	0	0
1	2	1

Gy

-1	0	1
-2	0	2
-1	0	1

Gx

Step-2 Detect edge i.e. gradient or magnitude calculation

Gy Output

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Gx Output

10.24	27.19	27.28	9.708	0
10.24	27.19	27.28	9.708	0
10.24	27.19	27.28	9.708	0

Magnitude

10.24	27.19	27.28	9.708	0
10.24	27.19	27.28	9.708	0
10.24	27.19	27.28	9.708	0

Canny Edge detector: Working example

Step-3 Non- Maximum Suppression

Magnitude

10.24	27.19	27.28	9.708	0
10.24	27.19	27.28	9.708	0
10.24	27.19	27.28	9.708	0

← Pixel →

NMS Output

0	0	27.28	0	0
0	0	27.28	0	0
0	0	27.28	0	0

Step-4 Double Thresholding

T1 > 27

Strong edge (output 1)

10<T2<27

weak edge

Else:

No edge

0	0	1	0	0
0	0	1	0	0
0	0	1	0	0

No weak edge

Step -5 Edge Tracking by Hysteresis

0	0	1	0	0
0	0	1	0	0
0	0	1	0	0

Comparison between LOG, Thresholded image and Canny

Original Image

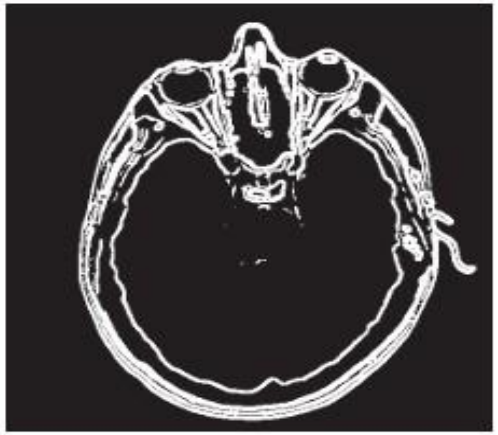
Thresholded Gradient Image

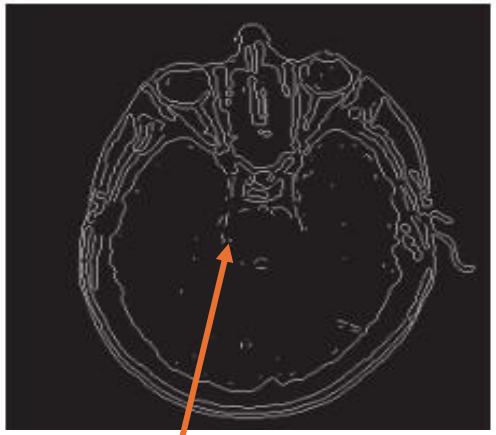
Laplacian of Gaussian

Canny Edge Detector

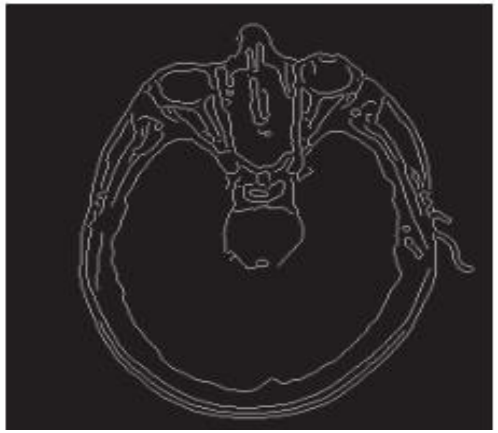
Original Image

Thresholded Gradient Image





Laplacian of Gaussian



Canny Edge Detector

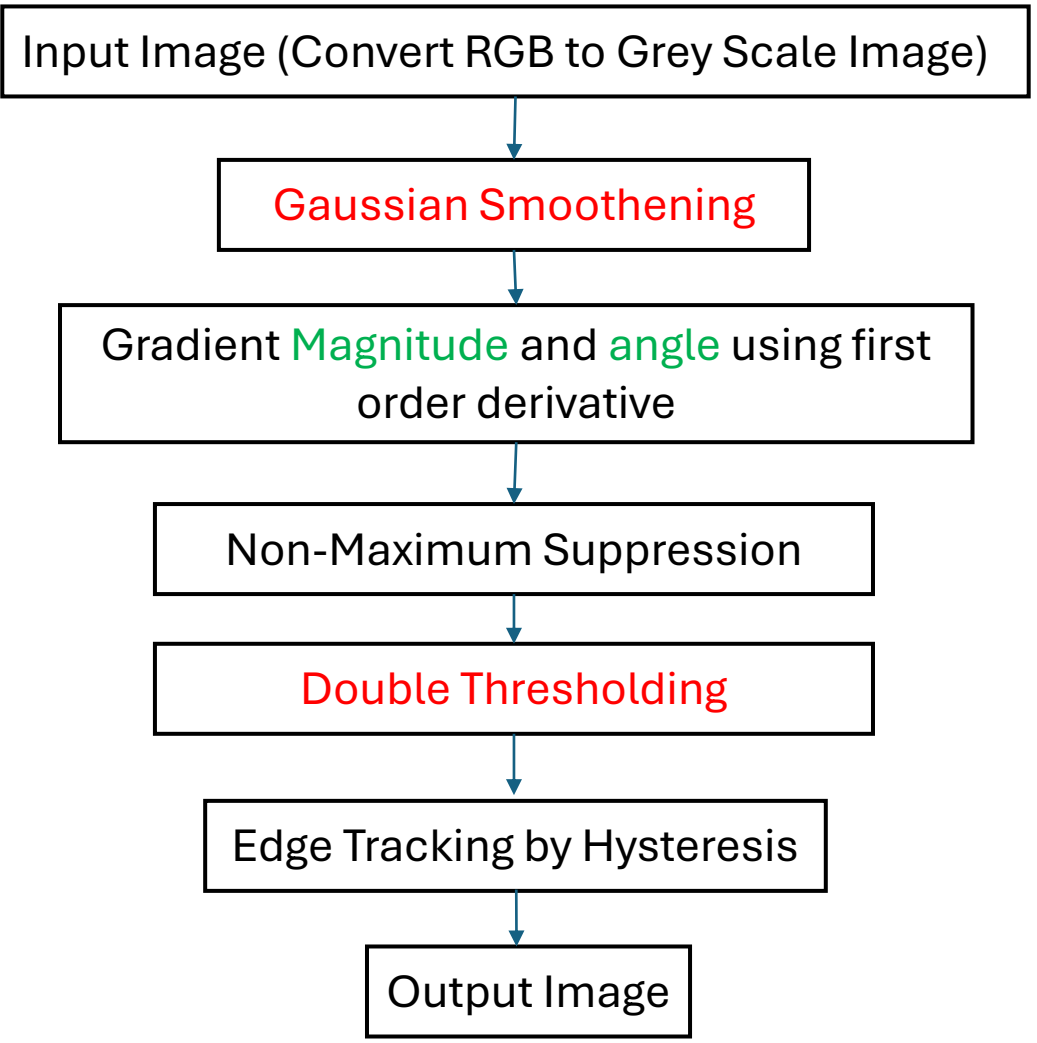
Either false edge or weak edge

Obs: Output of Canny is better than all other edge detector.

Comparison of all Edge Detector

Feature	Prewitt	Sobel	Laplacian of Gaussian (LOG)	Canny
Gradient Operator	Simple difference	Weighted difference (more robust)	Second derivative (finds zero crossings)	Optimized gradient & sophisticated criteria
Noise Sensitivity	High	Moderate	High (but Gaussian blur helps)	Low (due to Gaussian blur)
Edge Localization	Less precise	More precise	Moderate	Very precise
Spurious Responses	More	Fewer	More (but zero-crossing helps)	Fewer
Edge Thickness	Thicker	Thinner	Thinner	Very thin
Computational Cost	Low	Slightly higher than Prewitt	Higher (due to convolution)	Highest
Implementation	Simple	Relatively simple	More complex (Gaussian + Laplacian)	More complex (multiple stages)
Typical Use Cases	Simple image analysis, basic edge detection	General-purpose edge detection	Images with low noise, blob detection	High-quality edge detection, computer vision

Problems with Canny Edge Detector



Problem 1
Gaussian Smoothening – Blur the overall image even though blurring is required or not.

Solution – Apply adaptive smoothening where smoothening is applied Based on the presence of noise.

Problem 2

Double thresholding – Threshold is chosen manually, if two different persons select the different threshold then they will end up with Different edge map.

Solution – Instead of selecting it manually, the selection of threshold Should be automatic like it may select based on the statistical measure Like median.

Other issues with canny edge detector – Inability to handle complex conditions like uneven lightening (Create false gradient.) False edges are detected in the low contrast region.

CSET340

Advanced Computer Vision and Video Analytics

16th Feb. to 20th Feb. 2026

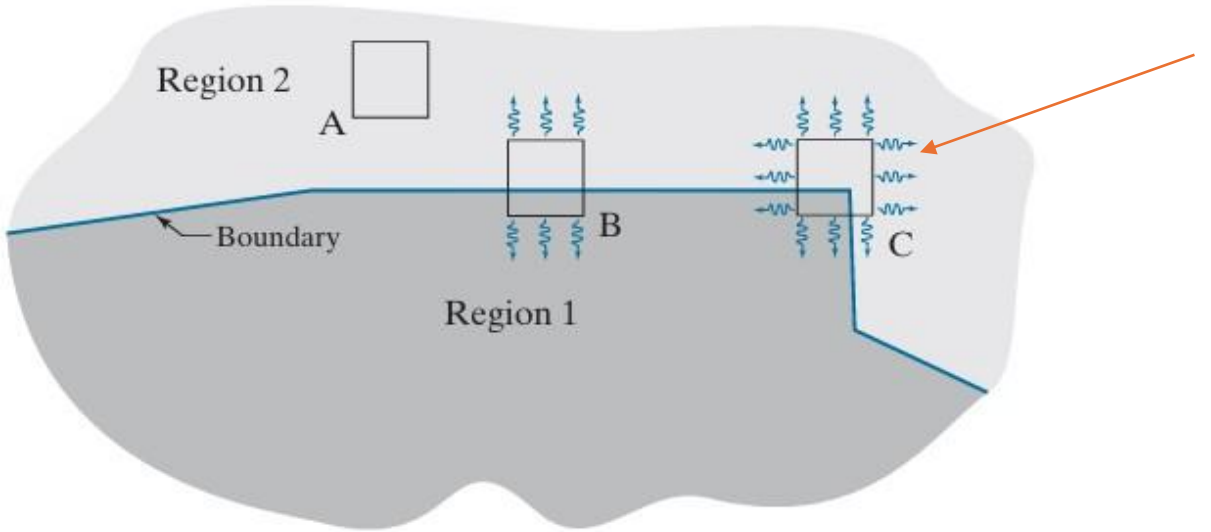
Overall Course Coordinator-
Dr. Gaurav Kumar Dashondhi
Gaurav.dashondhi@bennett.edu.in

Note : Any query related to course then first connect with overall course coordinator.

Harris Corner Detector or Harris Stephens Corner Detector

What is a corner ?

- It is rapid change in the direction of curve.
- Significant intensity changes in all the directions.
- A point where two edges are met.



Area of Significant intensity changes in all the directions.

Corners can be identified using spatial filtering: Take a window and apply it on image:

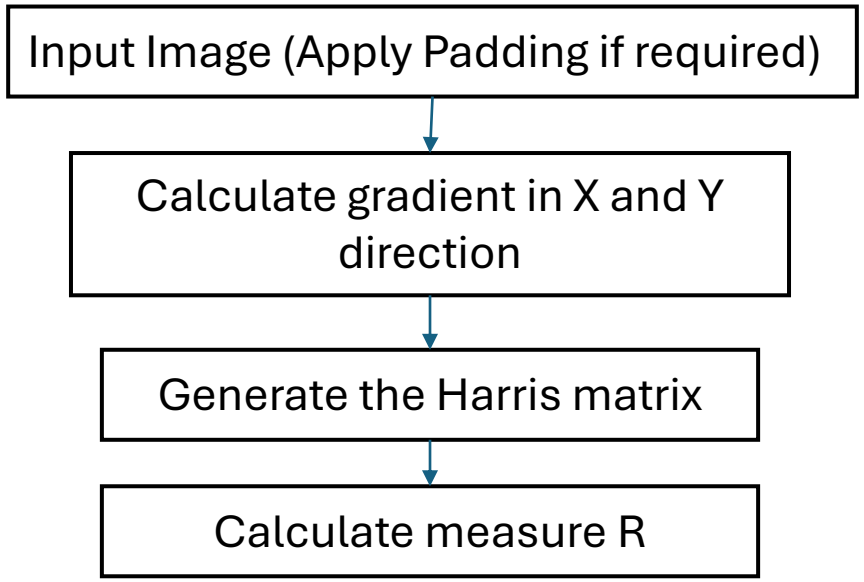
A : Area of zero intensity changes in all the direction i.e. a constant region. **No Edge**

B: Area of intensity changes in one direction but no change in the orthogonal direction. **Edge Present**

C: Area of intensity changes in all the direction: **Corner**

Harris Corner Detector

Steps in Harris corner detector



If R = Large positive value & both eigen value large:
Corner

Elif R = Large –ve value & one eigen value large & other small:
Edge

Elif |R| is small and both eigen values are small
flat region or constant intensity region

Let f denote an image, and let $f(s,t)$ denote a patch of the image defined by the values of (s,t). A patch of the same size, but shifted by (x,y), is given by $f(s+x,t+y)$. Then, the weighted sum of squared differences between the two patches is given by

$$C(x,y) = \sum_s \sum_t w(s,t)\left[f(s+x,t+y) - f(s,t)\right]^2 \qquad (11\text{-}56)$$

where $w(s,t)$ is a weighting function to be discussed shortly. The shifted patch can be approximated by the linear terms of a Taylor expansion

$$f(s+x,t+y) \approx f(s,t) + xf_x(s,t) + yf_y(s,t) \qquad (11\text{-}57)$$

where $f_x(s,t) = \partial f/\partial x$ and $f_y(s,t) = \partial f/\partial y$, both evaluated at (s,t). We can then write Eq. (11-56) as

$$C(x,y) = \sum_s \sum_t w(s,t)\left[xf_x(s,t) + yf_y(s,t)\right]^2 \qquad (11\text{-}58)$$

This equation can written in matrix form as

$$C(x,y) = \begin{bmatrix} x & y \end{bmatrix} \mathbf{M} \begin{bmatrix} x \\ y \end{bmatrix} \qquad (11\text{-}59)$$

where

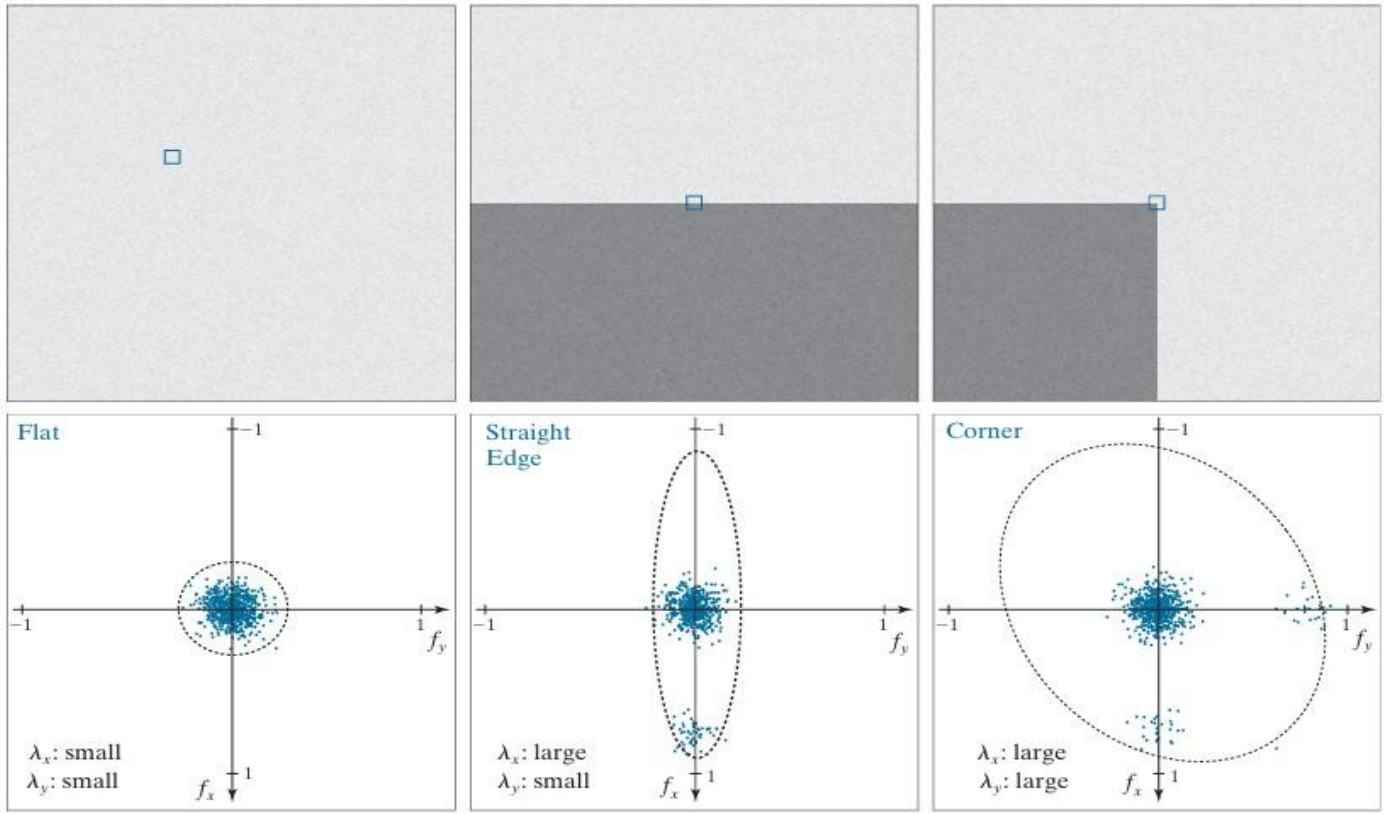
$$\mathbf{M} = \sum_s \sum_t w(s,t)\,\mathbf{A} \qquad (11\text{-}60)$$

and

$$\mathbf{A} = \begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix} \qquad (11\text{-}61)$$

Here M is Harris Matrix

Harris Corner Detector



Eigen values are almost identical

One eigen value large Another one is small.

Both the eigen values are large.

$$R = \lambda_x \lambda_y - k(\lambda_x + \lambda_y)^2$$
$$= \det(\mathbf{M}) - k\,\mathrm{trace}^2(\mathbf{M})$$

K is sensitivity factor; small value of K is better to find the corners.

Working example

Input Image

0	0	1	4	9
1	0	5	7	11
1	4	9	12	16
3	8	11	14	16
8	10	15	16	20

Change in Y direction

-1	0	1

-1
0
1

Change in X direction

Kernel in Y and X direction

Harris Corner Detector

Output after applying the kernel in x and y direction.

4	7	6
8	8	7
8	6	5

4	8	8
8	6	7
6	6	4

Ix^2 = 4^2+7^2+6^2+8^2+8^2+8^2+7^2+8^2+6^2+5^2 = 403

Iy^2 = 4^2+8^2+8^2+8^2+6^2+7^2+6^2+6^2+4^2 = 381

Ix * Iy = 4*4, 7*8..... = 385

Harris Matrix (H) or M =

403	385
385	381

R = Det (H) – k * (Trace)^2

R = 403 x 381 – (385)^2 - 0.04 x (784)^2 (approx.) = -19268.24

$$R = \lambda_x\lambda_y - k(\lambda_x + \lambda_y)^2$$
$$= \det(\mathbf{M}) - k\,\mathrm{trace}^2(\mathbf{M})$$

If R –ve then edge.
If R small then constant region.
If R is large then Corner

Note: If k is small then it will detect the corner in the better way.

Application and Problems of Harris corner detector

Applications of the Harris Corner Detector

•**Image Stitching:** Finding corresponding corners in multiple images to align and stitch them together.

•**Object Tracking:** Tracking objects in videos by identifying and following their corners.

•**3D Reconstruction:** Using corners as features to reconstruct 3D models from multiple images.

Problems

1. Sensitive to noise : Derivative operator used to detect corners.

2. window size is fixed : A fixed size window used to compute the corners. It may not capture the enough information to identify the corners.

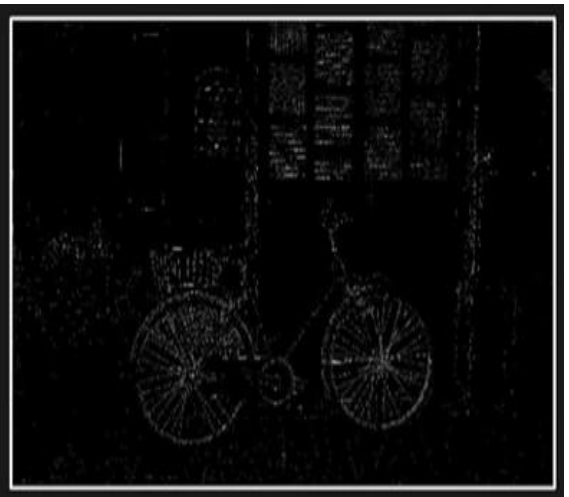
3. Lack of scale invariance: A corner that is detected at one scale may not be able to detect the same corner on higher scale.

Boundary Detection: Hough Transform

Lets say in the below image, you want to find or fit the lines to the circular part of the cycle.

Original Image

Edge map

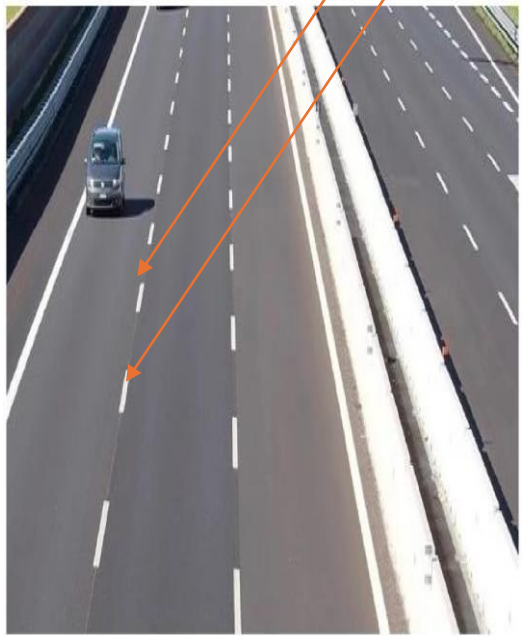


- Extraneous Data: Which points to fit to?
- Incomplete Data: Only part of the model is visible.
- Noise

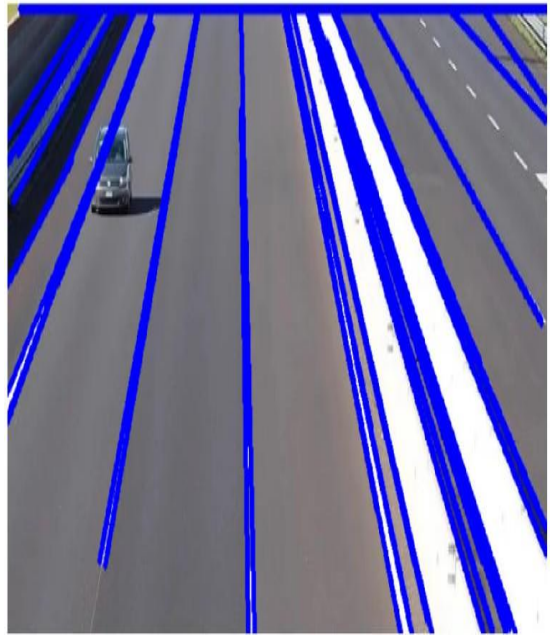
Solution: Hough Transform

Motivation: Edge Linking, Line Detection, Circle Detection and other generalized shape detection.

Perform edge linking



Hough Transform: Input Image

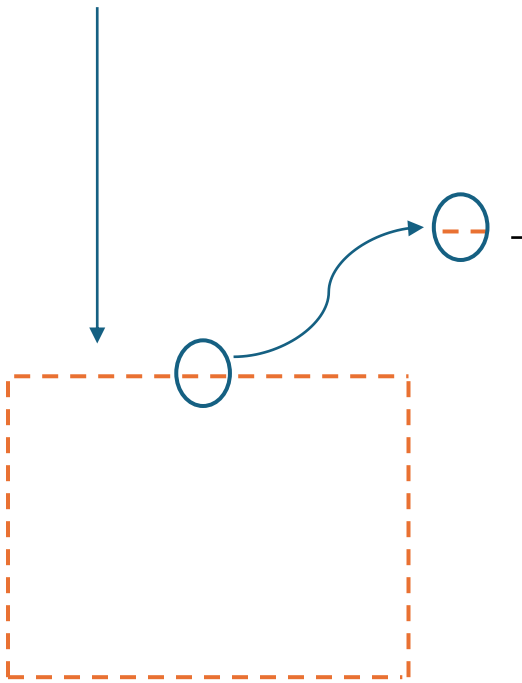


Hough Transform: Output Image

Hough Transforms: Line Detection

Motivation: Edge Linking, Line Detection, Circle Detection and other generalized shape detection.

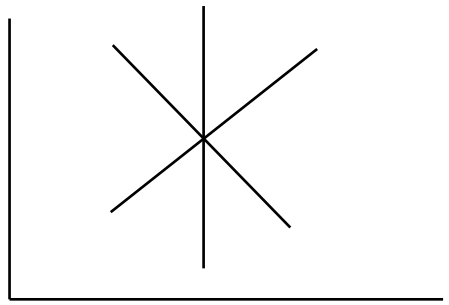
Edge detection output



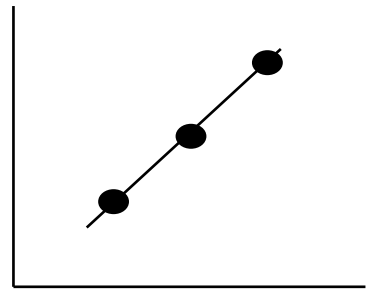
Here, output is dashed line.

Edges points are present then how to link them ?

Problem is



So many lines can pass through a single point.



A line may contain so many points.

One solution is to check the collinearity between points.

Hough Transforms: Line Detection

How to check collinearity between points ?

some points are said to be collinear if their slope is same.

Lets, consider three different points A(1,2), B(3,6), C(5,10). Check the collinearity ?

Slope m = (y2-y1) / (x2-x1)

M_{AB} = (6-2)/ (3-1) = (4/2) = 2

M_{BC} = (10-6)/ (5-3) = (4/2) = 2

Here, the **slope is same**, we can conclude points are collinear.

Hough Transforms: Line Detection

In Hough transform, point in the image space will be a line in the parameter Space.
If two points are colinear in the image space, then they will intersect each other in the parameter space.

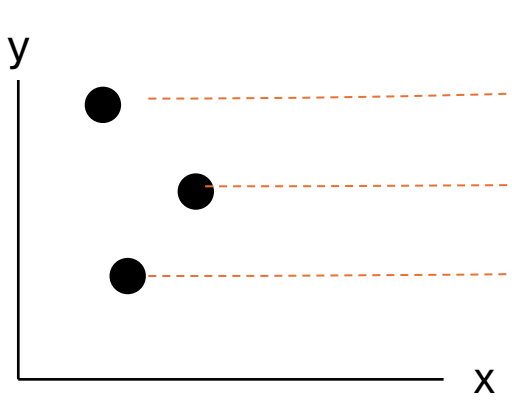
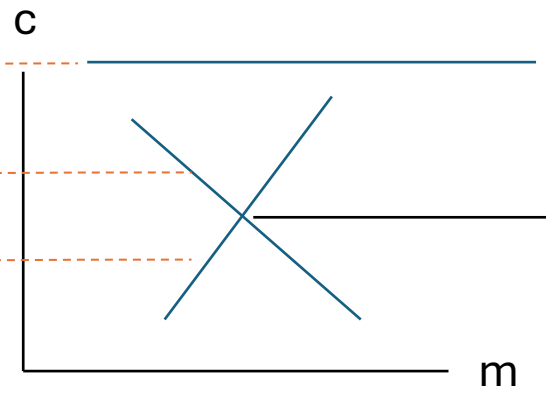


Image Space



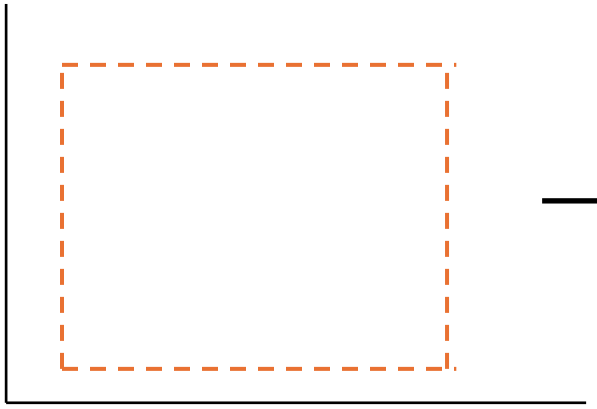
Parameter Space

c = -mx+y

Line from a non colinear point.

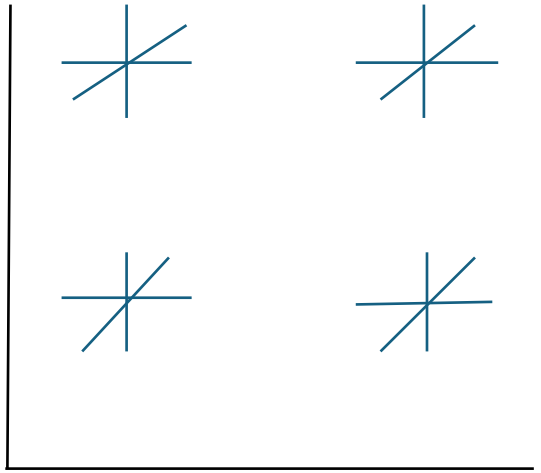
From here, you can Get the value of c and m which is intercept and slope respectively.

Y = mx+c, m = slope, c = Intercept

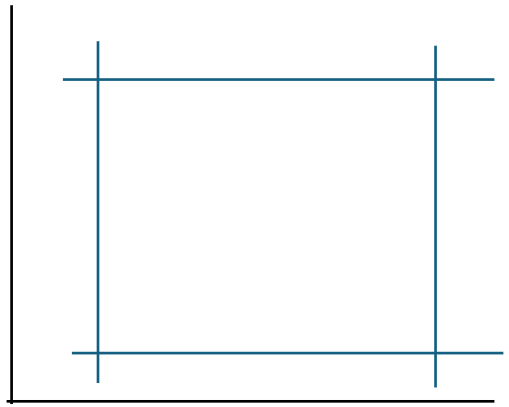


Image

For 4 lines in image Space you will get 4 different m,c in parameter space



Parameter Space



Output

Hough Transforms: Line Detection:

Working Example:

Point (1,2) and (2,3) are they colinear ?

Parameter space : c = -mx+y

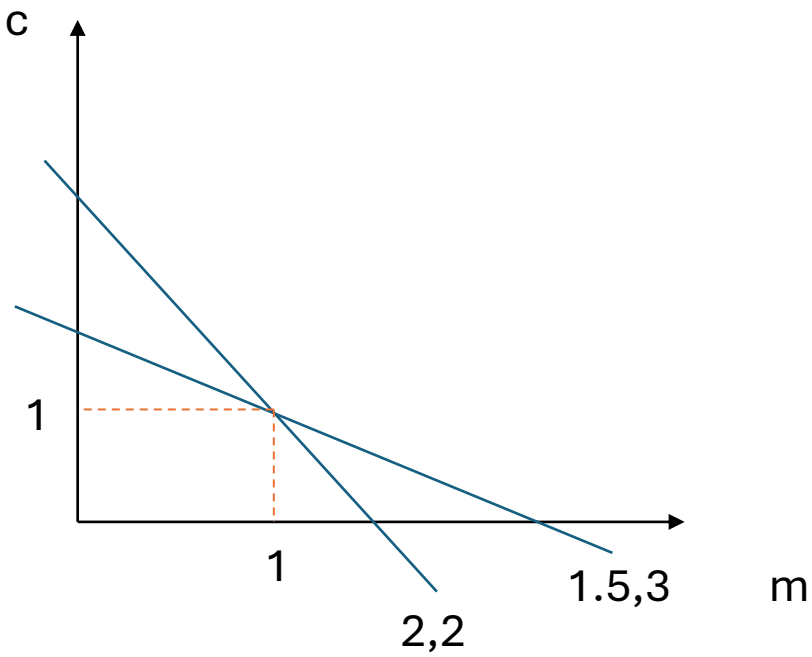
For (1,2) –

C = -m+2, if c=0 then m =2
if m=0 then c =2 output (2,2)

For (2,3)

C = -2m+3, if c=0 then m = 1.5
if m=0 then c = 3 output (1.5,3)

Parameter space.

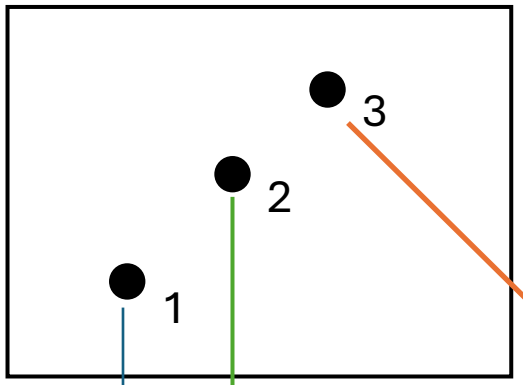


Y = mx + c
Y = x+1
Check it. Keep x=1 then y =2.
keep x=2 then y = 3.

These two points are colinear.

Hough Transforms: Line Detection

Original Image and 3 colinear points



Quantize the parameter space and fill the zeros.

c

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

m

Line Detection Algorithm

1. Quantize the parameter space.
2. Create an accumulator array A(m,c).
3. Set A(m,c) = 0, for all (m,c).
4. Fill the accumulator array and count the frequency or maximum value.

c

1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

m

0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	1	0	0	0
1	0	0	0	0

0	0	0	0	0
0	0	0	0	0
1	1	1	1	1
0	0	0	0	0
0	0	0	0	0

c

1	0	0	0	1
0	1	0	1	0
1	1	**3**	1	1
0	1	0	1	0
1	0	0	0	1

m

Hough Transforms: Line Detection: Polar form

Problems:

1. Slope $-\infty < m < \infty$
2. It directly makes the accumulator array very large (if we are using all possible slops)
3. It required more memory and computation cost.

Solution:

Polar form. Ro (rho) is distance from origin and theta is the angle for the same.

Theta is finite varies between 0 to 2pi Ro is finite = distance of straight line from origin

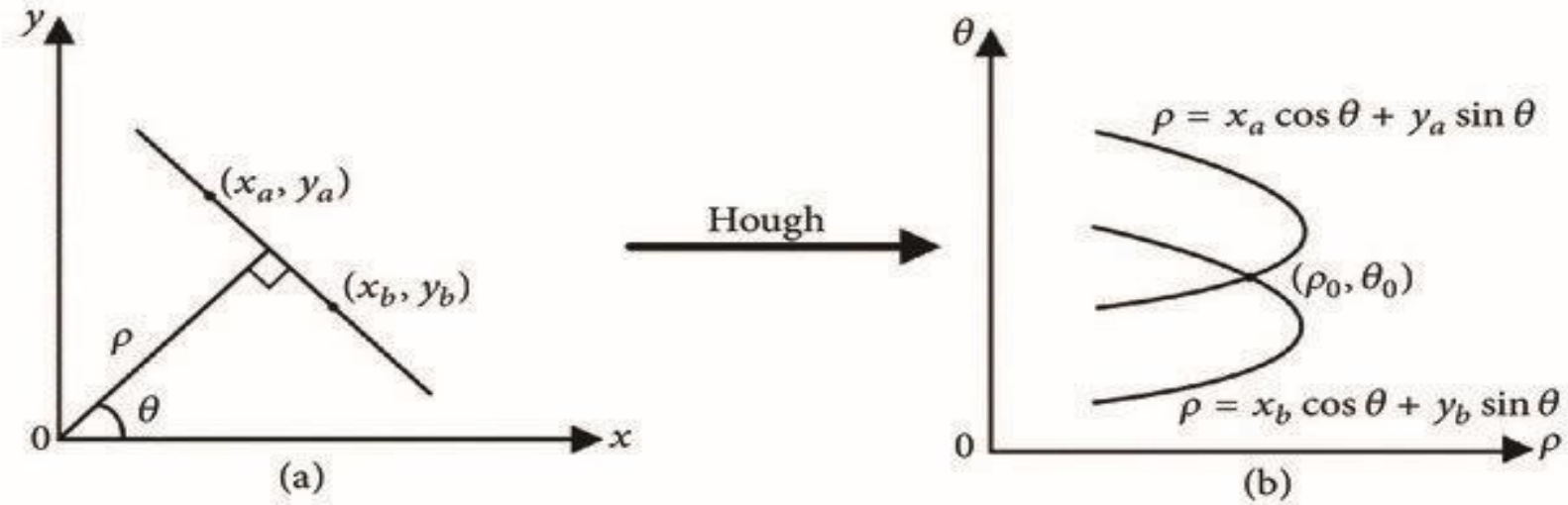
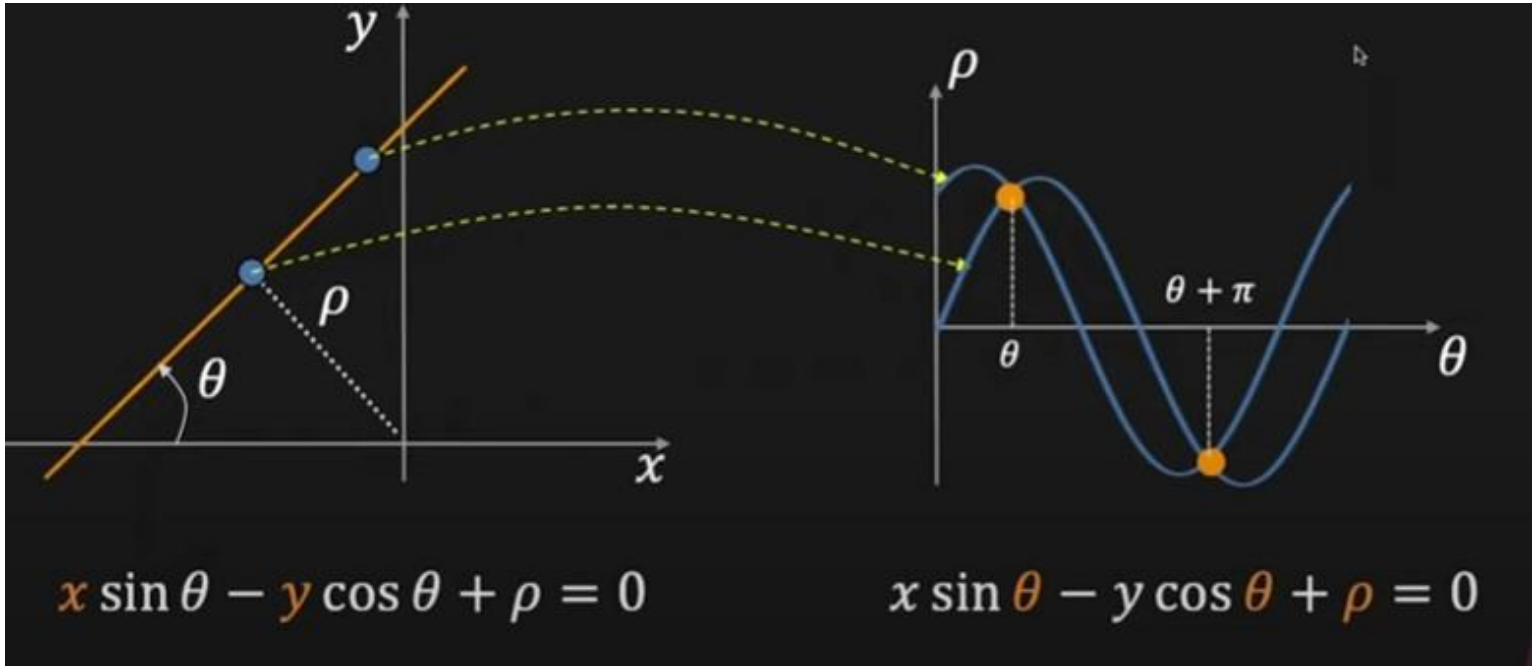


Image Space

Parameter Space

Image Space **Parameter Space**



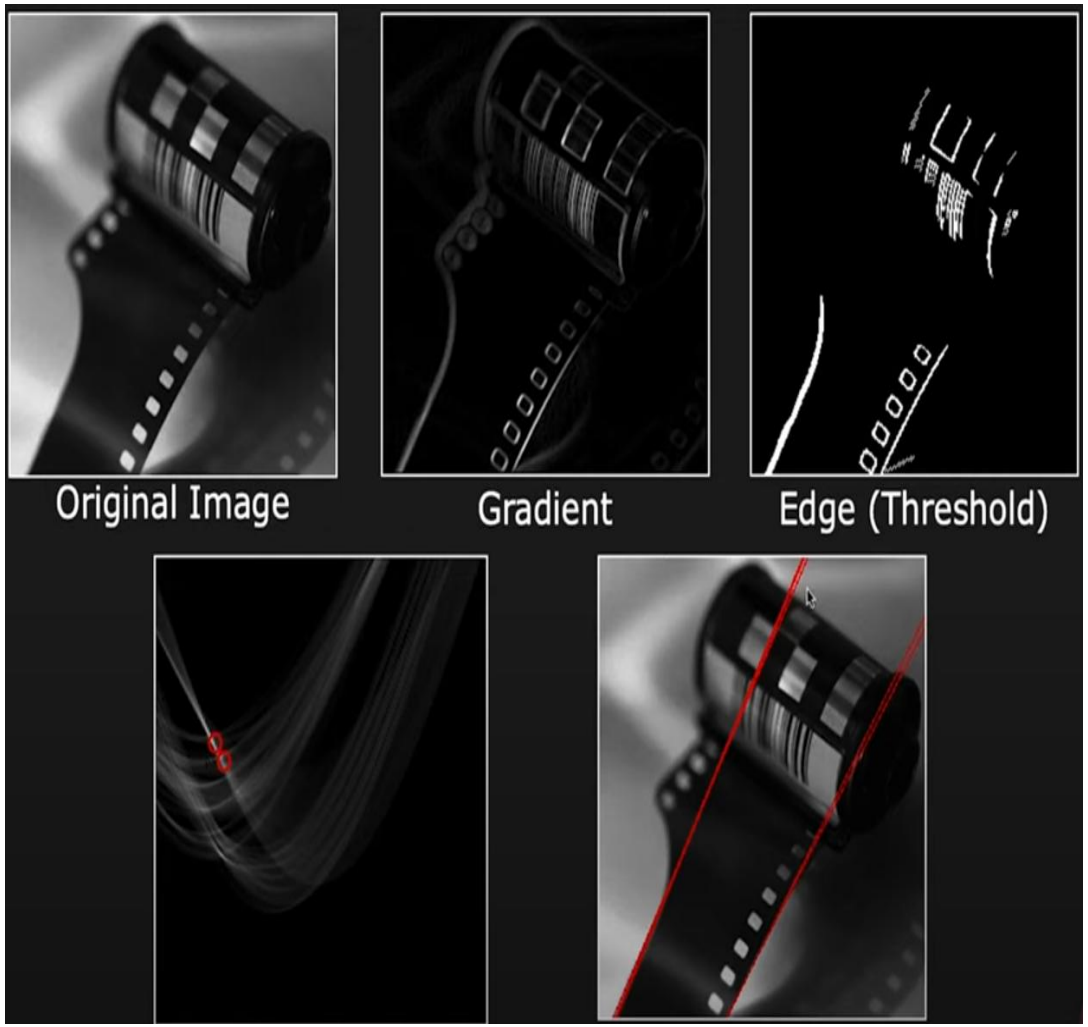
theta

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

rho

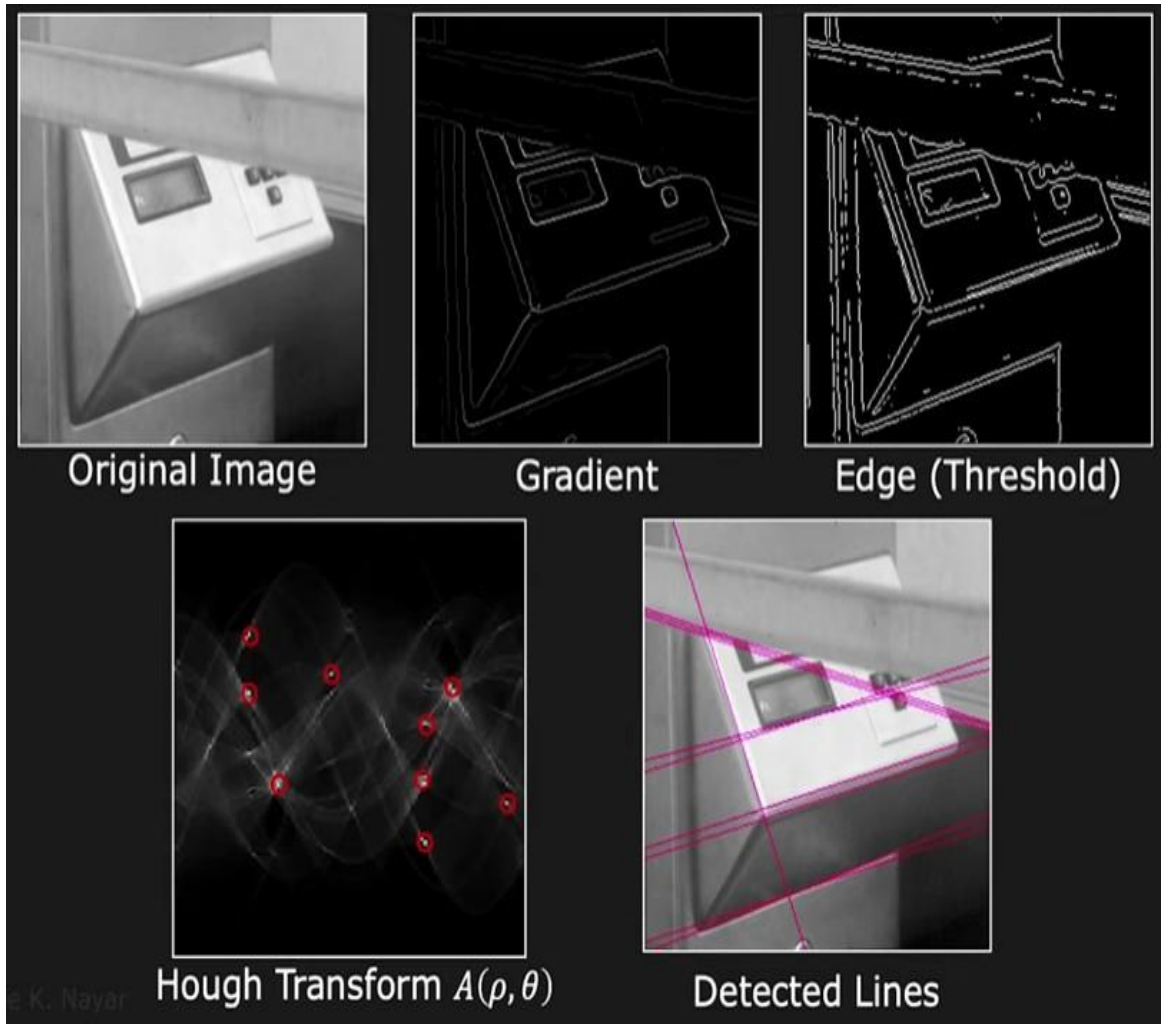
Here, accumulator array contains two parameters, one is rho and another one is theta.
Count the number of peaks.

Hough Transforms: line detection Examples



Hough Transforms (rho, theta space)

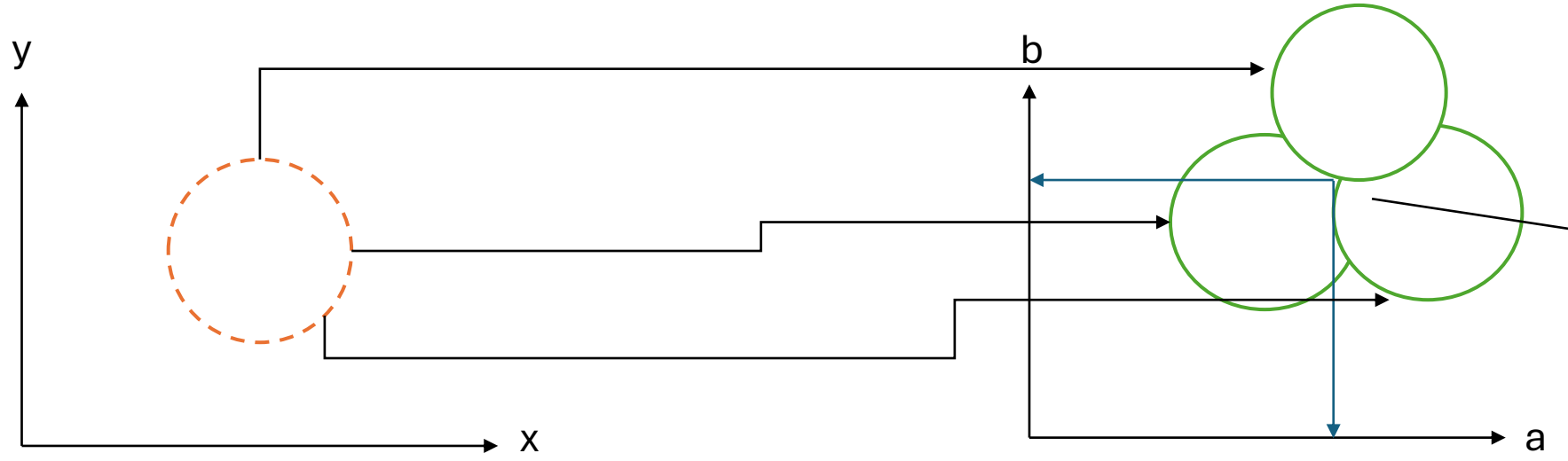
Detected Lines



Hough Transforms (rho, theta space)

Detected Lines

Hough Transforms: Circle Detection



Point where these circles are intersect each other will be Center of the circle that we want to detect.

Image Plane

Parameter Plane

For each circle point in the image plane their will be a circle in the parameter space.

$(x-a)^2 + (y-b)^2 = r^2$

a, b = centre of the circle.
r = radius of the circle.

If **r is known** then parameter space contains 2 parameters a and b. Here a,b is the centre of the circle.

b

0	1	1	1	0
1	0	0	0	1
1	0	3	0	1
1	0	0	0	1
1	1	1	1	0

a

Accumulator array for circle detection.

Accumulator array size:
Line (2D), Circle (3D if radius Not constant.)
(2D if radius is constant.)

Detect the different coins

Center of Coins with Radius r2.

Problems with Hough Transform.

1. As the number of parameters will increase then accumulator array size will increase which directly increase the computational cost.
2. Parameter Sensitivity.
3. Memory Usage.
4. Loss of Positional Accuracy:

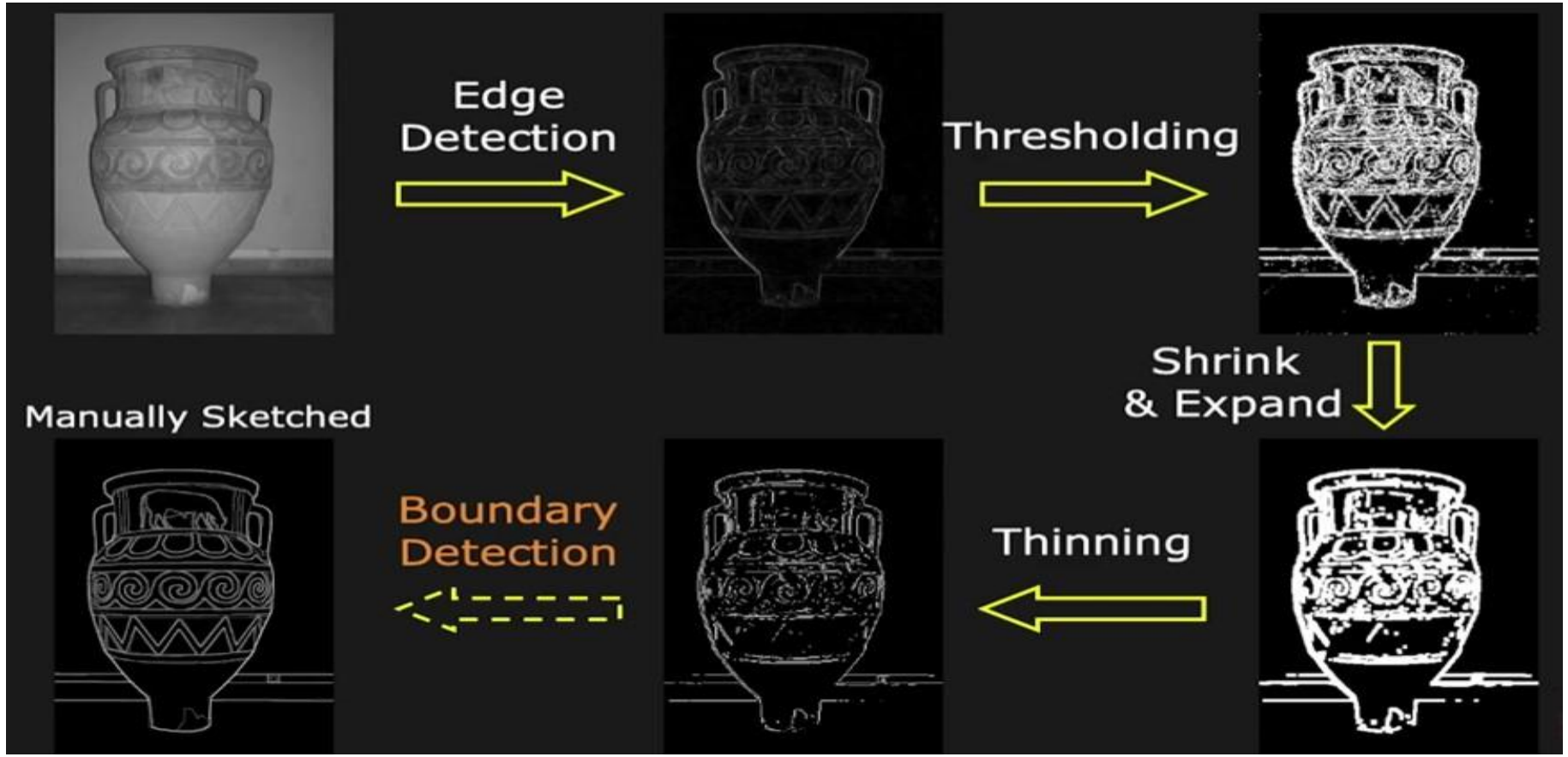
Boundary Detection

Overall objective of boundary detection is –

How to reach out - **form** edge map **to** the actual boundaries of the object.

Canny edge detector

Static or dynamic thresholding



Morphological operations.

Actual boundary ← Edge map

Boundary Detection: Active Contours or Snakes

Active Contour or Snakes are generally used to find the boundary of an object.

Given: Approximate boundary (contour) around the object

Task: Evolve (move) the contour to fit exact object boundary

Image

Draw an initial contour around it **or**
It may be hand sketched boundary.

What this initial contour will do –
It will evolve over time, so that finally it will latch around the boundary of the coin.

Active Contour:

Iteratively "deform" the initial contour so that:

- It is near pixels with high gradient (edges)
- It is smooth

Image

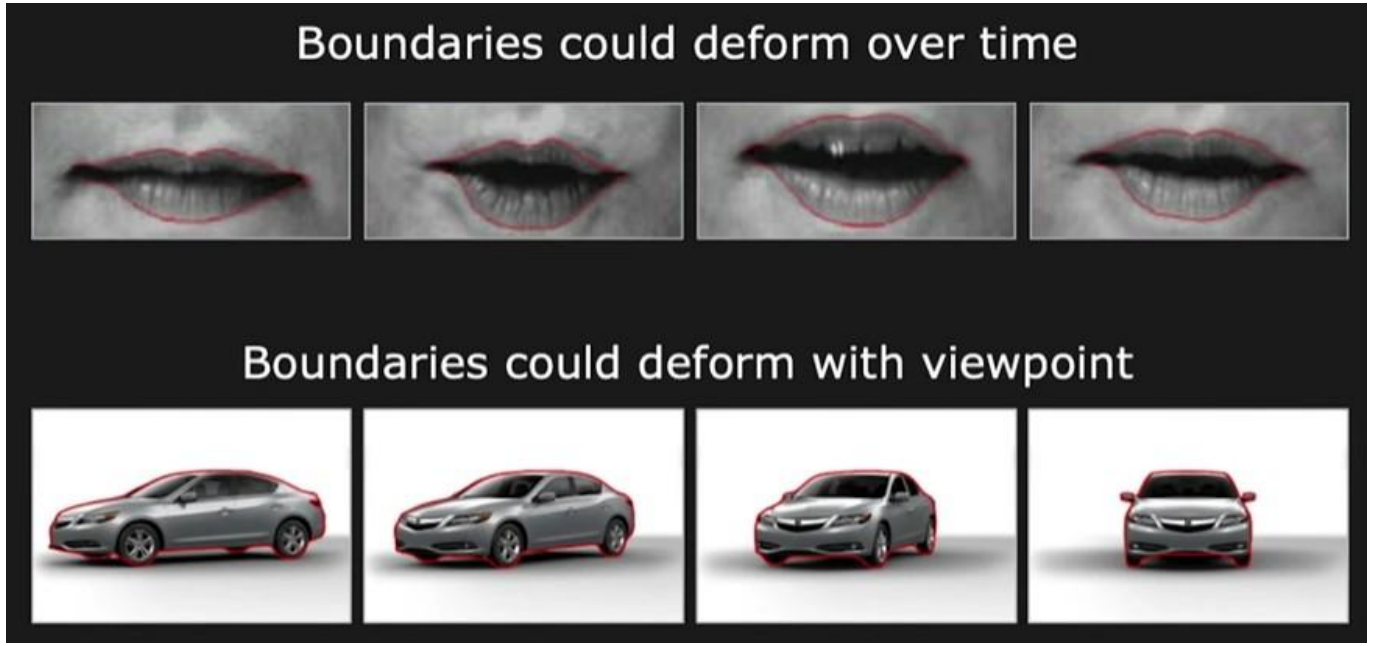
It will iteratively deform and ends up near the pixels on the objects that have the high gradient or edges which would be the boundary of the object.

At the same the contour you ends up does not have any knot or twist.

Boundary Detection: Power of the deformable contours

Deformable or flexible or stretchable

Boundary Tracking: Deformable contours helps to track the objects over a video sequence.



If you able to find the contour in the first frame, then that can be used for the initial contour for the next frames, and it can help to find out objects boundaries as it deforms over time.

If you have an object and view of object is changing.

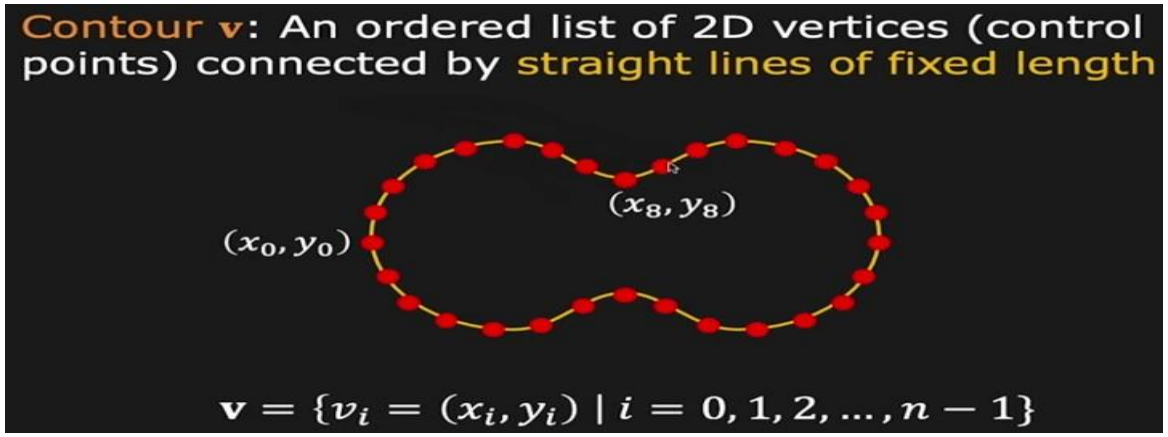
In that case also, you can start with initial contour and use it, to track over the subsequent frames.

What is contour or how to represent it –

Representing a contour:
Let's say we want to represent a contour with n points.

These points are nothing but the control points.

Distance between each control point is fixed.



Boundary Detection

Step -1

Take the initial contour, which is sitting around the object.

↓

The objective is to drawn it into the object itself.

↓

We need some kind of forces, that makes it evolve over time and come closer to object And latch on it.

Step -2

Apply some force and these forces depends on gradient of the image.

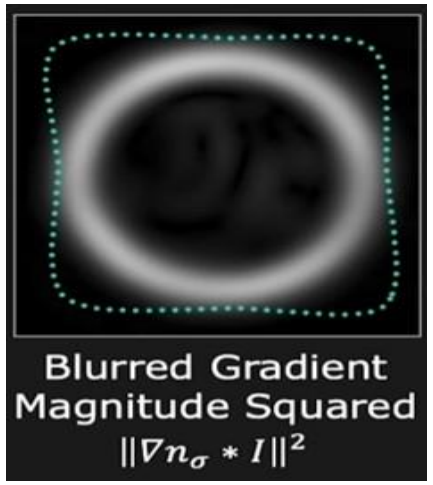
↓

Here we take the gradient Magnitude squared.

↓

But, the question is how do you attract this contour closer to the object when the gradients are sitting so far away.

Step -3

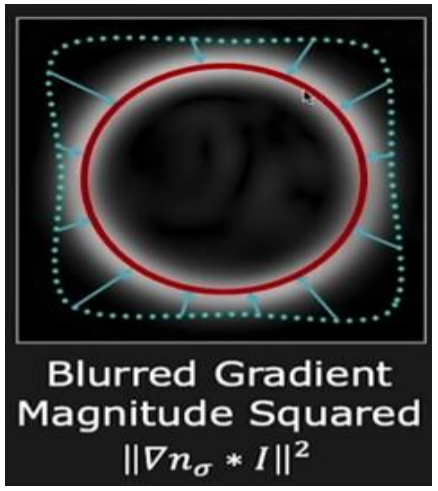


Apply Trick - blur the gradient.

↓

Due to this, even the Contour is sitting far away, is drawn towards it.

Step -4



Draw the contour towards the objects.

↓

It means we want to maximize The sum of gradient magnitude squ.

Flow diagram

Image with Initial Contours

↓

Gradient Magnitude Squared

↓

Blurred Gradient Magnitude Squared

↓

Maximize the sum of gradient Magnitude Squared

or

Minimize the sum of gradient Magnitude Squared

Maximize Sum of Gradient Magnitude Square

≡ Minimize –ve (Sum of Gradient Magnitude Square)

≡ Minimize $E_{image} = -\sum_{i=0}^{n-1} \|\nabla n_\sigma * I(v_i)\|^2$

Boundary Detection: Contour Deformation: Greedy algorithm

1. For each contour point v_i $(i = 0, \ldots, n-1)$, move v_i to a position within a window W where the energy function E_{image} for the contour is minimum.

2. If the sum of motions of all the contour points is less than a threshold, stop. Else go to Step 1.

Greedy solution might be suboptimal and slow.

Sensitivity to noise and initialization

Contour fitted to gradient magnitude

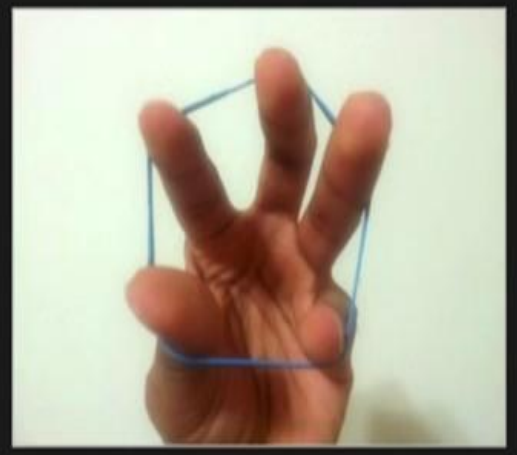
Contour fitted to gradient magnitude

Due to noise their is some Knots and twist are present.

Solution: Add constraints that make the contour contract and remain smooth

Boundary Detection

Making Contours Elastic and Smooth



Elastic and contracts
like a rubber band

Smooth
like a metal strip

Minimize Internal Bending Energy of the Contour:

$$E_{contour} = \alpha E_{elastic} + \beta E_{smooth}$$

(α, β): Control the influence of elasticity and smoothness

Combining the Forces

Image Energy, E_{image}: Measure of how well the contour latches on to edges

Internal Energy, $E_{contour}$: Measure of elasticity and smoothness

Total Energy of Active Contour:

$$E_{total} = E_{image} + E_{contour}$$

Minimize the Total Energy

Boundary Detection

Contour Deformation: Greedy Algorithm

1. Uniformly sample the contour to get n contour points.

2. For each contour point v_i $(i = 0, \ldots, n-1)$, move v_i to a position within a window W where the energy function E_{total} for the entire contour is minimum.

$$E_{total} = E_{image} + E_{contour}$$

3. If the sum of motions of all the contour points is less than a threshold, stop. Else go to Step 1.

Result: Effect of Contour Constraint

Without contour constraint

$$E_{total} = E_{image}$$

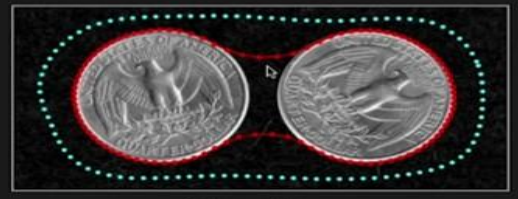
With contour constraint

$$E_{total} = E_{image} + E_{contour}$$

Boundary Detection

Result: Boundary Around Two Objects

Large α
(More like a rubber band)



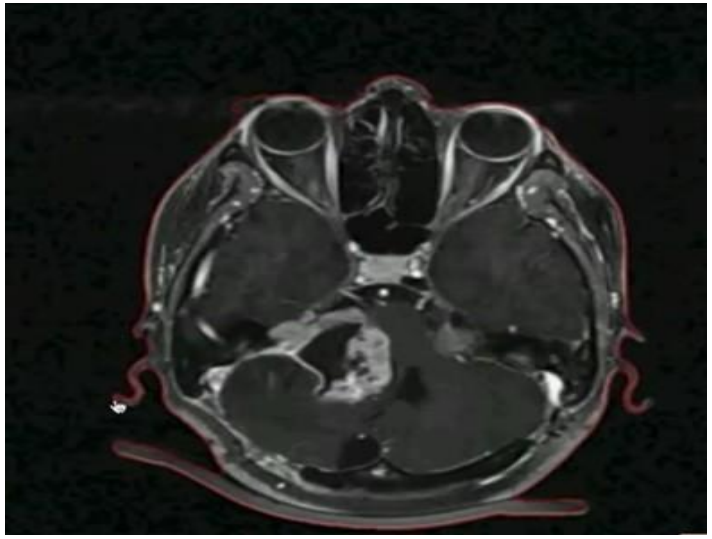
Small α
(Less like a rubber band)

Comments

- Additional energy constraints can be added
  - Penalize deviation from prior model of shape
- Requires good initialization
  - Edges cannot attract contours that are far away
- Elasticity makes contour contract
  - Replace contracting force with ballooning force to expand

Applications

Medical Image: Complex boundary detection



Interactive Image segmentation

CSET340

Advanced Computer Vision and Video Analytics

23rd Feb. to 27th Feb. 2026

Overall Course Coordinator-
Dr. Gaurav Kumar Dashondhi
Gaurav.dashondhi@bennett.edu.in

Note : Any query related to course then first connect with overall course coordinator.

Scale Invariant Feature Transform (SIFT): Motivation

What is feature ?
What is scale invariant feature ? And why we need such kind of features ?

Let's say, someone wants to recognize the given template in the Rich2D image (High resolution).

If you are using template matching (Because we know how to perform correlation) for the same purpose then -

1.User need to create a lot of templates with different orientation and scale because the size and orientation of the templates inside the image may be different.

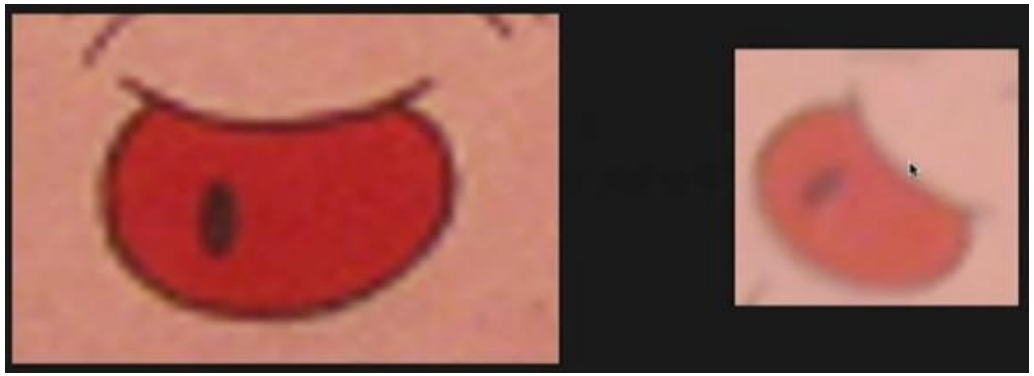
2. Apart form this, if template is partially not visible in the 2D image (covered by some other objects). The solution in this scenario is, again construct a lot of little templates and match all of them. At the end, this **overall process is time consuming and computationally not efficient**.

Instead of doing template matching, one can extract some important descriptive features known as **interest points** from template and match it inside the original image.

SIFT: Interest Point Detection

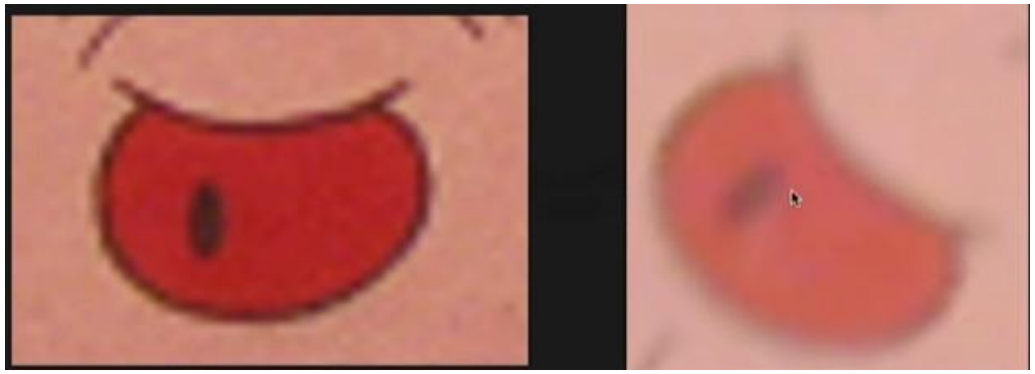
What is an interest point or feature

Consider two images with different –
1. Orientation. 2. Size 3. Lightening 4. Brightness

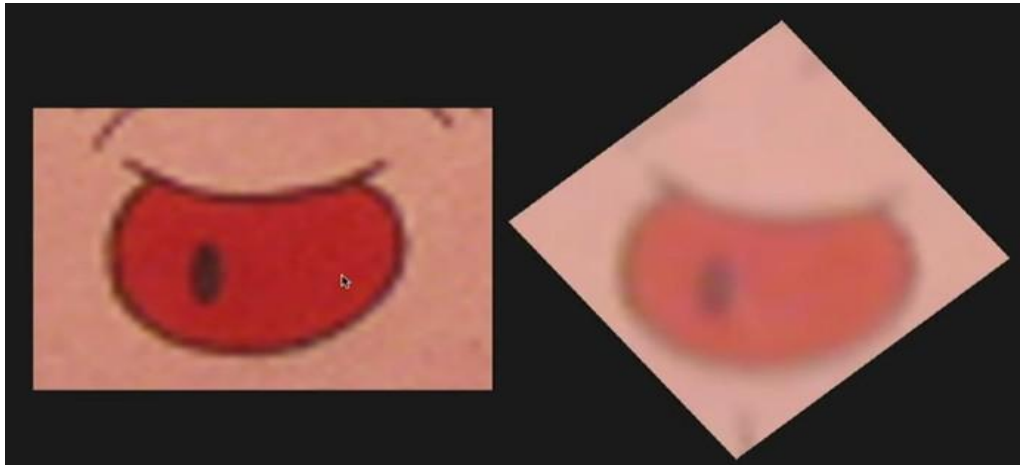


Lets take out a similar patch from both the images.
Here patch size is different.

We need to perform **rescaling** to compensate the difference in the size.



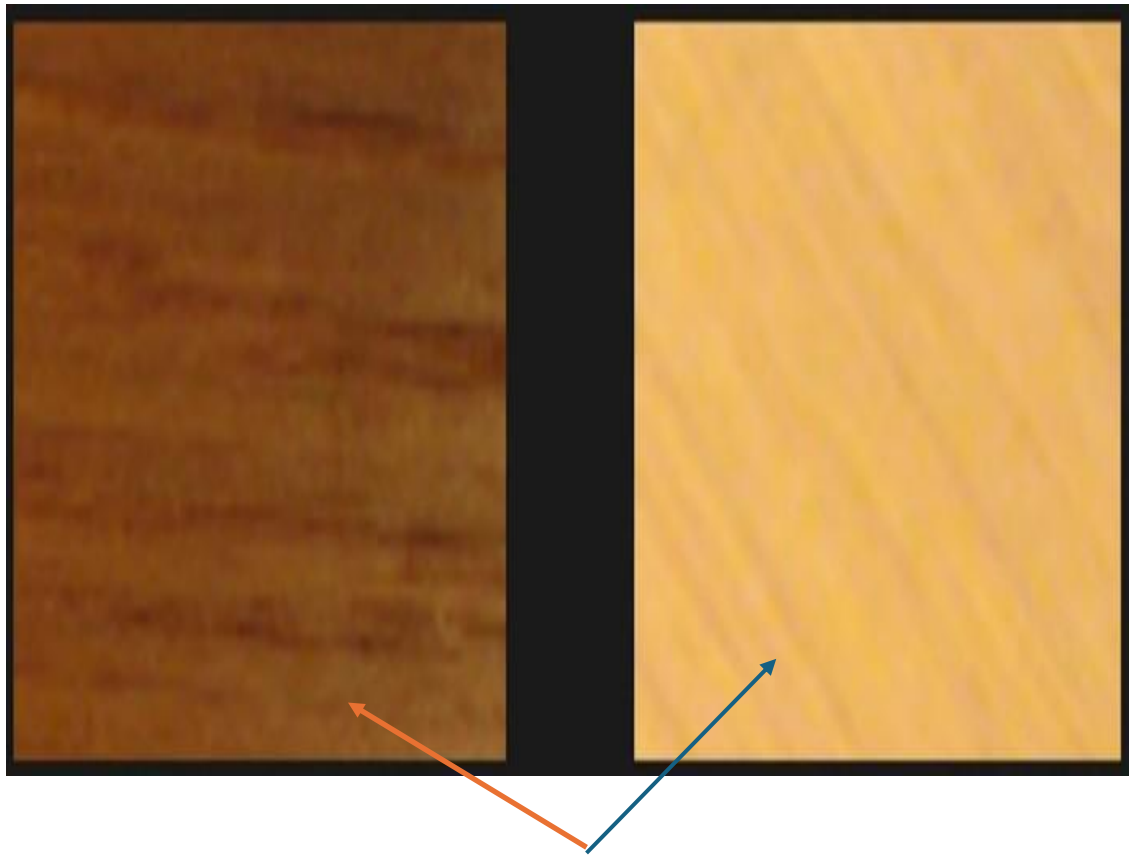
We need to perform **reorientation** to matches the Orientation.



Matching interest point becomes easier if we can remove the variations like size and orientation.

SIFT: Interest Point Detection

When you develop an interest point detector then you also don't want to respond a lot of things in the image.



Here, there is no unique information is present

What are the desirable points of a interesting point/ feature ?

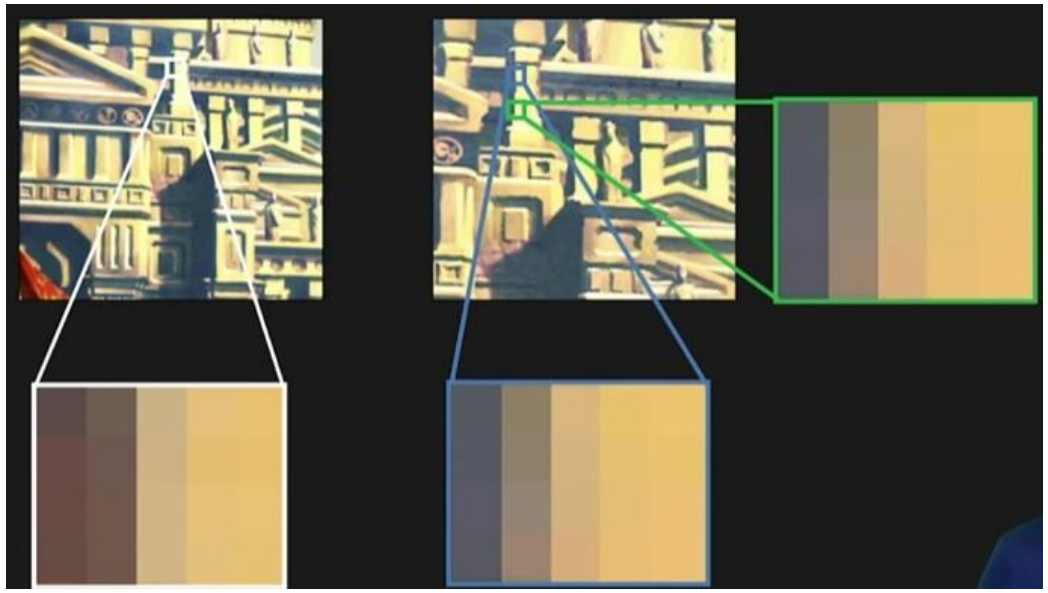
- Has rich image content (brightness variation, color variation, etc.) within the local window
- Has well-defined representation (signature) for matching/comparing with other points
- Has a well-defined position in the image
- Should be invariant to image rotation and scaling
- Should be insensitive to lighting changes

Again the questions is, with the given above list which one is the Interesting point or Interest point ?

Like - Line or edge or corners ?

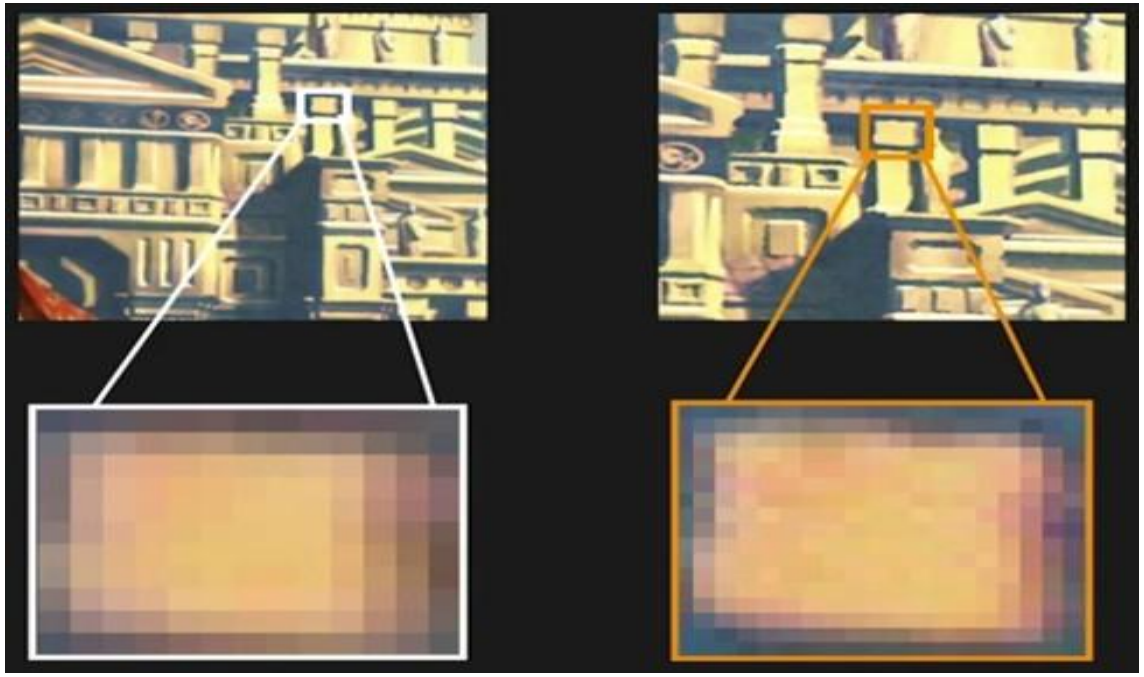
SIFT: Interest Point Detection

Is Line, Corner, Edges any one of them are good interest point ?



There is not lot of variation in the Line, edge and in the corner. Due to this, it cannot consider as good interest point.

Are blobs are more interesting ?
Blob means a patch with local appearance.



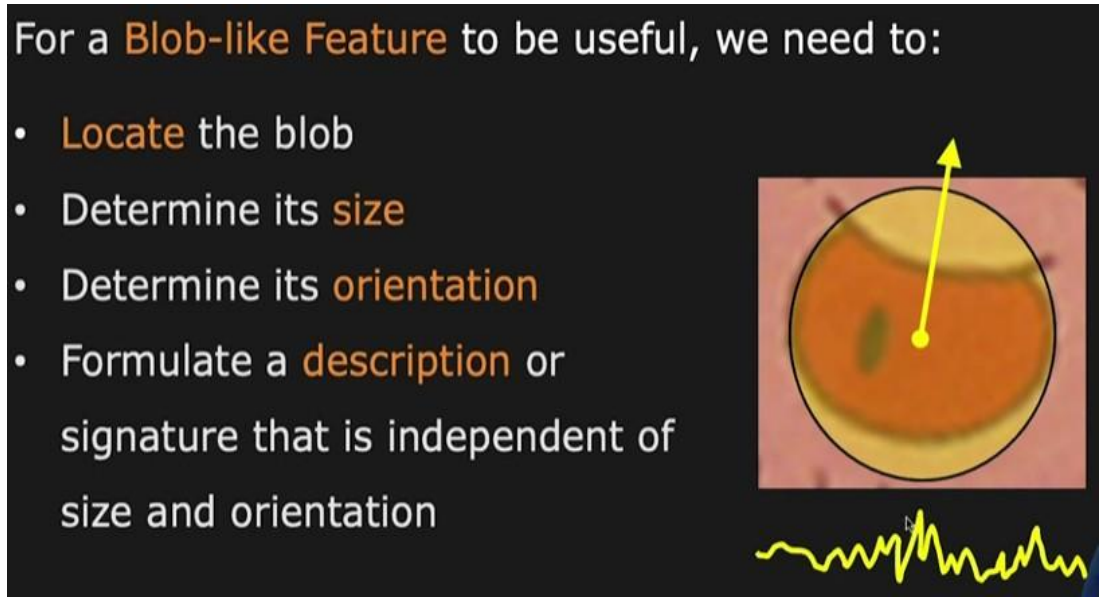
Yes, blobs has fixed position and definite size and also It contains a lot of variations or unique features.

Overall Steps in the SIFT Detector-

Blob Detection → Scale Invariance → Rotational Invariance → Generate or find signature → Match the features with distance measure.

SIFT: Interest Point Detection

Lets consider blob as an interest point.



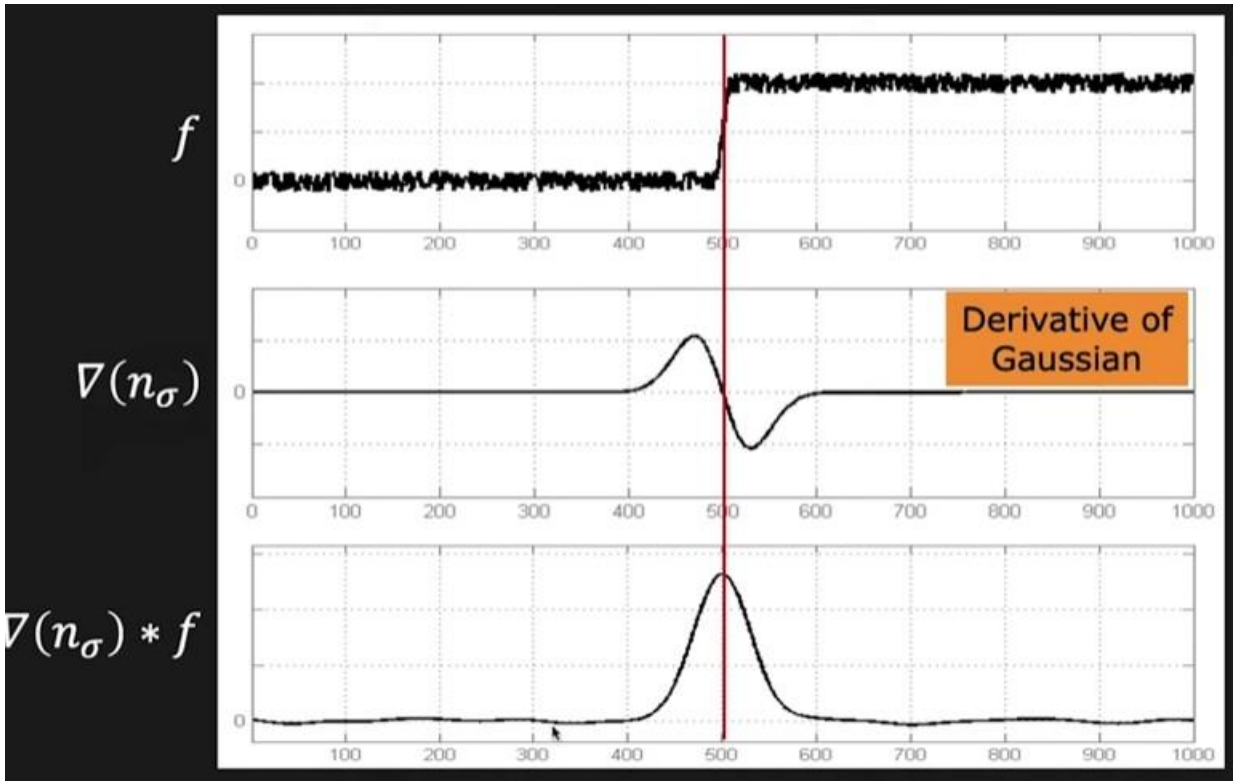
Multi-resolution is the process of looking at the same data (an image or signal) at different levels of detail or "zoom."

It is based on the idea that an image contains different types of information at different scales.

For example, the "big picture" of a face is a low-resolution feature, while the individual "pores" or "eyelashes" are high-resolution features.

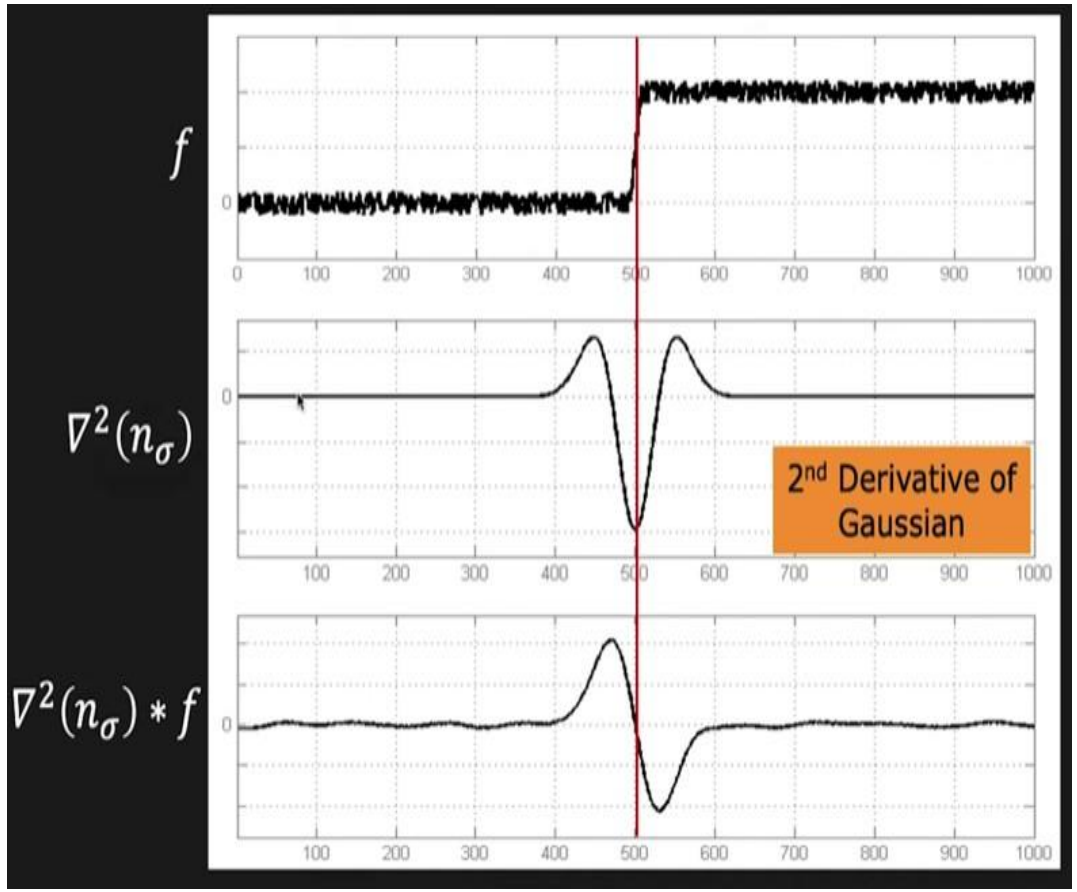
How to find blob in an image ?

Review – First to second order derivative for the edge detection.



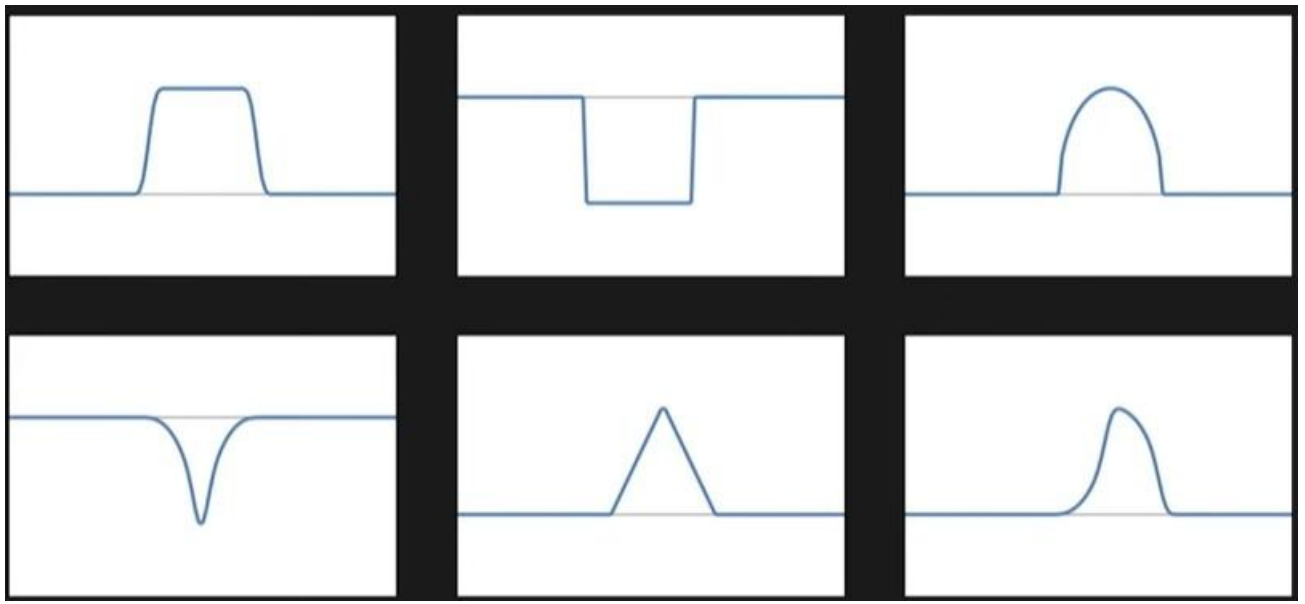
Here, the original signal (f) contains some noise, if we convolve the original signal with the derivative of gaussian then at the output we will get clean and clear peak which helps to detect the location on an edge.

SIFT: Interest Point Detection



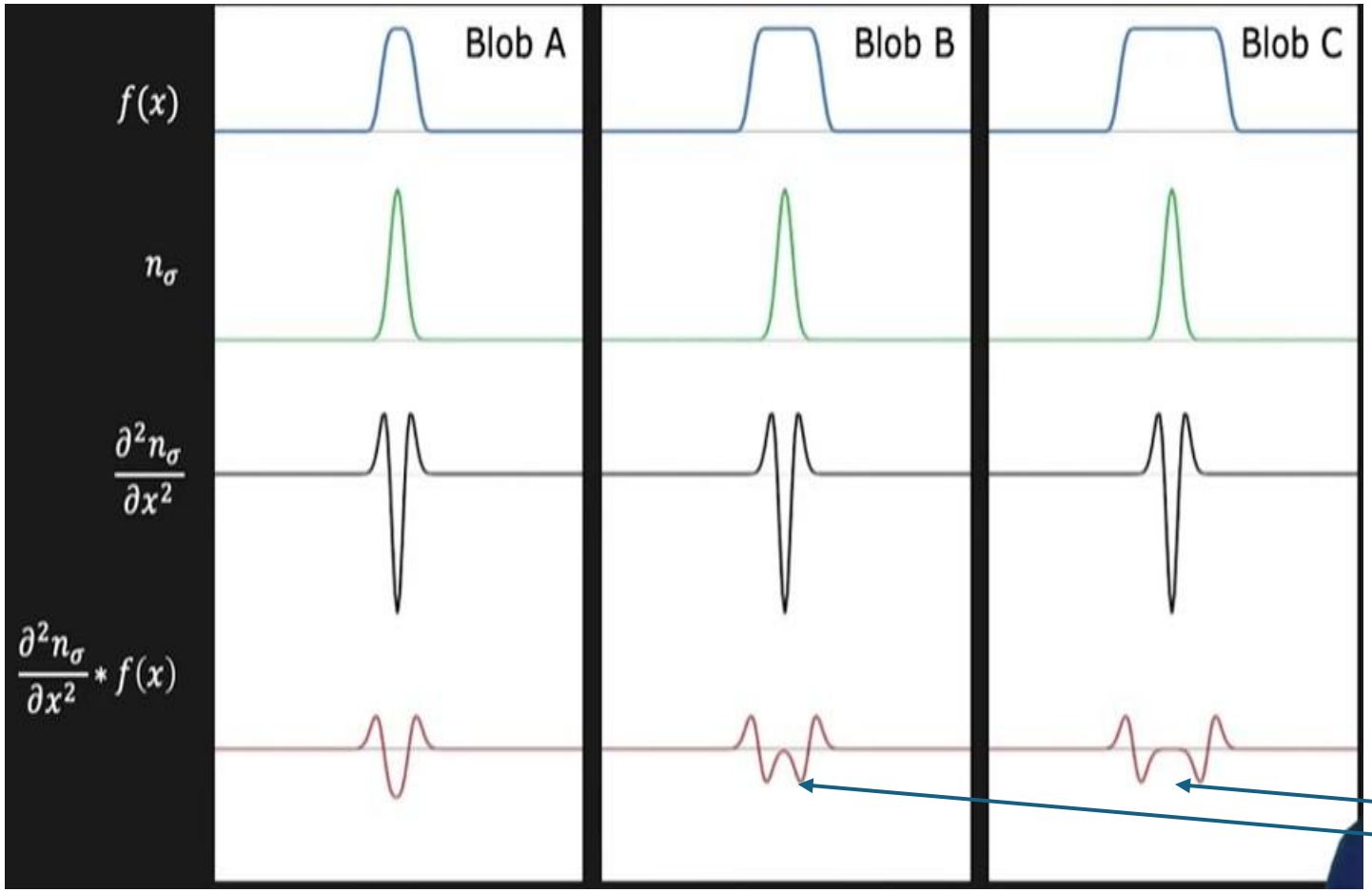
Here, the original signal (f) contains some noise, if we convolve the original signal with the Second derivative of gaussian then at the output we will get zero crossing which helps to find the better localization of edge.

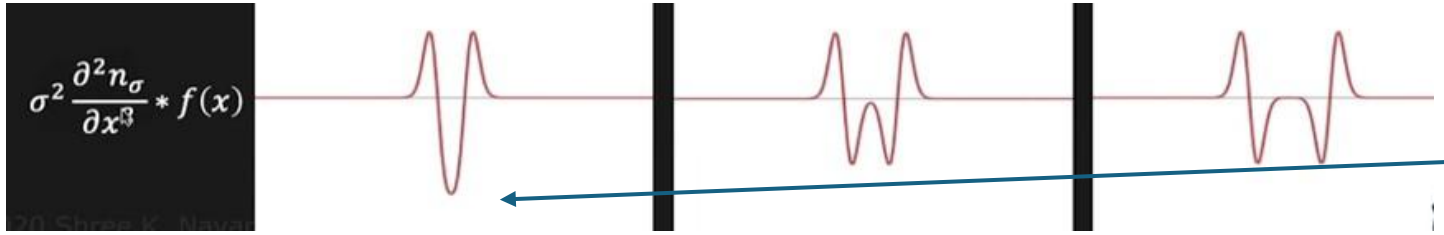
1-D Blob



Here we have 1-D blob of different shapes and size and we are going to find out all of them in image.

SIFT: Interest Point Detection





Here, Blob C is triple in size as compared to blob A.

Sigma

Second derivative of sigma

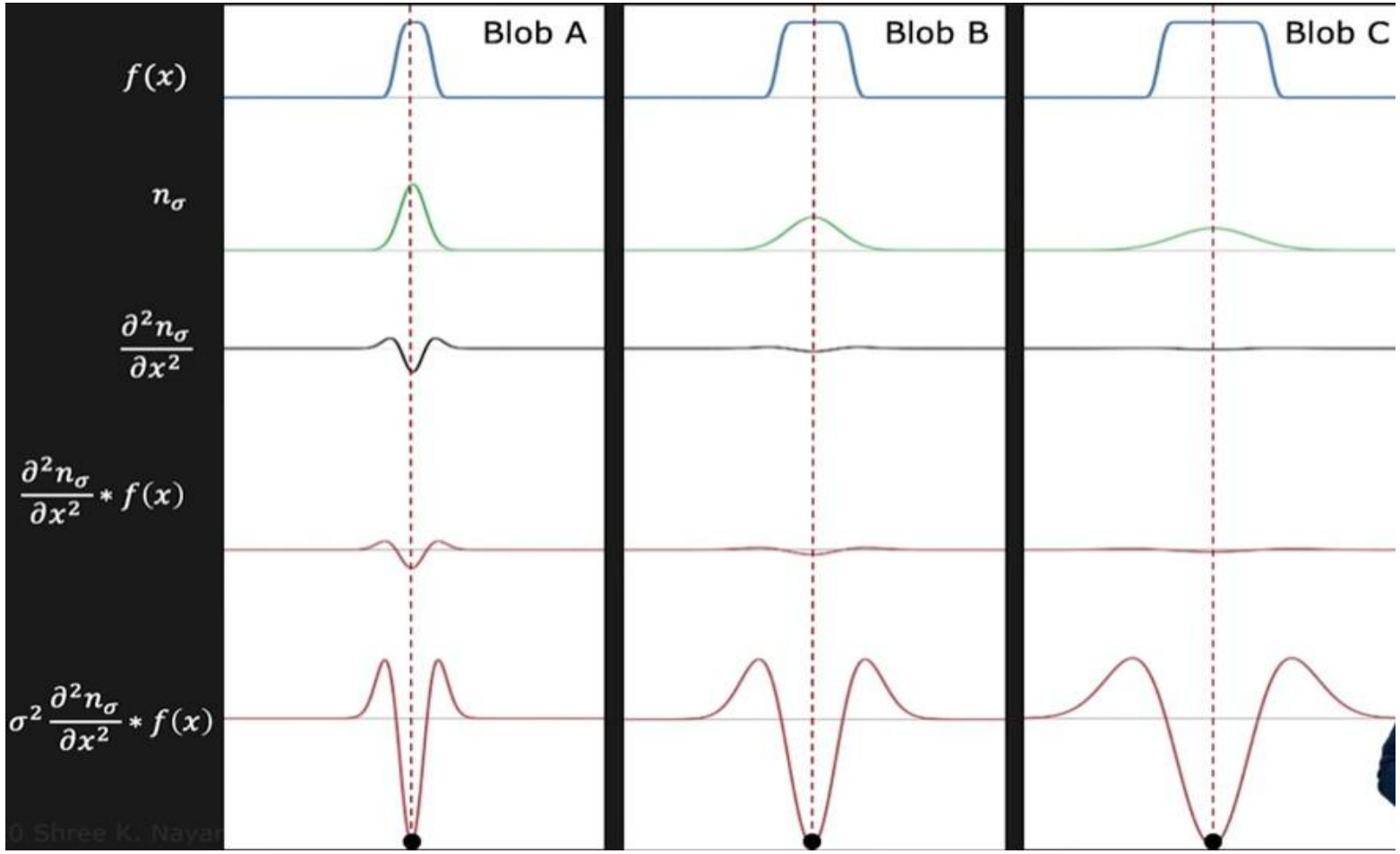
Second derivative of sigma applied on the image.

Here, Zero crossing response are begin to overlap

Perform sigma normalization. It only impact the Amplitude of it.

Overall goal - what is the impact of the sigma value to detect the different blobs at different resolution.

SIFT: Interest Point Detection



At different value of sigma, you will get different blob positions.

Here size of gaussian is changing and size of the gaussian is proportional to the width of the Blobs.

What exactly we did over here -
Applied second derivative of the gaussian.

But it actually applied for the large number of sigma. (sigma is refer to as scale.)

When you do it then irrespective of the size of the blobs, sooner or later you are going to produce the maxima at the location of the blobs.

SIFT: Interest Point Detection

2D Blob Detector

Normalized Laplacian of Gaussian (NLoG) is used as the 2D equivalent for Blob Detection.

Laplacian

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

Gaussian

n_σ

LoG

$\nabla^2 n_\sigma$

NLoG

$\sigma^2 \nabla^2 n_\sigma$

Location of Blobs given by Local Extrema after applying Normalized Laplacian of Gaussian at many scales.

Scale - Space

$S(x, y, \sigma_0)$ $S(x, y, \sigma_1)$ $S(x, y, \sigma_2)$ $S(x, y, \sigma_3)$

Increasing σ, Higher Scale, Lower Resolution

Scale Space: Stack created by filtering an image with Gaussians of different sigma (σ)

$$S(x, y, \sigma) = n(x, y, \sigma) * I(x, y)$$

SIFT: Interest Point Detection

What you have to do for finding the blobs in the images

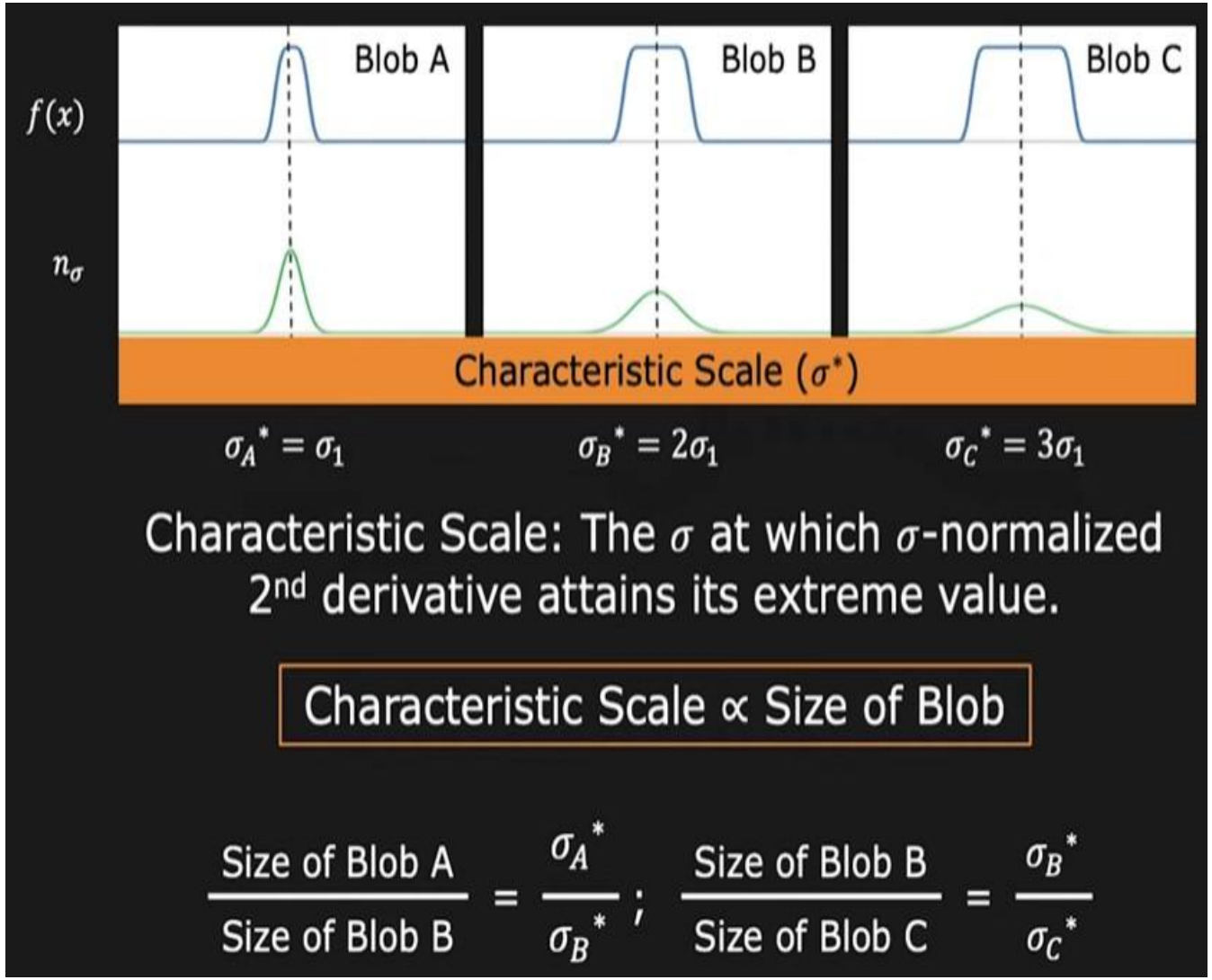
You have stack of images that corresponds to the different scales

Output of second derivative of gaussian applied to the image at multiple sigma values and it is defined with two parameters

Spatial Coordinates (x)

Sigma

You are going to find out the local extrema in the x-sigma space.



SIFT: Interest Point Detection

Blob detection summary

Given an image I(x,y)

↓

Convolve I(x,y) with NLoG operator at many different scales.

↓

Variation in different scales can be achieved by varying the sigma value.

At the end, you will end up with stack of images.

↓

Inside the stack of image find the extrema.

↓

Each extrema is the position of the blobs and sigma at that point is corresponds to the size of blobs.

2D Blob Detection Summary

Given an image $I(x,y)$

Convolve the image using NLoG at many scales σ

Find:

$$(x^*, y^*, \sigma^*) = \arg\max_{(x,y,\sigma)} |\sigma^2 \nabla^2 n_\sigma * I(x,y)|$$

(x^*, y^*): Position of the blob

σ^*: Size of the blob

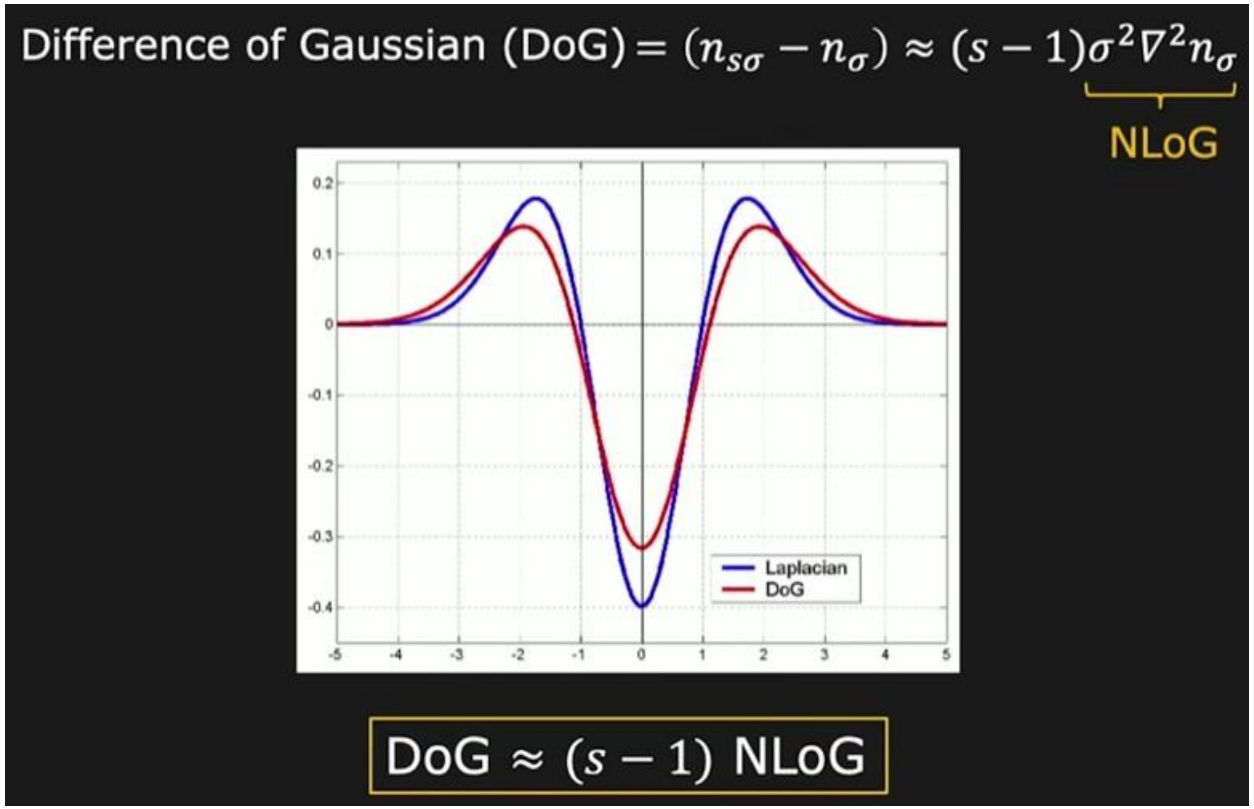
SIFT: Interest Point Detection

Now, we have all the tools to develop the SIFT detector.

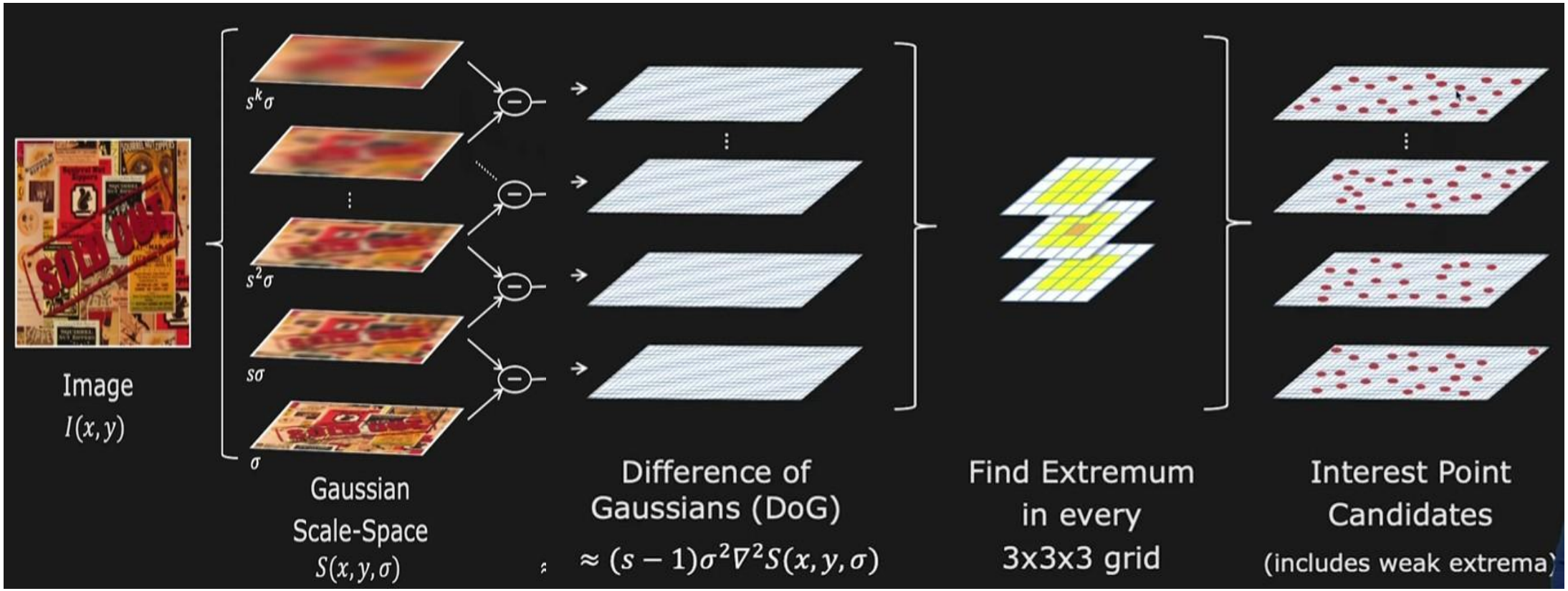
Here, Difference of gaussian is the approximation of NLOG operator.

Rather than taking LoG, one can simply take your stack of images (which is nothing but the original image smooth With different sigma) and choose 2 consecutive images And find the difference between them.

By doing so, difference of gaussian is approximately equal to the NLOG.



SIFT: Implementation



Create stack of images.

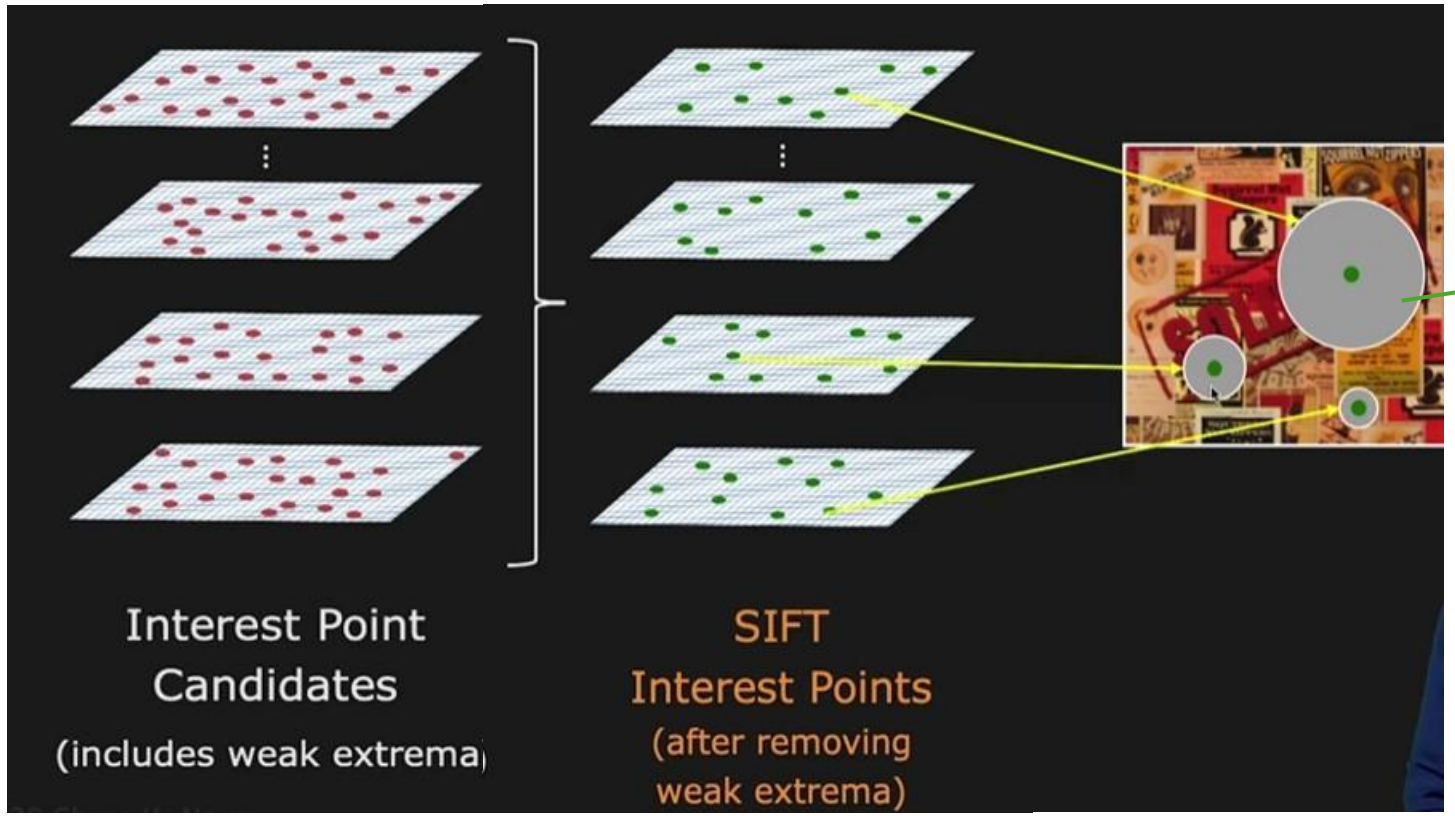
Perform Difference of Gaussian instead of NLOG.

These are the interest point or features

SIFT: Implementation

Apply some threshold and remove the weak extrema.

How does it look like in terms of images..



Here, position of blob is the position of interest point in image but the size of the blob corresponds to the scale.

Here, it is at higher scale so that diameter of blob is more.

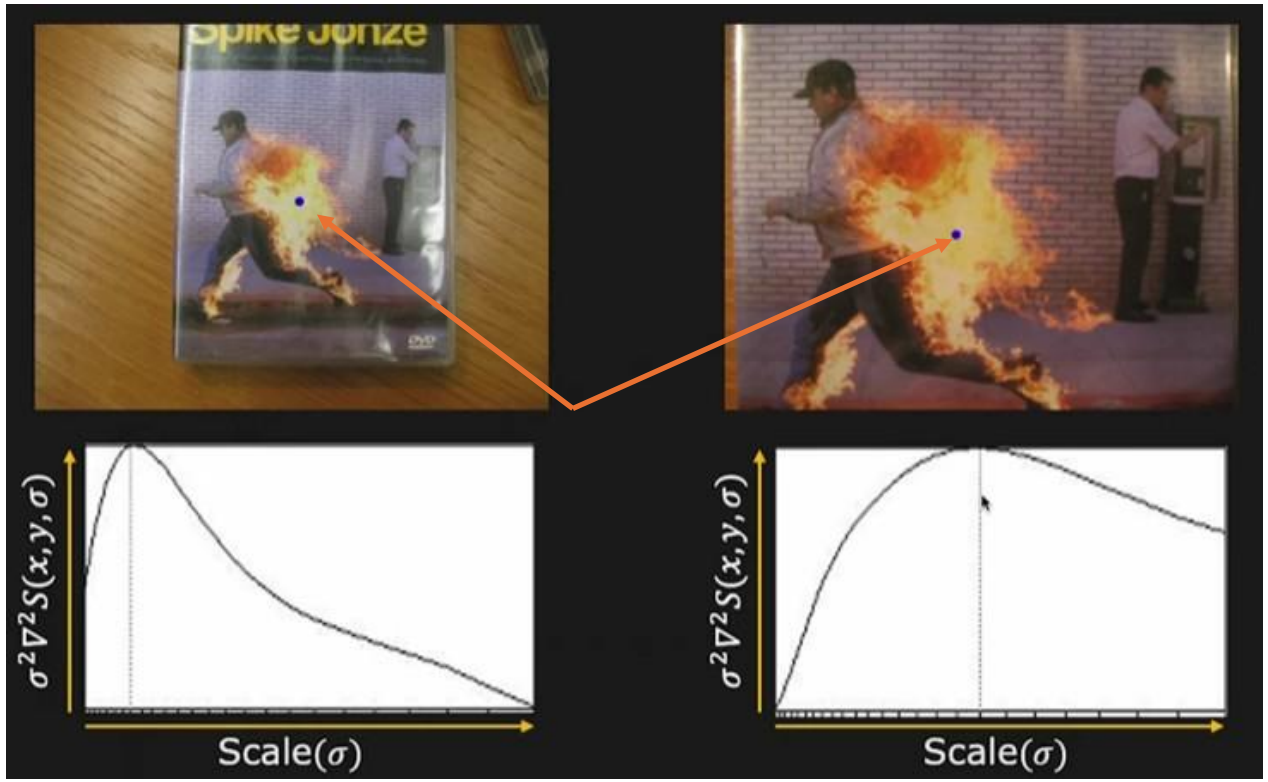
Different interest points or blobs, present inside the image.

Now we know how to find the interest points -
we know the locations,
we know the scales.
So, lets talk about **scale invariance,** because when you match these things ultimately, you want to remove the scale.

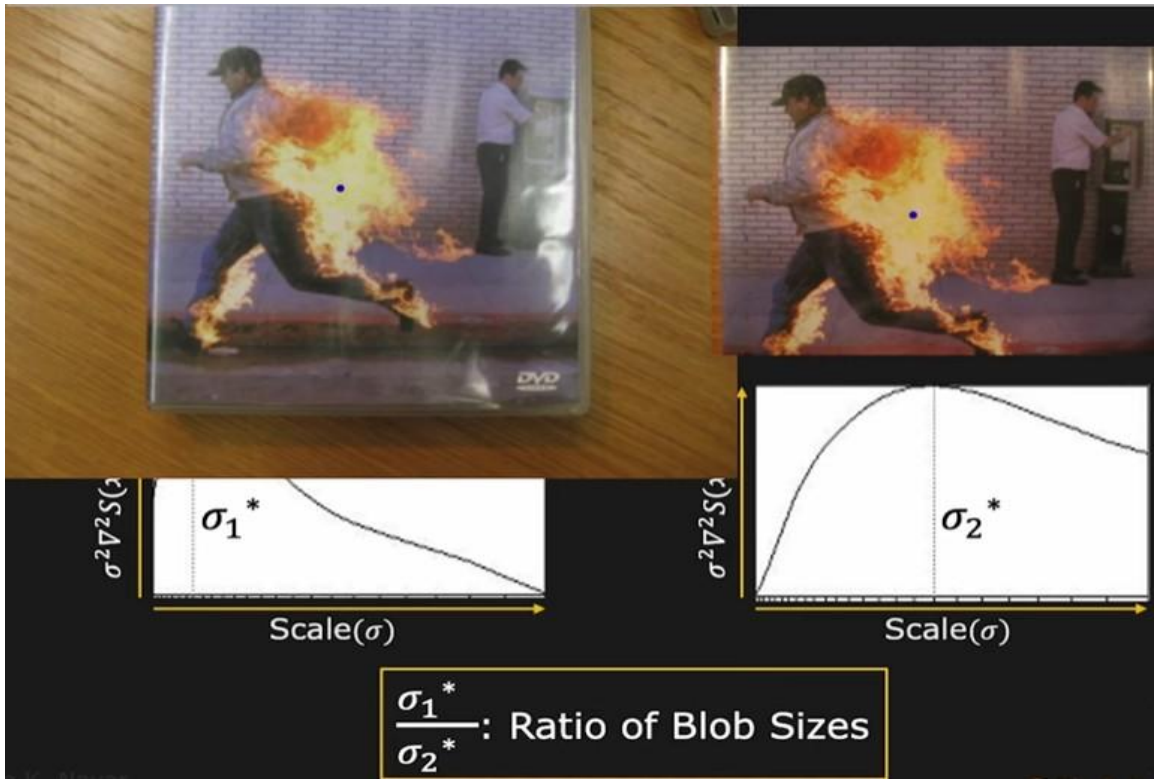
SIFT: Scale Invariance

First image

Closer view of the First image



Peak or extrema position is different in both the cases due to the difference in the scale.



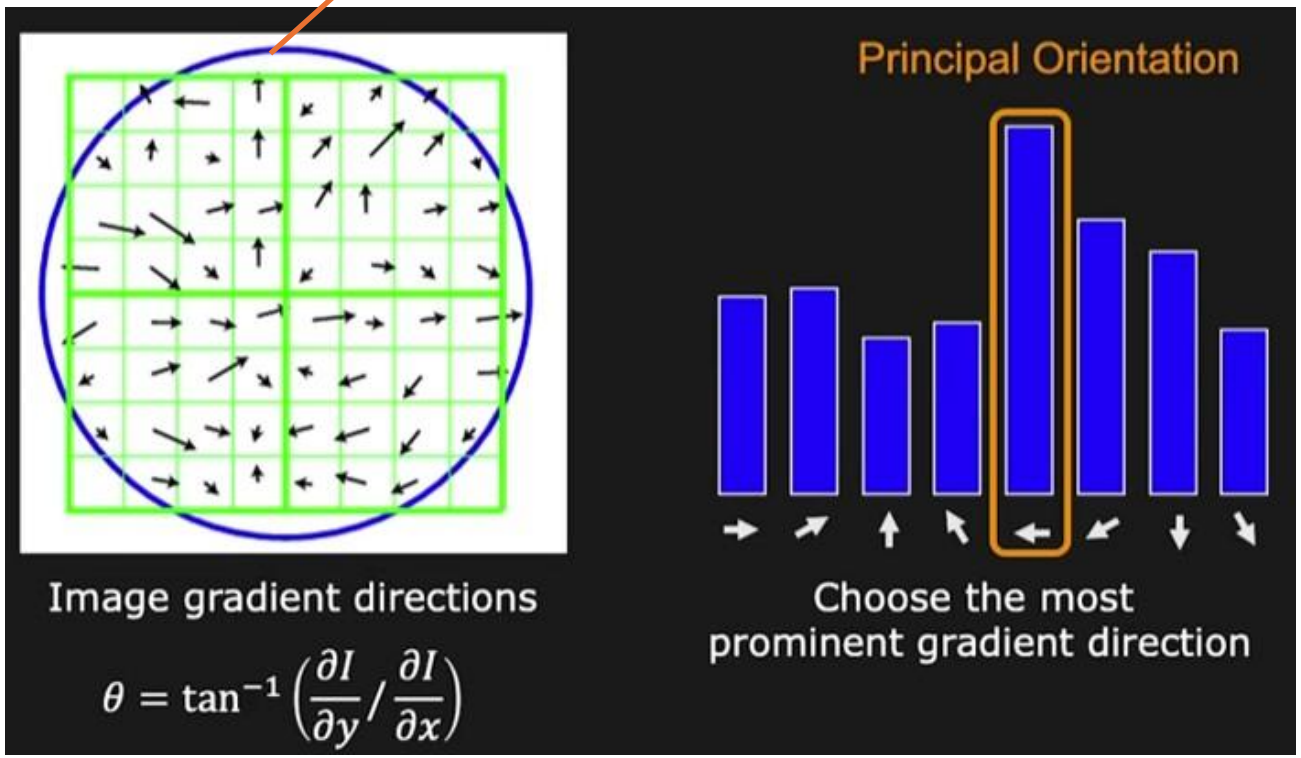
As we know the sigma then we can normalize it. Normalization means, we can adjust the scale of one From another image.

NOTE: Scale Invariance: you can remove the effect of scales, ones you know the interest points.

SIFT:

Rotation Invariance: Remove the effect of the orientation

Size of the blob



A grid is placed on the blob.

For Each pixel in the grid find the gradient.

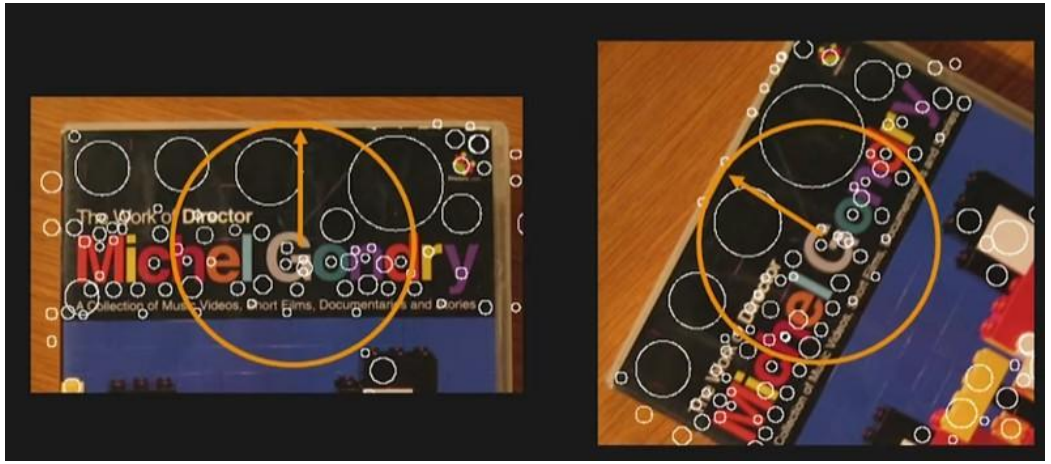
Here, we are completely ignoring the magnitude.

We are only considering the direction of each pixel.

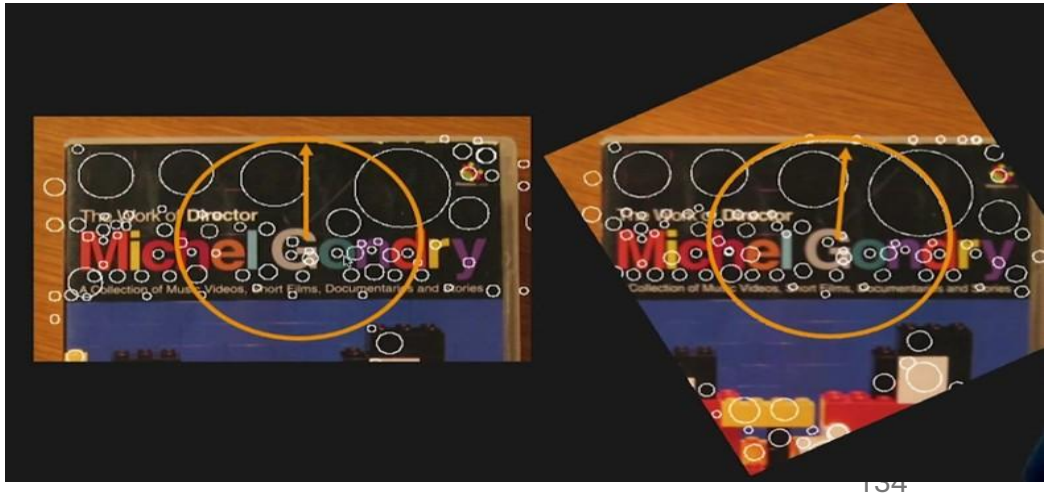
Plot the histogram of Gradient direction.

Here, maximum value Helps to identify the principle Orientation of your feature.

Lets say two images with different orientation.



Find the principle orientation of feature and rotate It according to the reference image.

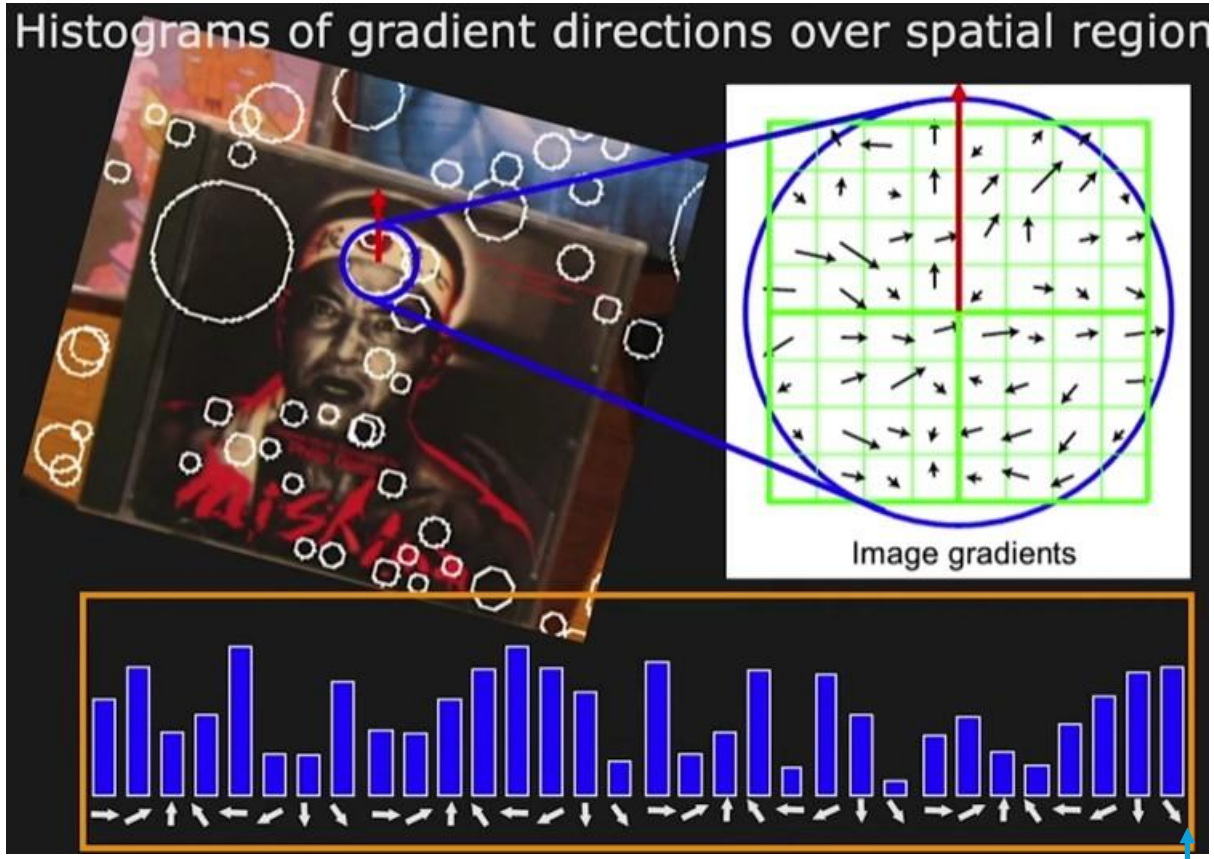


SIFT: Interest Point Detection

Comparing SIFT Descriptor

SIFT Descriptor: we are looking for one kind of signature that you can compute from the SIFT features.

It will help to match one feature to other feature .



I quadrant II quadrant III quadrant IV quadrant

Histogram of different quadrant.

This signature can be utilized to match the feature

Essentially comparing two arrays of data.

Let $H_1(k)$ and $H_2(k)$ be two arrays of data of length N.

L2 Distance:

$$d(H_1, H_2) = \sqrt{\sum_k (H_1(k) - H_2(k))^2}$$

Smaller the distance metric, better the match.

Perfect match when $d(H_1, H_2) = 0$

Normalized Correlation:

$$d(H_1, H_2) = \frac{\sum_k [(H_1(k) - \bar{H}_1)(H_2(k) - \bar{H}_2)]}{\sqrt{\sum_k (H_1(k) - \bar{H}_1)^2} \sqrt{\sum_k (H_2(k) - \bar{H}_2)^2}}$$

where: $\bar{H}_i = \frac{1}{N} \sum_{k=1}^{N} H_i(k)$

Larger the distance metric, better the match.

Perfect match when $d(H_1, H_2) = 1$

Intersection:

$$d(H_1, H_2) = \sum_k \min(H_1(k), H_2(k))$$

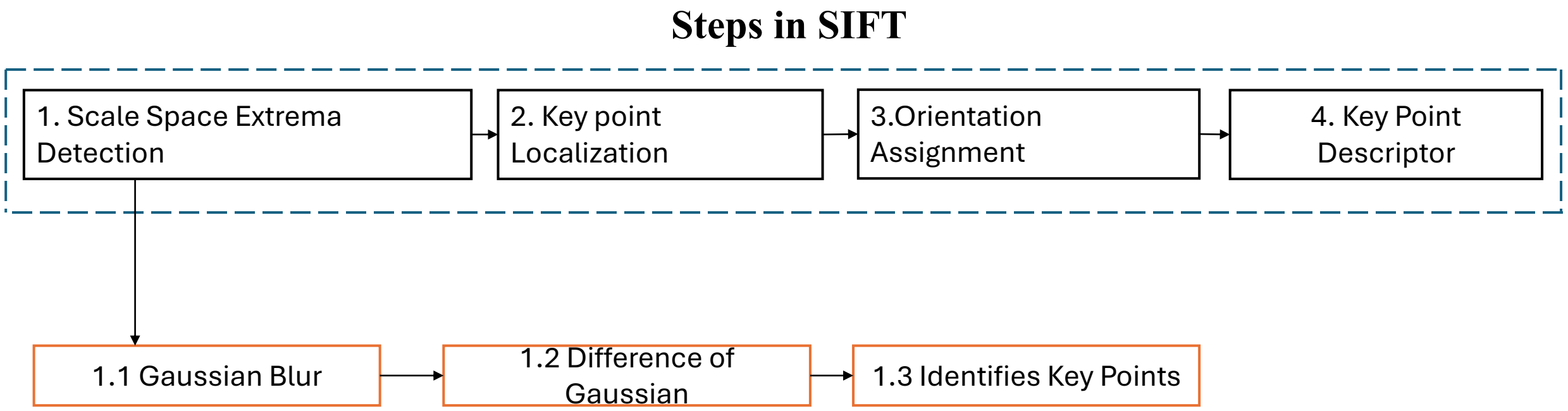
Larger the distance metric, better the match.

SIFT : Overall Flow Diagram

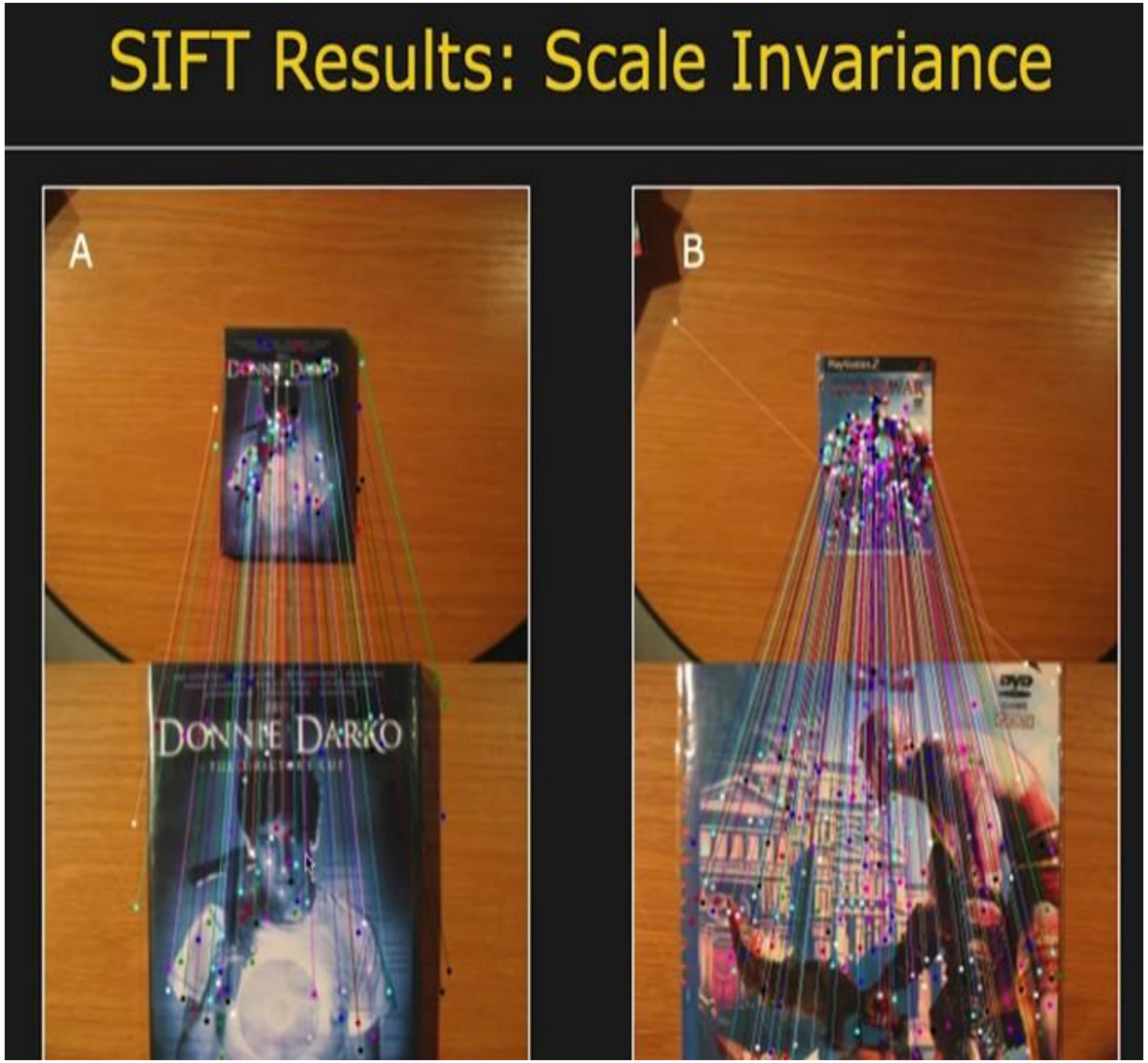
SIFT: It is a robust algorithm designed **to identify and describe local features** in an images that is **invariant to scale, rotation and illumination changes.**

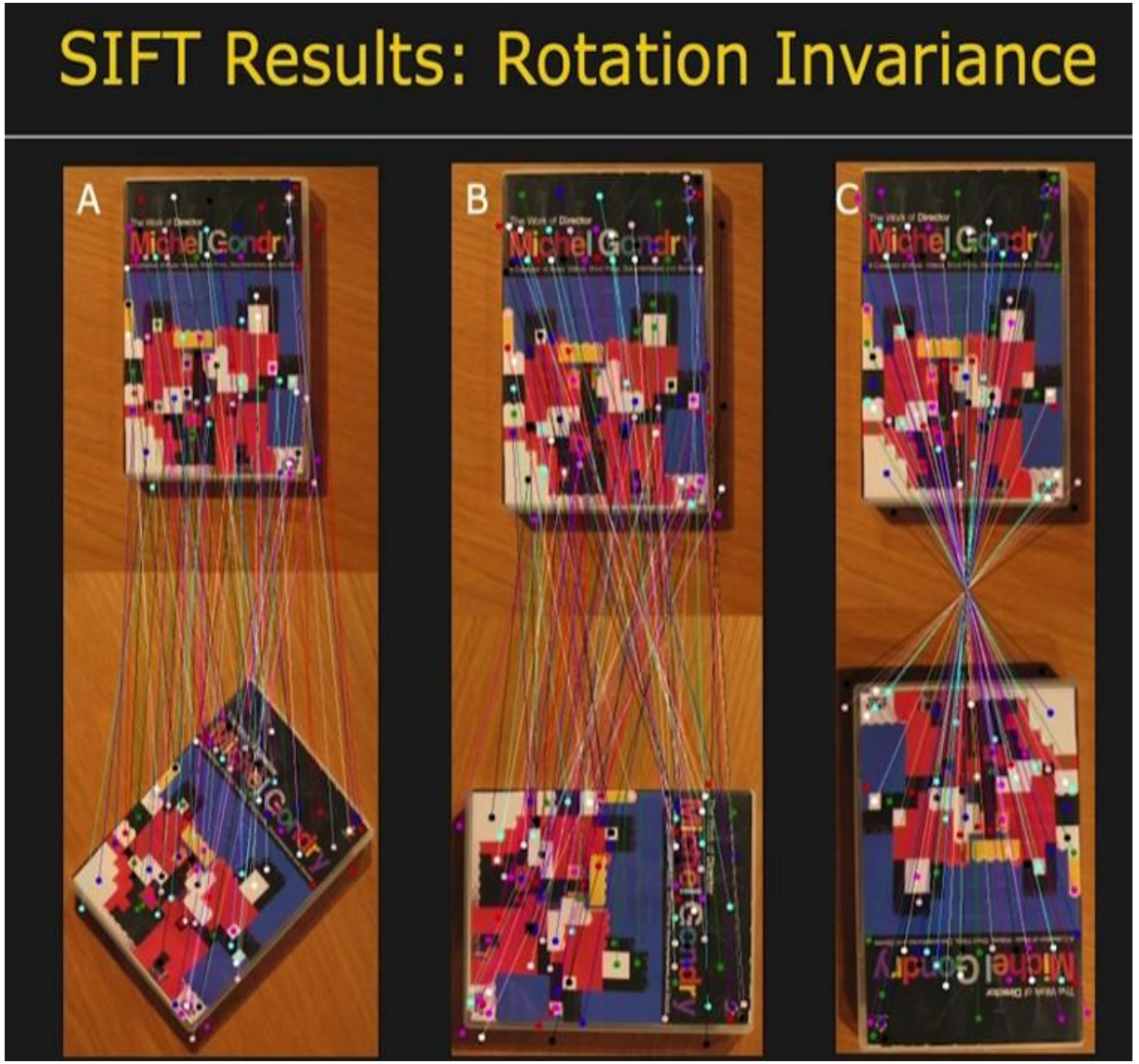
or

SIFT can detect the same feature in an image even if the image is **resized**, **rotated** and **viewed under different lightening conditions**.



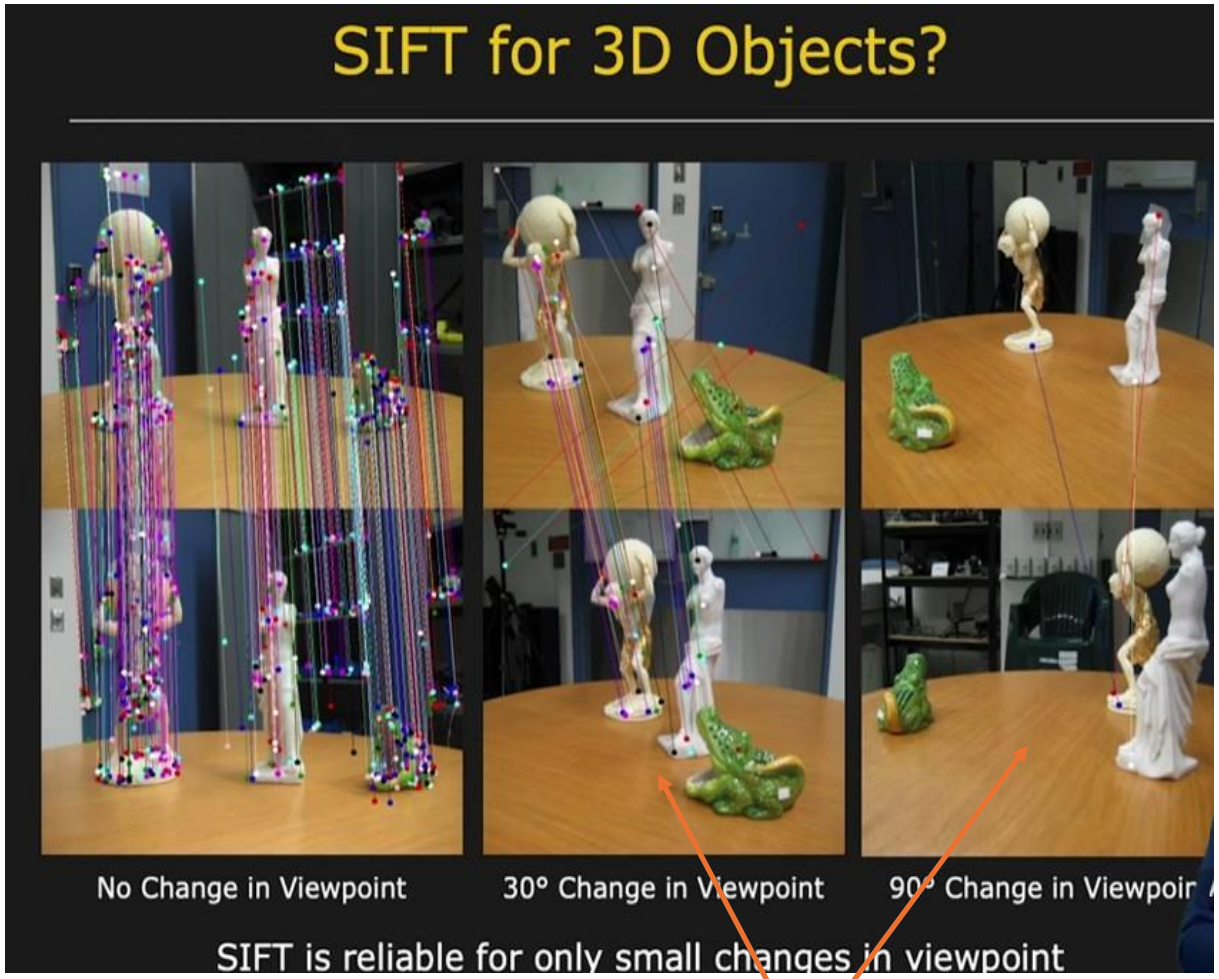
SIFT: Application





SIFT: Application

If object is partially occluded, then also SIFT can detect it.



If viewpoint is not change then it matches most of the interest point or key point.

Slight change in the viewpoint is directly proportional to decrement in the feature matching.